DevOps

Développement et Opération

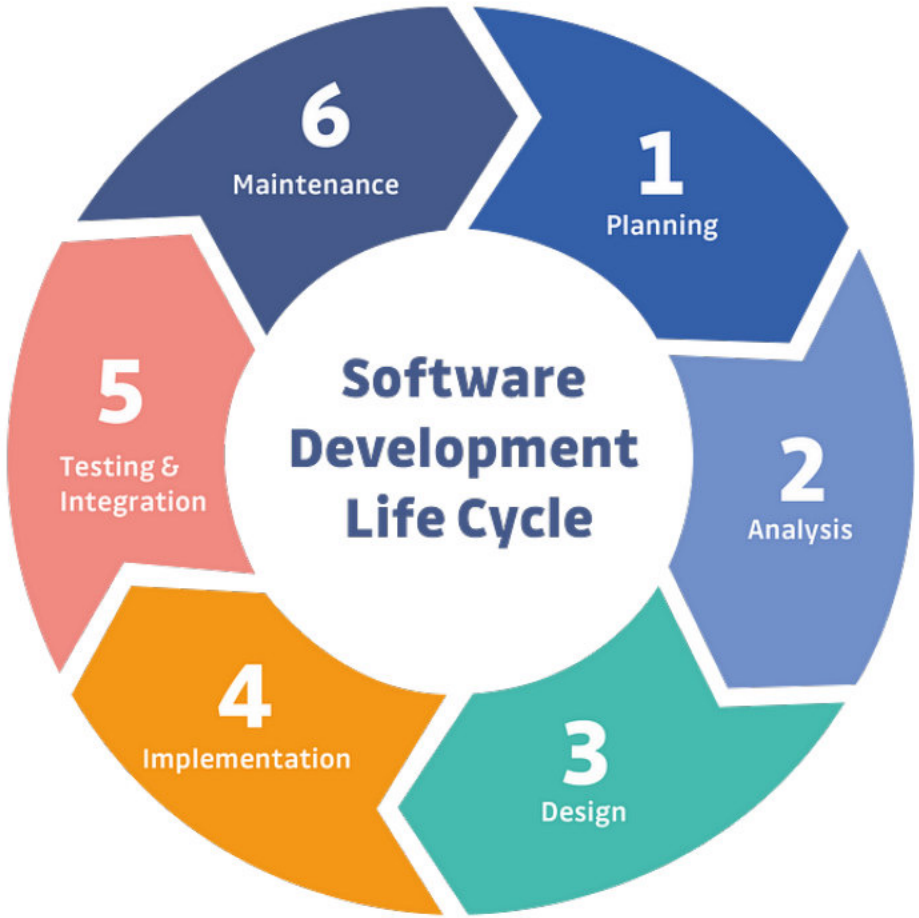
A.U. : 2026-2027

PLAN

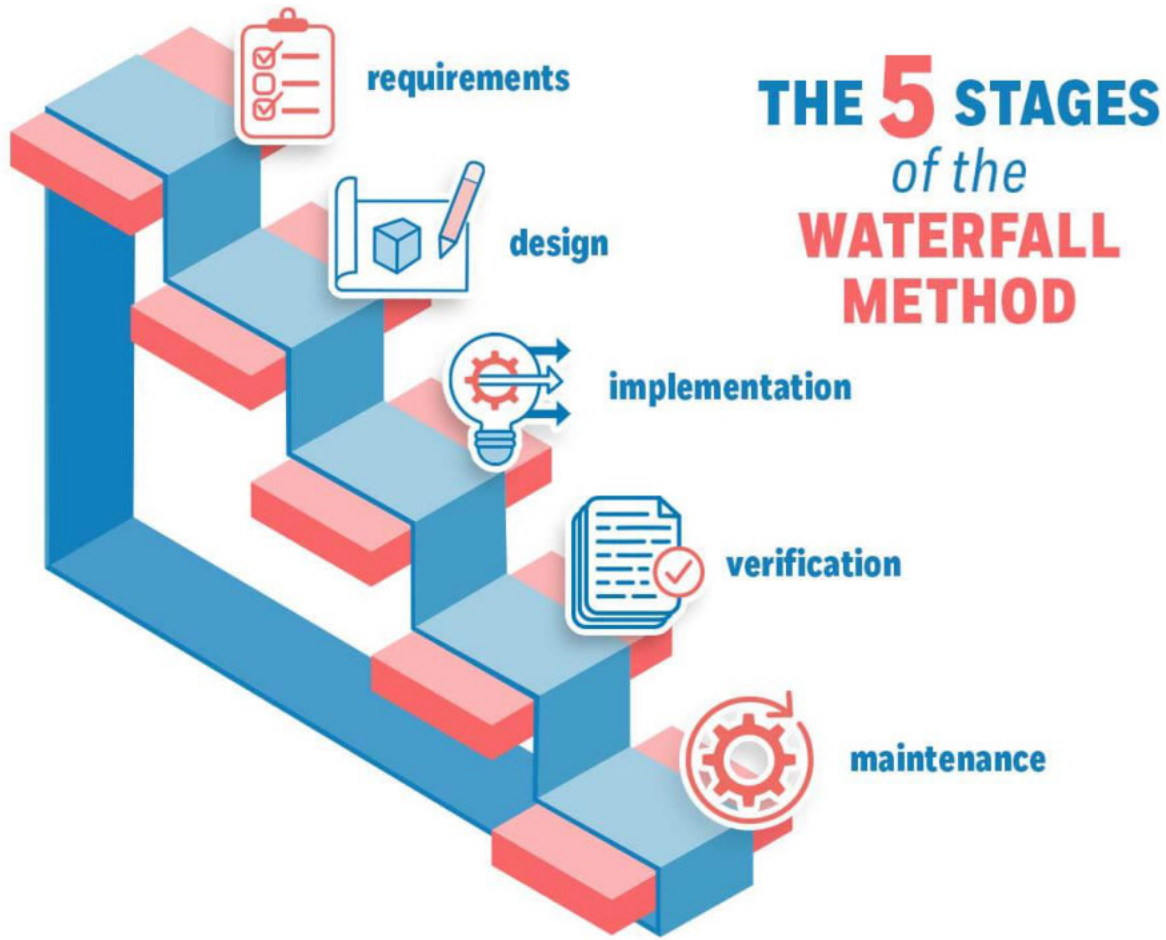
1. Introduction au concept DevOps
   - SDLC, agilité, intégration et livraison continues, piliers CALMS
   - Panorama des outils DevOps
2. Contrôle de code source : Git
   - Bases, dépôts distants, branches, merge, conflits, rebase, cherry-pick, stash
   - Annuler, tags, stratégies de branches, Pull / Merge Request
3. Planification agile avec Jira (Scrum et Kanban)
4. Conteneurisation : Docker et Docker Compose
5. Orchestration : Kubernetes
6. CI/CD et pipelines (GitHub Actions)
7. Pour aller plus loin : Infrastructure as Code, supervision, DevSecOps

Introduction à DevOps

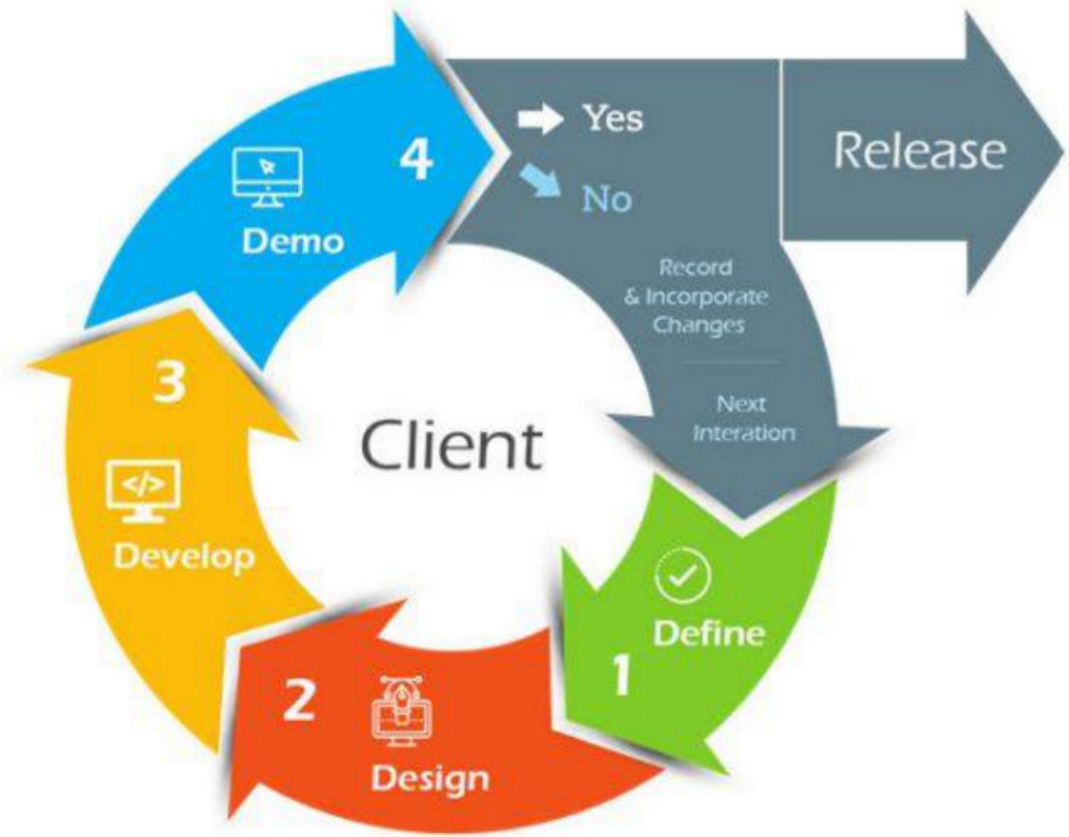
SDLC : Software Development Lice Cycle



WATERFULL : MODÈLE EN CASCADE



AGILE


Yes
No
Release
4
Demo
Record
& Incorporate
Changes
Next
Interation
3
Develop
Client
Define
1
2
Design

Le terme génie logiciel désigne l'ensemble des **méthodes***, des **techniques*** et **outils*** concourant à la production d'un logiciel, au delà de la seule activité de programmation.

Ensemble des méthodes, des techniques et outils * : Le génie logiciel ne se limite pas à une seule approche ou méthode. Il englobe un large éventail de méthodes, de techniques et d'outils utilisés pour développer un logiciel de manière efficace et de haute qualité. Cela comprend la planification, la conception, la construction, les tests, la maintenance et la gestion de projet.

LES DÉFIS

La taille des projets: pour certains, des millions de ligne de code (MLOC)

Contribution à des projets existants

- Peu de projets démarrés from scratch
- Plus de valeur d'ajouter 100 LOCs à un grand projet largement utilisé que d'écrire 10000 LOCs dans son coin. *Exemples de ces projets : systèmes d'exploitation, bases de données massives, ERP, plateformes de médias sociaux, etc.*
- La taille des projets impacte divers aspects : complexité du développement, ressources requises, planification, gestion du code et des versions, performances et maintenance.
- Gérer efficacement de grands projets exige une planification minutieuse, une coordination d'équipe efficace, ainsi que des outils et des processus appropriés.

LES DÉFIS

Réutilisation de code existant

Est ce que le concepteur d'une voiture commence par réinventer la roue?

- Pratique courante et bénéfique dans le développement logiciel.
- Évite de réinventer la roue à chaque projet.
- Recherche et utilisation de bibliothèques, frameworks et modules existants.
- Permet d'économiser du temps et des ressources.
- Favorise la cohérence, la fiabilité et la qualité du logiciel.
- Utilisation de composants éprouvés et souvent maintenus par une communauté plus large.

LES DÉFIS

Collaboration avec d'autres développeurs

Comment interagir?

- Utilisation d'outils de gestion de versions comme Git.
- Plateformes de gestion de projets telles que GitHub ou GitLab.
- Communication via des forums de discussion et des réunions virtuelles.

Comment fournir du code réutilisable?

- Assurer une documentation claire et complète du code.
- Adopter une conception modulaire et bien encapsulée.
- Publier le code sur des plateformes accessibles aux autres développeurs.
- Encourager une communication ouverte sur les normes de codage et les bonnes pratiques.

LES DÉFIS

Utilisateurs/Clients :

- Compréhension des besoins et des attentes des utilisateurs.
- Collecte de retours d'expérience et de feedbacks.
- Validation des fonctionnalités du logiciel en collaboration avec les utilisateurs.
- Développement de produits offrant une valeur réelle et une expérience utilisateur positive.

LES ENJEUX

L'industrie du logiciel travaille surtout sur des projets existants : les projets partant de zéro (from scratch) sont minoritaires.

- **Réutiliser :** utiliser des composants existants pour éviter de recréer des fonctionnalités.
- **Faire évoluer :** ajouter de nouvelles fonctionnalités ou améliorer les existantes.
- **Étendre :** augmenter les capacités ou la portée du système (nouveaux cas d'usage).
- **Adapter :** faire fonctionner le logiciel dans de nouveaux environnements ou technologies.
- **Maintenir :** corriger les défauts et assurer un support continu.
- **Réorganiser :** restructurer le code pour améliorer sa qualité et sa maintenabilité.

DEVOPS : DÉFINITION

- **Définition simple :** ensemble de pratiques et d'outils qui facilitent le passage du développement à la production.
- **Plus qu'un outil :** une culture qui rapproche les équipes **Dev** (livrer vite) et **Ops** (garder le système stable) autour d'objectifs communs.
- **Ce que cela change :**
  - un modèle de fonctionnement de l'entreprise ;
  - un modèle d'interactions entre les équipes ;
  - l'intégration du retour d'expérience de la production vers le développement ;
  - l'automatisation de tout ce qui est répétitif.

> Sans changement de culture, les outils ne font qu'automatiser le désordre.

DEVOPS

Relation entre Dev et Ops

Dev: Equipes de développeurs logiciels
Ops: Equipes en charge de la mise en production des produits

Antagonisme fort

Dev: Modifications aux moindres coûts, le plus rapidement possible
Ops: Stabilité du système, qualité

L'automatisation est au cœur de l'approche DevOps !

Le DevOps est un **ensemble de pratiques** qui met l'emphase sur **l'automatisation des processus** entre les équipes de développement, et les équipes en charge du maintien en conditions opérationnelles de l'application développée.

- gain de confiance des équipes entre elles ;
- accélération des livraisons et des déploiements ;
- résolution des tickets plus rapide ;
- gestion plus efficace des tâches non planifiées…

Méthodes agiles

INTÉGRATION, LIVRAISON ET DÉPLOIEMENT CONTINUS

Intégration continue (CI)

- Le code est intégré souvent dans la branche principale
- Build et tests automatiques à chaque modification
- Erreurs et conflits détectés très vite

Livraison continue (CD)

- Chaque version validée est prête à être déployée
- La mise en production est déclenchée **manuellement**
- Livraisons fiables et sans stress

Déploiement continu (CD)

- Tout changement qui passe les tests part en production **automatiquement**
- Aucune validation manuelle
- Exige tests fiables, supervision, retour arrière rapide

CI : jusqu'aux tests **Livraison : validation manuelle** **Déploiement : automatique**

L'intégration continue

L'intégration continue se réfère à plusieurs pratiques :

Construire une version fonctionnelle chaque jour :

Garantit la disponibilité d'une version testable du logiciel quotidiennement.

Assure une vision claire de l'état du prototype.

Exécuter les tests quotidiennement :

Permet de détecter rapidement les erreurs et les problèmes de compatibilité.

Réduit les risques d'introduire des défauts majeurs.

Commiter les changements quotidiennement :

Favorise une approche incrémentale du développement.

Facilite la collaboration et la gestion des versions.

Système d'observation des changements :

Automatise la vérification des modifications dans le dépôt.

Réalise des actions en réponse aux changements détectés.

La livraison continue

Les étapes principales d'une procédure de livraison continue sont:

1. **Commit d'une modification**
2. **Exécution des tests unitaires (Intégration continue)**
3. **Tests de validation fonctionnelle (acceptance test)**
   - Tests black-box
   - Testent si le logiciel répond bien au besoin du client
   - Peuvent aussi être automatisés
4. **Tests de performances**
   - Testent si le logiciel peut répondre à la charge
   - Performance et passage à l'échelle
5. **Tests exploratoires**

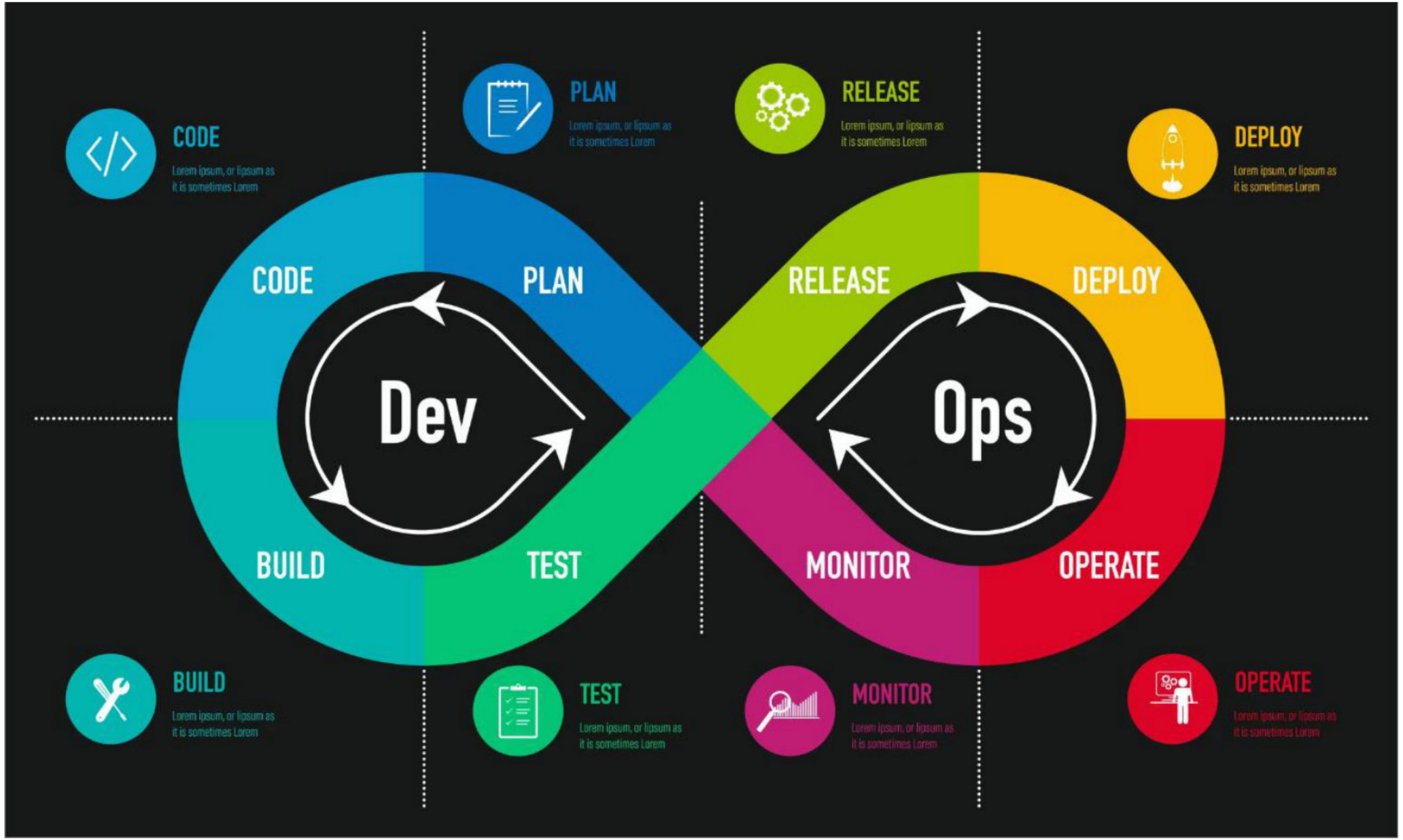
Tests non-automatisés effectués par des experts

6. **Le déploiement en production**

LES PILIERS DE DEVOPS : CALMS

Culture	Automatisation	Lean	Mesure	Partage
• Collaboration Dev, Ops et métier • Responsabilité partagée • Droit à l'erreur	• Build, tests, déploiement • Infrastructure en code • Moins d'erreurs, moins de délais	• Petits lots • Flux continu • Éliminer les gaspillages	• Délais, échecs, disponibilité • Détecter les incidents tôt • Décider avec des données	• Connaissances et outils • Une cible commune • You build it, you run it

DevOps = Culture + Automatisation + Lean + Mesure + Partage


CODE
Lorem ipsum, or lipsum as it is sometimes Lorem
PLAN
Lorem ipsum, or lipsum as it is sometimes Lorem
RELEASE
Lorem ipsum, or lipsum as it is sometimes Lorem
DEPLOY
Lorem ipsum, or lipsum as it is sometimes Lorem
CODE
PLAN
RELEASE
DEPLOY
Dev
Ops
BUILD
TEST
MONITOR
OPERATE
BUILD
Lorem ipsum, or lipsum as it is sometimes Lorem
TEST
Lorem ipsum, or lipsum as it is sometimes Lorem
MONITOR
Lorem ipsum, or lipsum as it is sometimes Lorem
OPERATE
Lorem ipsum, or lipsum as it is sometimes Lorem

Avantages à adopter l'approche DevOps

- Accélération et amélioration de la fourniture des produits
- Résolution plus rapide des problèmes et complexité réduite
- Stabilité accrue des environnements d'exploitation
- Meilleure utilisation des ressources
- Automatisation accrue
- Meilleure visibilité sur les résultats du système
- Innovation renforcée

Panorama des outils DevOps

Une chaîne d'outils pour chaque étape du cycle

LA CHAÎNE D'OUTILS DEVOPS

Étape	Objectif	Outils courants
Planifier	organiser le travail de l'équipe	Jira, Trello, GitHub Projects
Coder	versionner et collaborer	Git, GitHub, GitLab, Bitbucket
Construire et tester	compiler, vérifier automatiquement	Maven, npm, pytest, JUnit, Selenium
Intégrer et livrer	automatiser avec des pipelines	GitHub Actions, GitLab CI/CD, Jenkins
Packager	empaqueter l'application	Docker, registres (Docker Hub, GHCR)
Déployer et orchestrer	exécuter à grande échelle	Kubernetes, Helm, Argo CD
Infrastructure	décrire l'infrastructure en code	Terraform, Ansible
Superviser	métriques, logs, alertes	Prometheus, Grafana, ELK, Loki

LES FORGES LOGICIELLES

Plateforme	Demande de fusion	CI/CD intégré	Particularité
GitHub	Pull Request	GitHub Actions	plus grande communauté, open source
GitLab	Merge Request	GitLab CI/CD	tout-en-un, auto-hébergeable
Bitbucket	Pull Request	Bitbucket Pipelines	intégration native avec Jira
Azure DevOps	Pull Request	Azure Pipelines	écosystème Microsoft
Gitea	Pull Request	Gitea Actions	léger, auto-hébergé (TP5)

> Pull Request et Merge Request : même concept, une demande de fusion relue par l'équipe.

LES OUTILS UTILISÉS DANS CE COURS

Code
Git + GitHub / GitLab

Planification
Jira

Conteneurs
Docker, Docker Compose

Orchestration
Kubernetes (Minikube)

CI/CD
GitHub Actions

> Les concepts se transposent d'un outil à l'autre : seule la syntaxe change.

Contrôleur de code source

COMMENT GÉREZ-VOUS ACTUELLEMENT UN PROJET ?

- L'envoyer par message (réseaux sociaux...) : très mauvaise idée
- L'envoyer par mail : un peu moins mauvaise
- Utiliser Dropbox, Google Drive... : déjà mieux, mais risqué et peu de fonctionnalités

Solution : utiliser un système de gestion de version distribué (Git)

geek & poke
YESTERDAY IT WORKED
YOU KNOW YOU'RE IN A SOFTWARE PROJECT

DÉFINITION

- Source contrôle est la pratique du suivi et de la gestion des modifications du code
- Les sources contrôles fournissent un historique de développement du code et aident à résoudre les conflits lors de la fusion de contributions provenant de plusieurs sources.
- Le contrôle des sources protège le code source des dégradations occasionnelles suite aux erreurs humaines.
- Les avantages incluent: la réutilisabilité, la traçabilité, portabilité, l'efficacité, la collaboration et l'apprentissage.

CE QUE PERMET DE FAIRE UN SYSTÈME DE CONTRÔLE DU CODE SOURCE

- Conserver tout le code source
- Prendre note de tous les changements effectués au code source et à sa documentation

  => permet de retourner à une version antérieure (qui, elle, fonctionnait ! !)
- Identifier quels fichiers ont été modifiés
- Déterminer qui a modifié un bout de code
- Comparer des versions
- Fusionner des versions développées de façon concurrente
- Identifier et gérer les releases*, les versions**, les branches de développement ***

Release* : des versions stables et officielles du logiciel, généralement destinées à être utilisées par les utilisateurs finaux

Versions** : Les versions correspondent à différentes itérations du logiciel, avec des fonctionnalités ajoutées, des bogues corrigés, etc.

Branches de développement ***: sont des versions du code source qui évoluent séparément les unes des autres.

Par exemple, il peut y avoir une branche de développement pour travailler sur de nouvelles fonctionnalités tandis qu'une autre branche est dédiée à la correction de bugs. Un système de contrôle de code source permet de gérer ces différentes entités de manière organisée, en les identifiant clairement et en facilitant leur gestion.

LES DIFFÉRENTS SCCS : SCCS CENTRALISÉ VS. SCCS DISTRIBUÉ

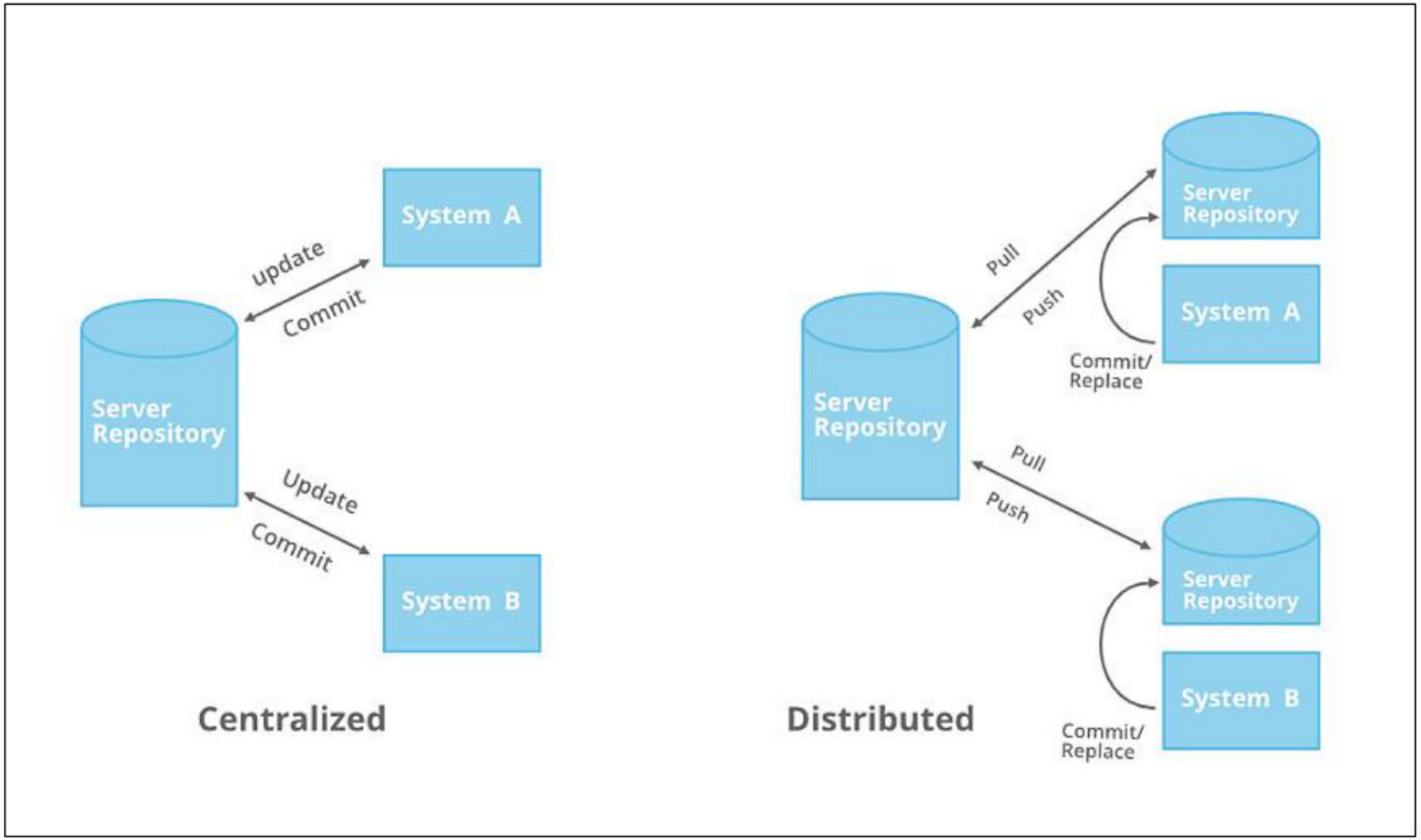
SCCS Centralisé

Tout l'historique des changements est conservé sur un serveur central (distant), duquel n'importe qui peut obtenir la version la plus récente ou envoyer les changements les plus récents.

SCCS distribué

Chaque usager a une copie locale de tout l'historique des changements. Il n'est donc pas nécessaire d'être connecté à un réseau/serveur pour sauvegarder des changements. En outre, n'importe quel usager peut se synchroniser avec n'importe quel autre.

Les différents SCCS : SCCS Centralisé vs. SCCS Distribué



LES AVANTAGES ET LES INCONVÉNIENTS DU SCCS DISTRIBUÉ

AVANATGES

- Chaque développeur a son espace privé — son sandbox
- On peut travailler — et faire des commits ! — hors ligne
- Plusieurs opérations sont très rapides — exécution locale
- La création et fusion de branches est efficace

INCONVÉNIENTS

- Quand même préférable d'avoir un dépôt central (sauvegarde)
- Pas nécessairement «une» version «la plus récente»

DÉMARRER AVEC GIT

- Git est un système de contrôle de version distribué gratuit qui permet aux programmeurs de suivre les modifications du code, via des "instantanés" (commits), dans leur état actuel.
- L'utilisation de validations permet aux programmeurs de tester, déboguer et créer de nouvelles fonctionnalités en collaboration. Tous les commits sont conservés dans ce que l'on appelle un « référentiel Git » pouvant être hébergé sur votre ordinateur, des serveurs privés ou des sites Web open source, tels que Github.
- Git permet également aux utilisateurs de créer de nouvelles "branches" du code, ce qui permet aux différentes versions du code de cohabiter.

LES CONCEPTS DE GIT

- **Répertoire de travail :** les fichiers et dossiers que vous voyez et modifiez.
- **Zone de transit (index) :** les modifications choisies pour le prochain commit.
- **Dépôt local :** le dossier caché `.git` qui contient tout l'historique (commits, branches).
- **Commit :** un instantané (snapshot) du projet à un moment donné, avec auteur, date et message.
- **Historique :** la chaîne de tous les commits, du plus ancien au plus récent.
- **Dépôt distant :** un dépôt sur un serveur (GitHub, GitLab) pour collaborer et sauvegarder.

LES ZONES DU GIT

Git a 3 "zones" différentes pour votre code:

- Répertoire de travail : zone dans laquelle vous allez travailler (création, modification, suppression et organisation des fichiers)
- Zone de transit : la zone dans laquelle vous listerez les modifications apportées au répertoire de travail
- Référentiel : où Git stocke en permanence les modifications que vous avez apportées sous

différentes versions du projet

CONFIGURER ET INITIALISER UN DÉPÔT

```
$ git config --global user.name "Prénom Nom"
$ git config --global user.email "prenom.nom@exemple.com"
$ git config --list           # affiche la configuration
$ git init                    # crée le dossier caché .git
$ git status                  # état du dépôt
```

- **--global :** identité valable pour tous vos dépôts ; **`--local`** : pour le dépôt courant seulement.
- **L'identité** apparaît dans chaque commit : elle sert à savoir qui a fait quoi.
- **git init** crée un nouveau dépôt ; **`git clone URL`** récupère un dépôt existant.

SUIVRE LES CHANGEMENTS : STATUS, DIFF, LOG

Commande	Rôle
`git status`	fichiers modifiés, indexés, non suivis
`git diff`	différences entre répertoire de travail et index
`git diff --staged`	différences entre index et dernier commit
`git log`	historique des commits
`git log --oneline --graph --all`	historique compact avec toutes les branches
`git show <commit>`	détail d'un commit (message et modifications)
`git blame <fichier>`	auteur et commit de chaque ligne du fichier

.GITIGNORE : CE QU'ON NE VERSIONNE PAS

```
# .gitignore (à la racine du dépôt)
node_modules/       # dépendances
dist/               # fichiers générés
*.log               # journaux
.env                # secrets et configuration locale
.DS_Store           # fichiers du système
.idea/              # fichiers de l'éditeur
```

- **Ignorer :** dépendances, fichiers générés, fichiers d'éditeur ou de système, **secrets**.
- **Limite :** n'agit pas sur un fichier déjà suivi (`git rm --cached fichier`).
- **Secrets :** un mot de passe déjà poussé reste dans l'historique : il faut le révoquer.

CONTRÔLE WORKFLOW

Le contrôle de version a un flux de travail général que la plupart des développeurs utilisent pour écrire du code et le partager avec l'équipe

- Obtenez une copie locale du code s'ils n'en ont pas encore.
- Apportez des modifications au code pour corriger les bogues ou ajouter de nouvelles fonctionnalités.
- Une fois le code prêt, rendez-le disponible pour examen par votre équipe (Pull Request).
- Une fois le code révisé, fusionnez-le dans la base de code partagée de l'équipe.

ACTIONS AVEC GIT (1/2)

```
$ git clone URL_DU_DEPOT          # copie un dépôt distant sur son poste
$ git add fichier.txt             # indexe le fichier (git add . : tout)
$ git commit -m "message"         # crée un commit avec un message
```

- **clone :** récupère le dépôt avec tout son historique.
- **add :** place les modifications dans la zone de transit : elles seront prises en compte au prochain commit.
- **commit :** crée une nouvelle version contenant tout ce qui est indexé, avec un message qui décrit les changements.
- **Attention :** les commandes sont en minuscules et les options prennent un tiret simple (`-m`).

ACTIONS AVEC GIT (2/2)

```
$ git log                    # consulter l'historique
$ git push                   # envoyer ses commits au dépôt distant
$ git pull                   # récupérer les changements distants
$ git branch nom             # créer une branche
$ git checkout nom           # changer de branche (ou : git switch nom)
$ git branch -a              # lister toutes les branches
$ git branch -d nom          # supprimer une branche déjà fusionnée
$ git merge nom              # fusionner la branche nom dans la branche courante
```

- **merge :** Git fusionne automatiquement ; s'il n'y arrive pas, il signale un **conflit** à résoudre.

DÉPÔTS DISTANTS : REMOTE, FETCH, PULL, PUSH

```
$ git remote add origin URL      # déclare le dépôt distant "origin"
$ git remote -v                  # liste les dépôts distants
$ git push -u origin main        # 1er push : lie main à origin/main
$ git fetch                      # télécharge sans modifier son travail
$ git pull                       # = git fetch + git merge
```

- **origin :** nom habituel du dépôt distant ; **`origin/main`** est la copie locale de la branche distante.
- **fetch :** sans risque, il ne modifie pas votre branche. **`pull`** = **`fetch`** + intégration (**`--rebase`** pour rejouer vos commits).
- **Push refusé :** le distant a avancé, faites d'abord un **`pull`**.

Bonnes pratiques pour le contrôle des sources

- Faites de petits changements, avec des commit régulières.
- Ne pas comité des fichiers personnels.
- Mettre à jour souvent et juste avant de pousser pour éviter les conflits de fusion.
- Vérifiez votre changement de code avant de le pousser dans un référentiel, assurez-vous qu'il compile et que les tests réussissent.
- Portez une attention particulière à la validation des messages car ceux-ci vous indiqueront pourquoi un changement a été effectué.
- Lier les modifications de code aux éléments de travail (WorkItem).

Gestion des Branches

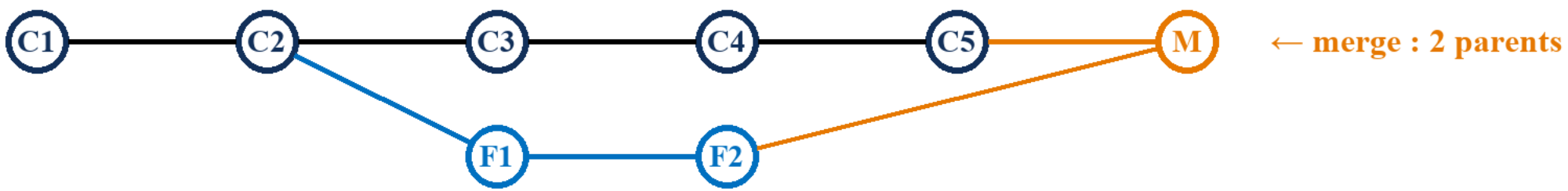
Motivations

Une équipe de développeurs participe à la réalisation d'une application:

- Comment conserver un historique?
- Comment revenir en arrière?
- Comment travailler à plusieurs en parallèle sur le même code?
- Comment gérer plusieurs versions du code à la fois?
- Comment savoir ce qui a modifié et par qui (et pourquoi)?

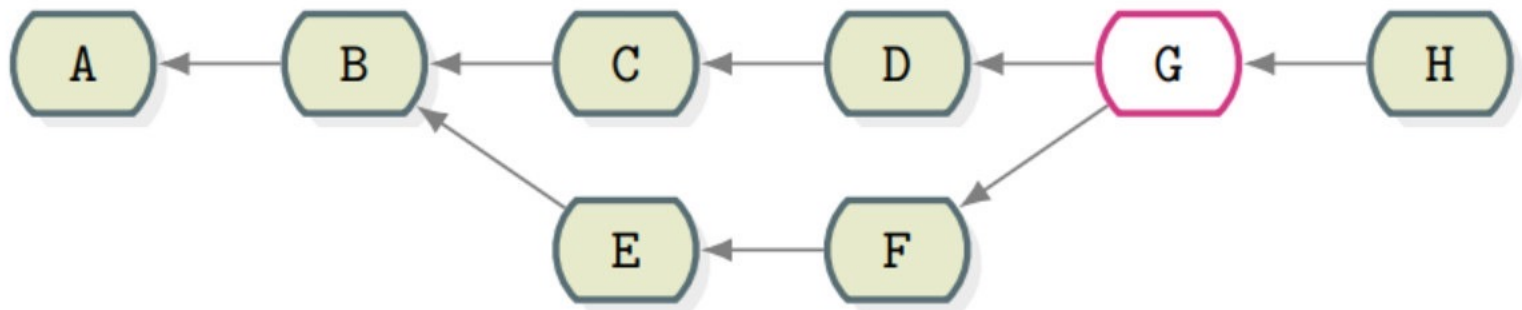
LA NOTION D'HISTORIQUE

- L'historique est un graphe orienté sans cycle de commits : chaque commit pointe vers son ou ses parents.
- Chaque commit est un **instantané** (snapshot) complet du projet ; les différences (diff) sont calculées à la demande.
- Il peut contenir plusieurs branches : des chemins qui évoluent en parallèle.
- Un commit de fusion (merge) a deux parents ou plus : il réunit plusieurs branches.



Historique: les merges

On appelle merge toute version ayant un degré sortant strictement supérieur à 1.
Cette version correspond alors à la fusion des patchs de plusieurs branches.



Gestion des accès concurrents

Gestion pessimiste

- Un seul contributeur à accès en écriture à un fichier
- Pas de conflits
- Pas pratique

Gestion optimiste

- Chaque développeur peut modifier sa copie locale en parallèle
- Risques de conflits

▶ Modifications concurrentes de la même zone de texte

Les objets dans Git

- *Blobs*
- *Tree*
- *Commit*
- *Tag*

Blob « Binary Large Object »

On appelle Blob, l'élément de base qui permet de stocker le contenu d'un fichier.
C'est simplement le contenu d'un fichier, stocké de manière compressée dans la base de données Git. Les blobs sont identifiés par des clés SHA-1 qui sont calculées à partir de leur contenu.

- Chaque Blob est identifié de manière unique par sa clé
- A chaque révision du fichier correspond un nouveau Blob
    - Le blob stocke le contenu entier du fichier

Le Blob ne dépend pas du nom ou de l'emplacement

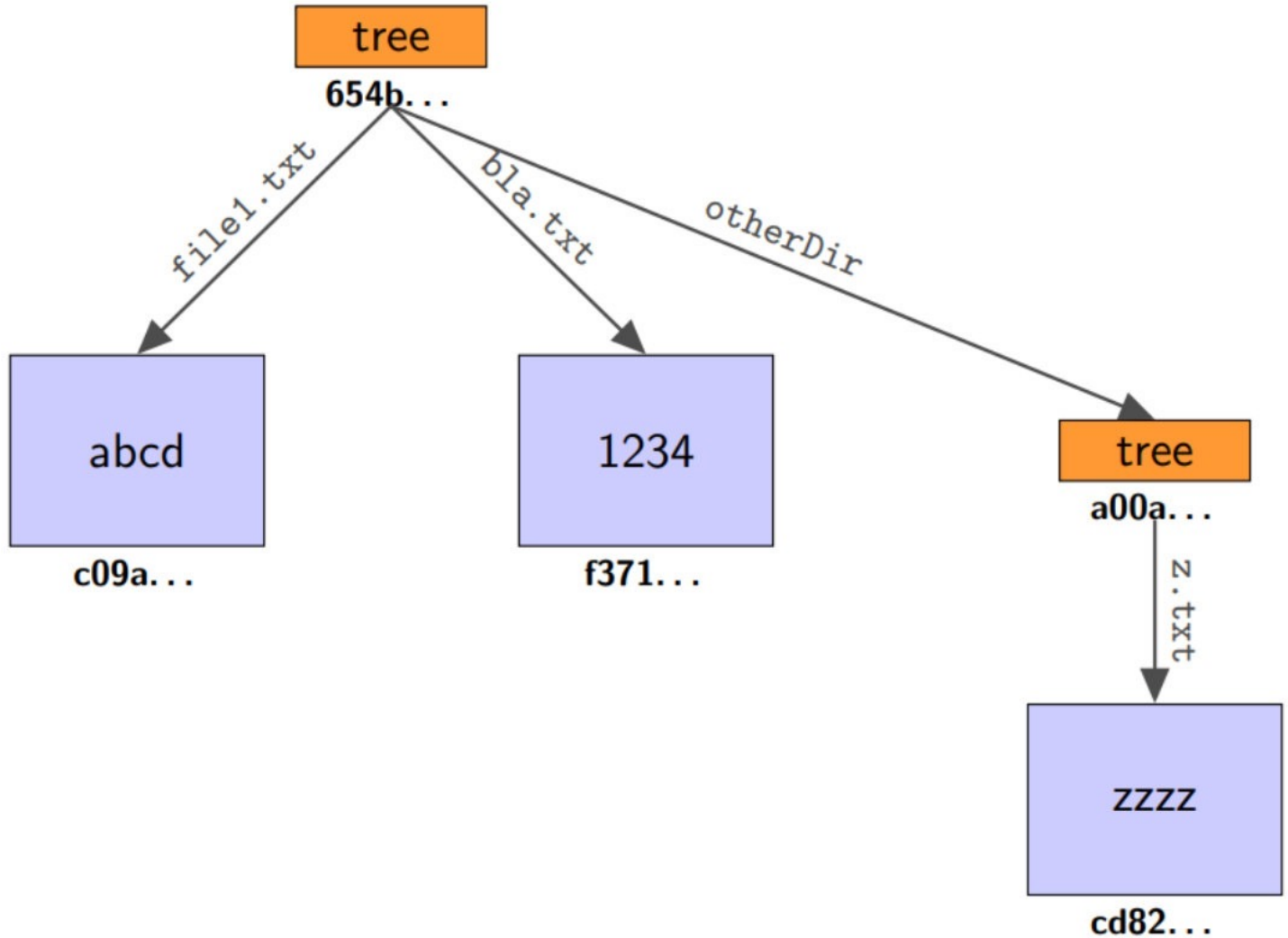
- Si un fichier est renommé, pas de nouveau Blob
- Si un fichier est déplacé, pas de nouveau Blob

- Le contenu du Blob est compressé avec zlib. Il contient:
    - Le type d'objet (blob)
    - La taille du fichier initial
    - Le contenu du fichier

Tree

- Un arbre représente un répertoire (ou dossier) dans un projet Git.
- Chaque arbre contient des entrées qui pointent vers des blobs correspondant aux fichiers présents dans ce répertoire, ainsi que vers d'autres sous-arbres pour les sous-répertoires.
- L'entrée d'un arbre contient des métadonnées telles que le nom du fichier ou du sous-répertoire, le mode (permissions), et l'identifiant SHA-1 du blob ou de l'arbre auquel elle pointe.

Tree et Blob: Exemple

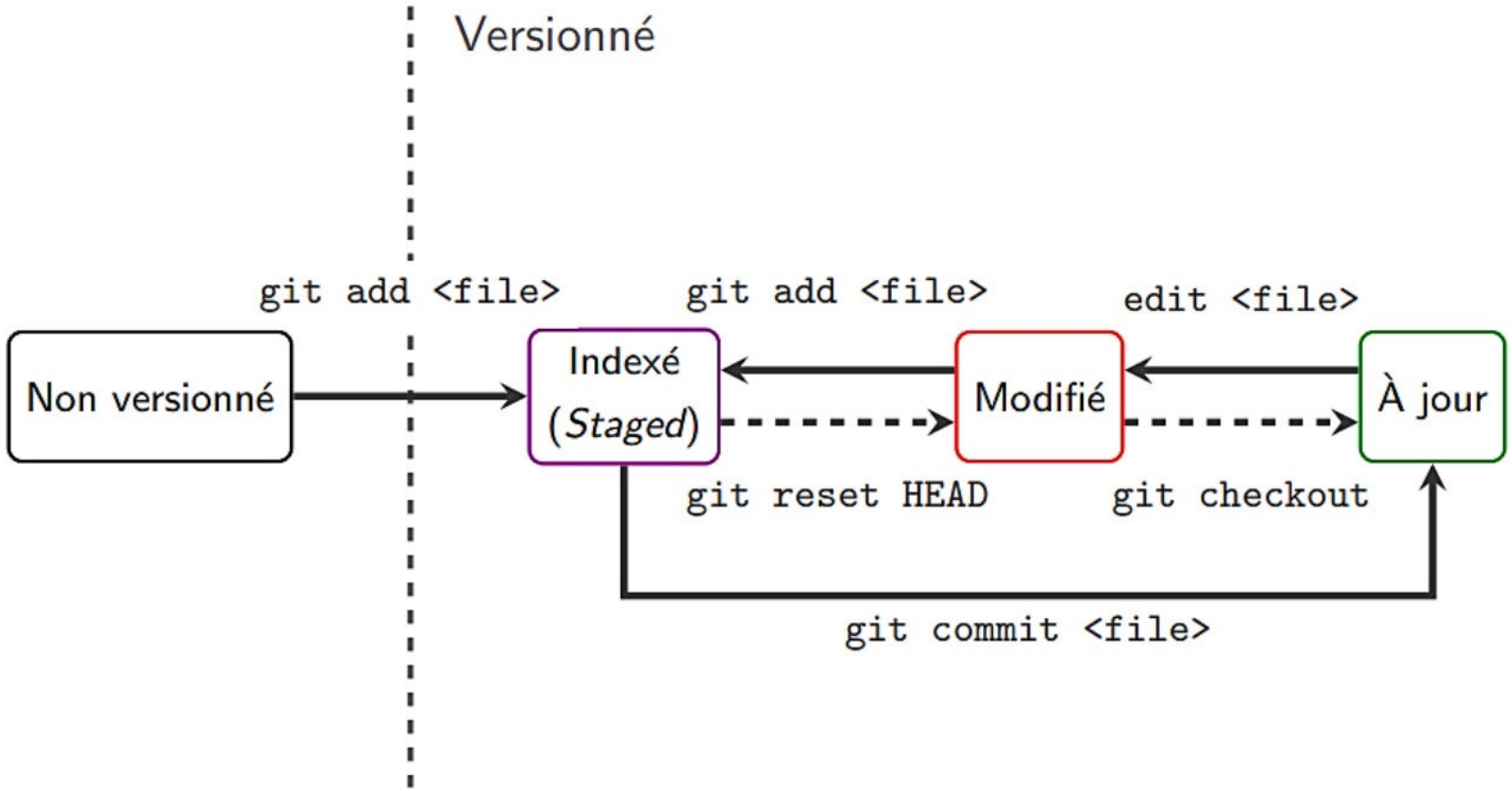


Commit

Un Commit stocke l'état d'une partie du dépôt à un instant donné.
Il contient :

- Un pointeur vers un Tree (arbre racine) dont on souhaite sauver l'état.
- Un pointeur vers un ou plusieurs autres Commits pour constituer un historique.
- Les informations sur l'auteur du Commit.
- Une description sous forme d'une chaîne de caractères.

Le cycle de vie d'un fichier



Les branches

Dans Git

- Une branche est un pointeur sur un commit
- Chaque commit pointe vers son prédécesseur
- La variable HEAD pointe sur la branche sur laquelle on travaille actuellement.

Les branches – Les commandes

branch : liste les branches avec une * pour la branche active.
branch <nom> : crée une nouvelle branche <nom>.
branch -m : permet de renommer une branche.
branch -d : permet de supprimer une branche.
checkout : change (ou/et crée) de branche active.
show-branch : affiche les branches et leurs commits.

Exemple

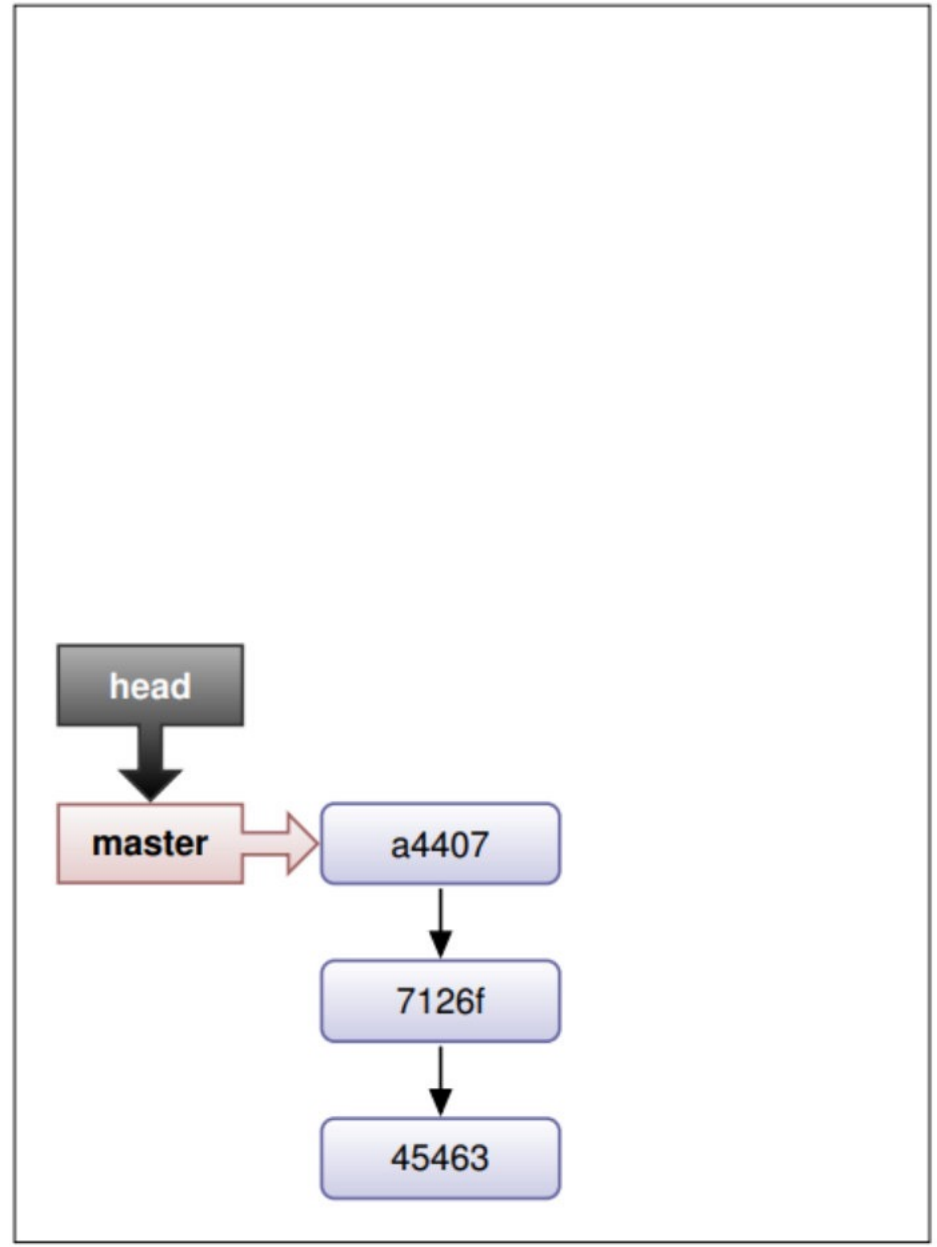
```
$ git branch
    * master
$ git branch maBranche
$ git branch
      maBranche
    * master
$ git checkout maBranche
$ git branch
    * maBranche
      master
```

LES MERGES

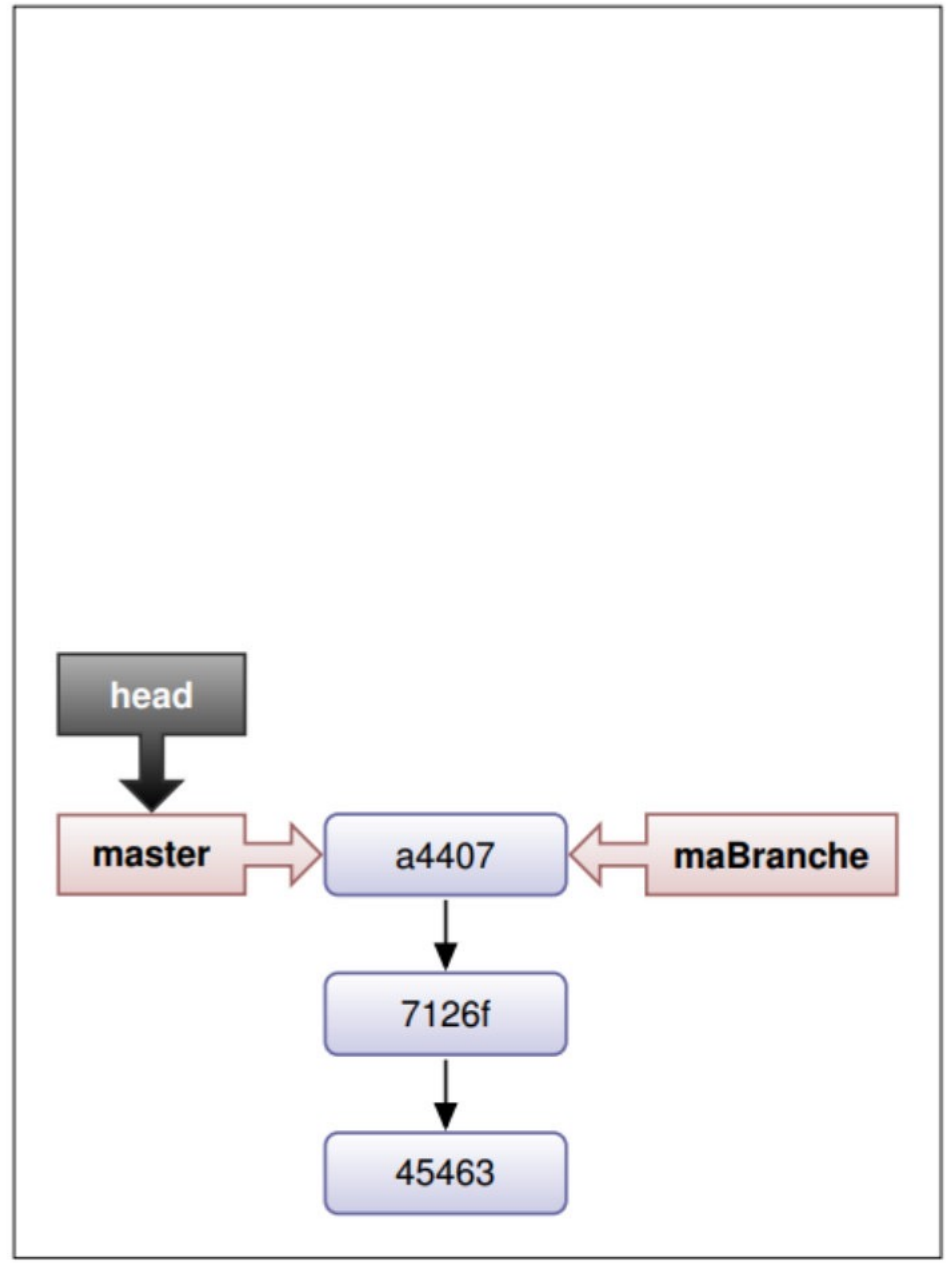
```
$ git checkout brancheDestination
$ git merge brancheSource
```

- Crée un commit de fusion qui a pour parents les deux branches.
- La branche courante avance à ce commit ; la branche source ne bouge pas.
- **Si l'historique n'a pas divergé :** simple avance du pointeur, c'est le **fast-forward**.
- **Si les mêmes lignes ont été modifiées :** c'est un **conflit** à résoudre.

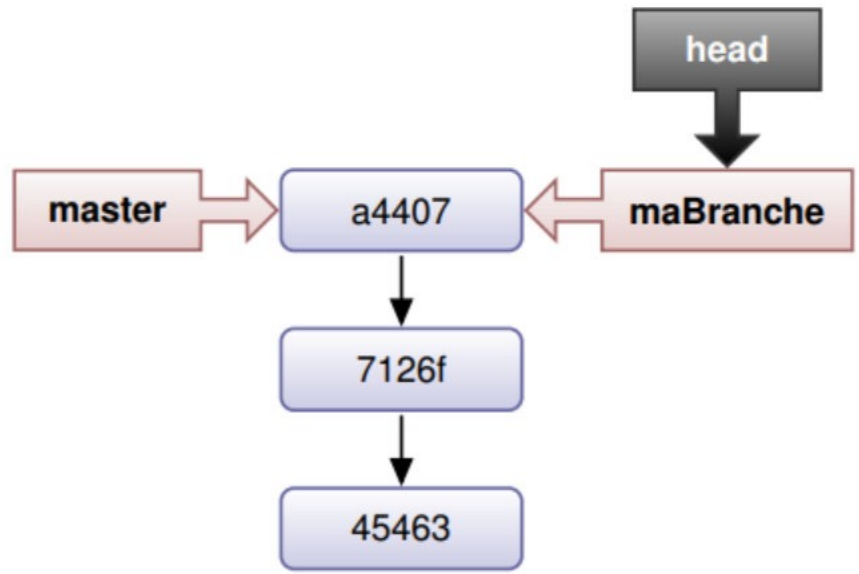
```
ls
 foo.txt dir
```



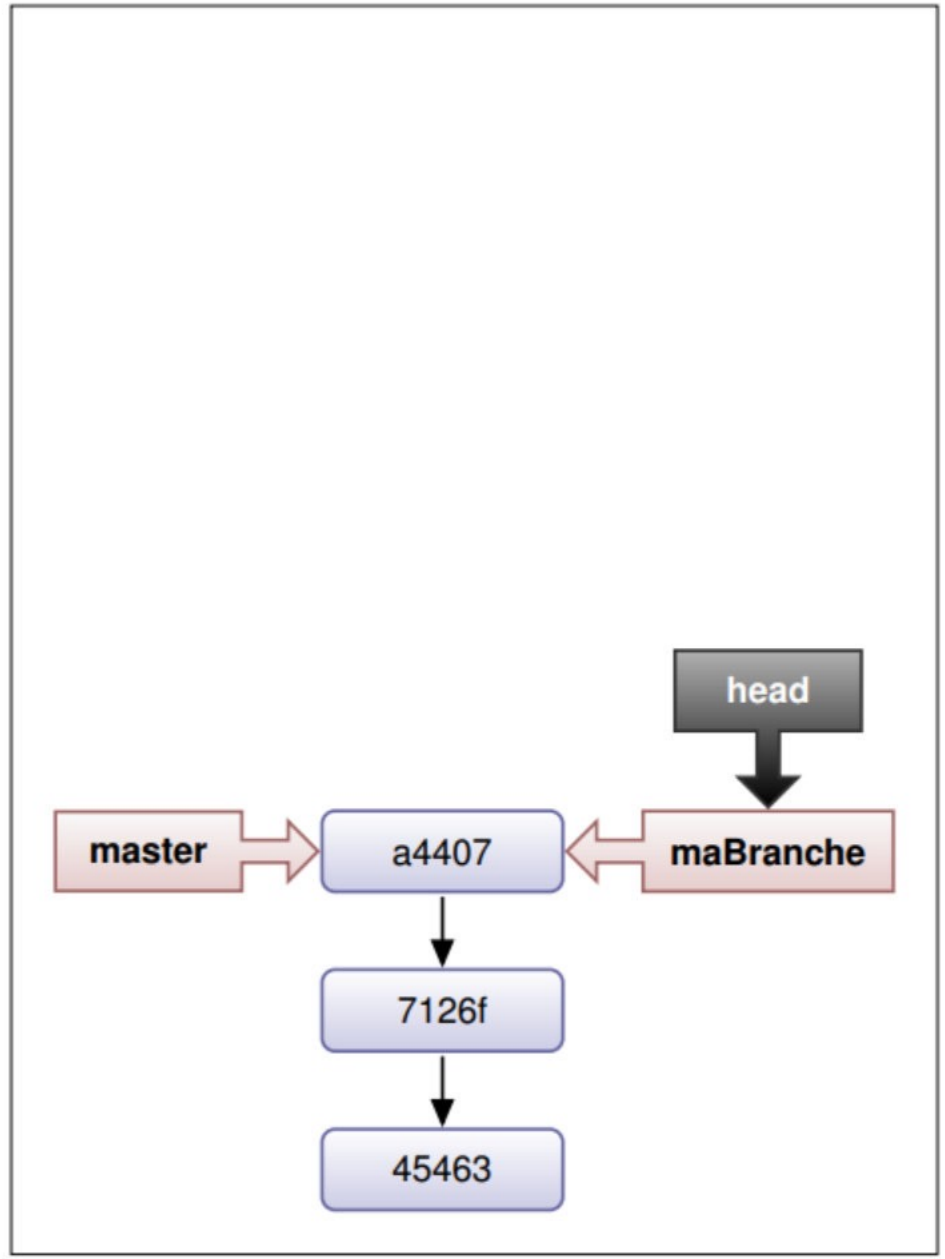
```
ls
 foo.txt dir

git branch maBranche
```



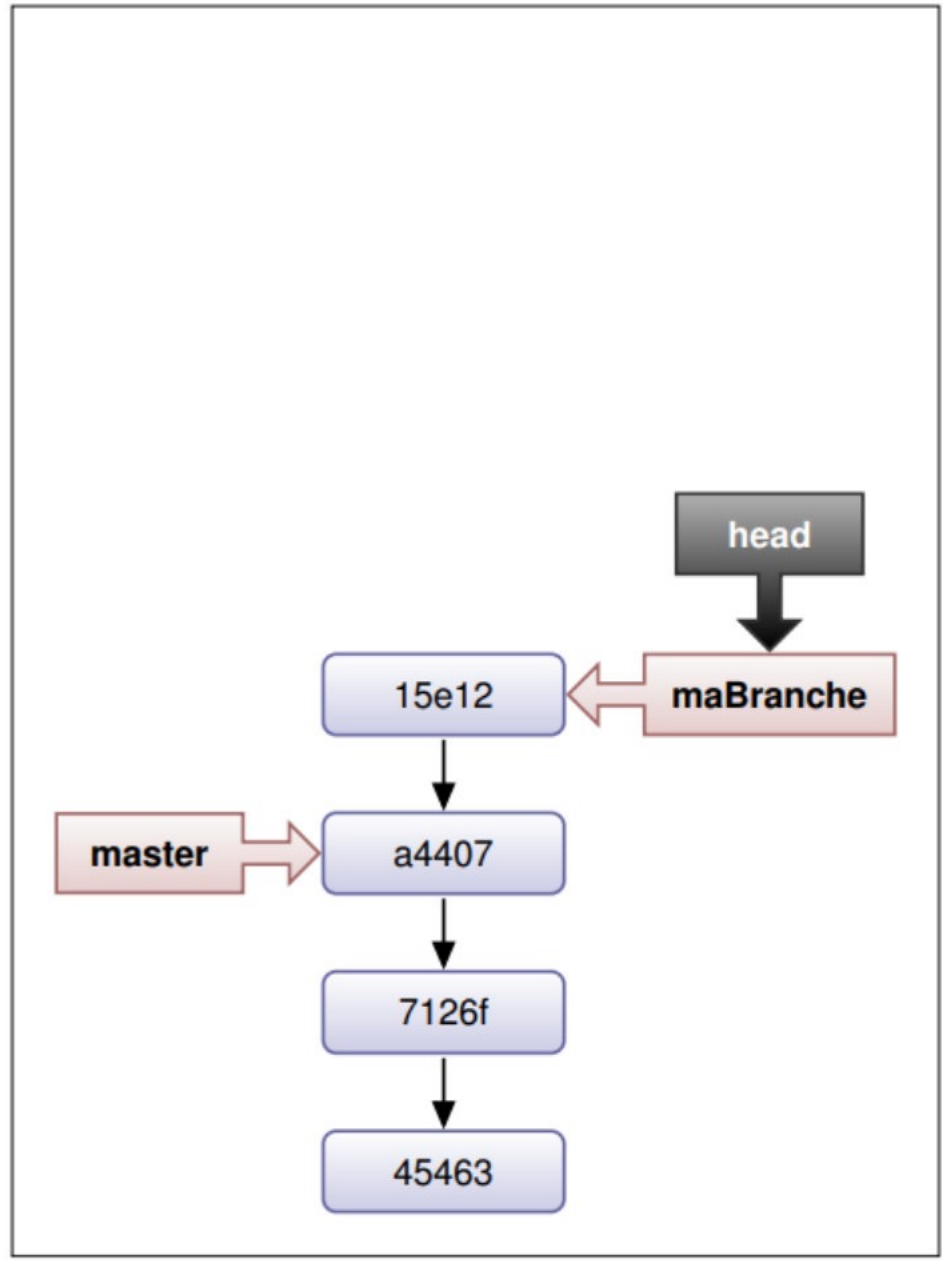
```
ls
 foo.txt dir

git branch maBranche

git checkout maBranche
```



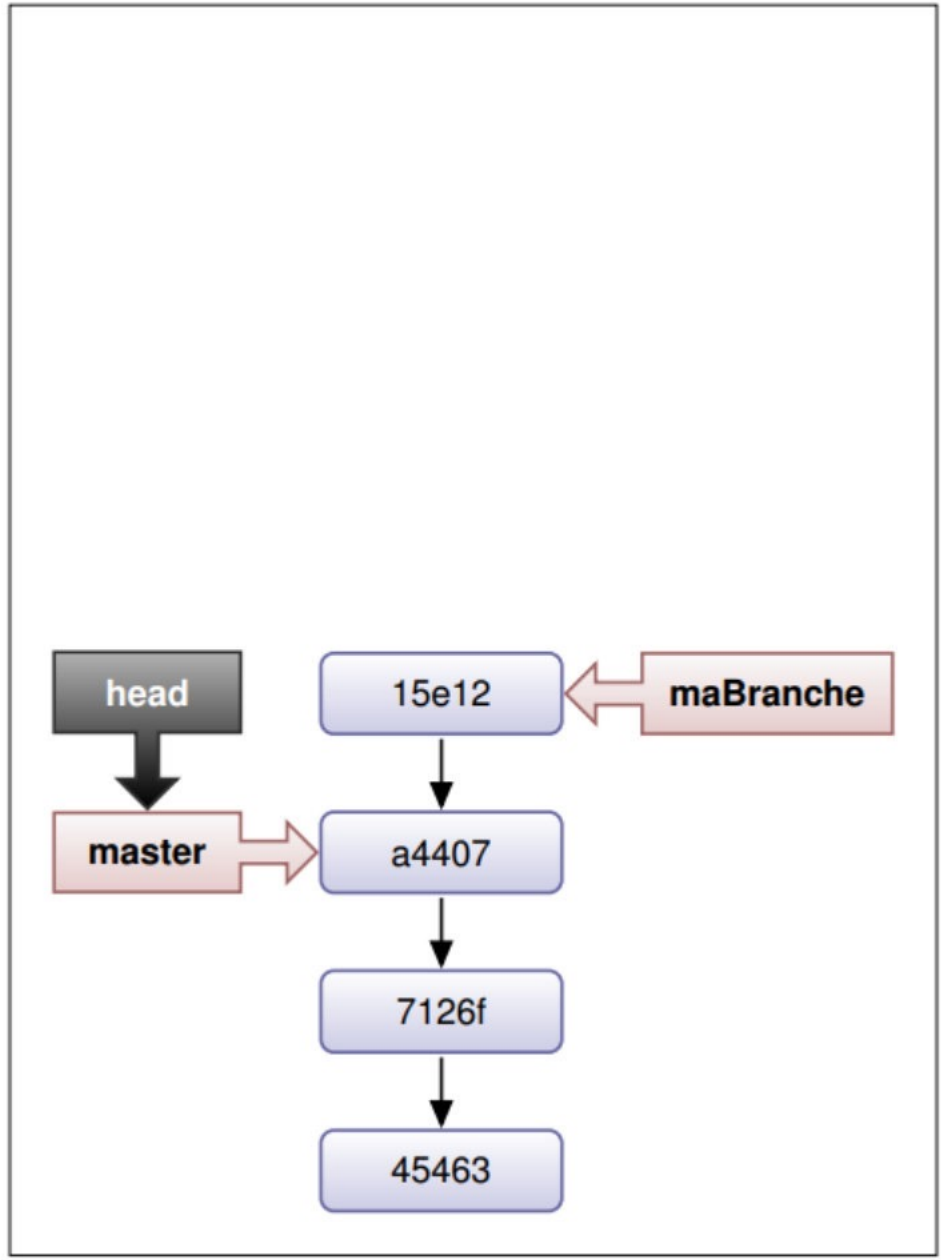
```
ls
 foo.txt dir

git branch maBranche

git checkout maBranche
```



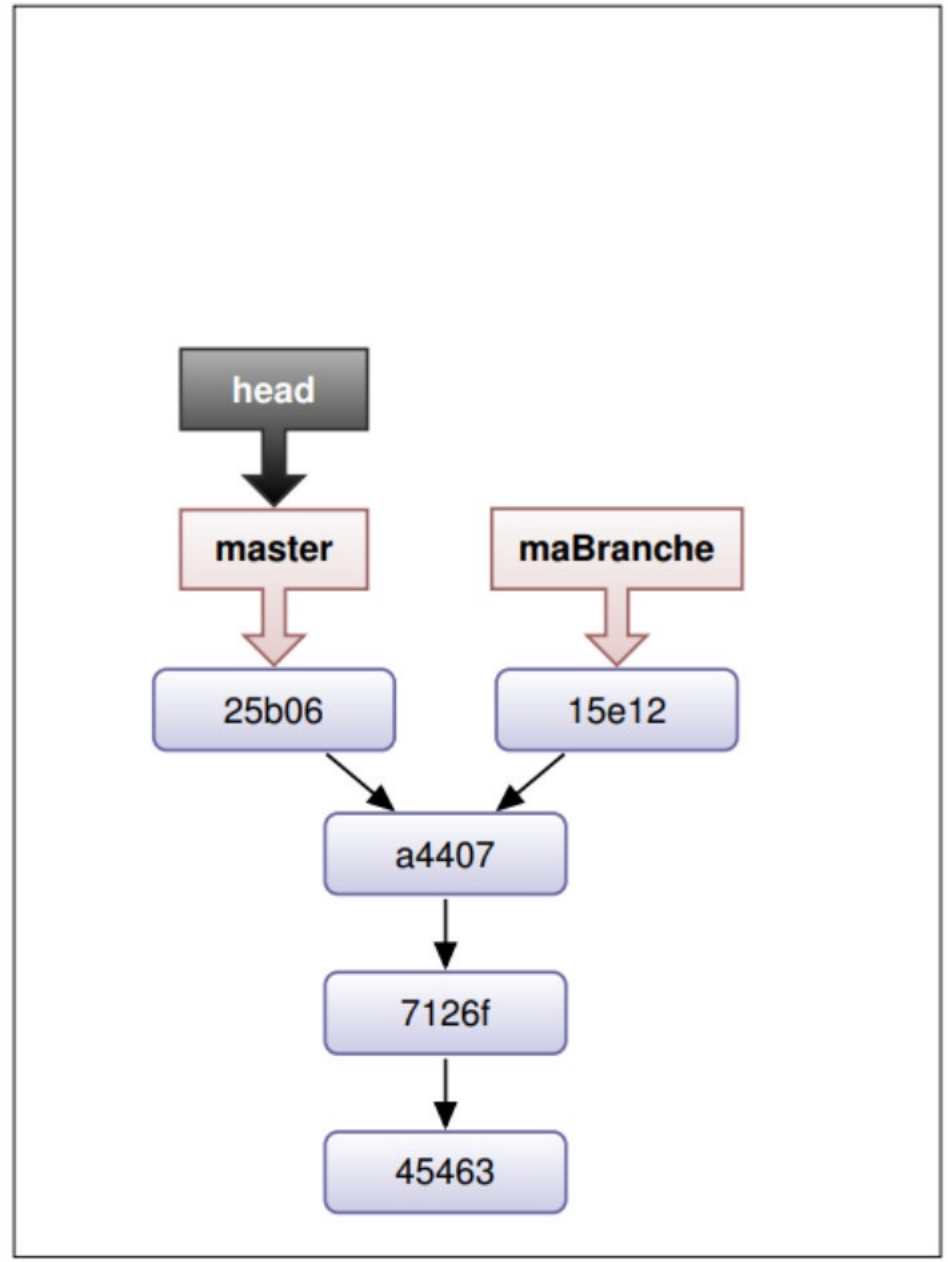
```
ls
 foo.txt dir

git branch maBranche

git checkout maBranche

touch fichier1.txt
ls
 dir fichier1.txt foo.txt
git add fichier1.txt
git commit -m "Add fichier1.txt"
```



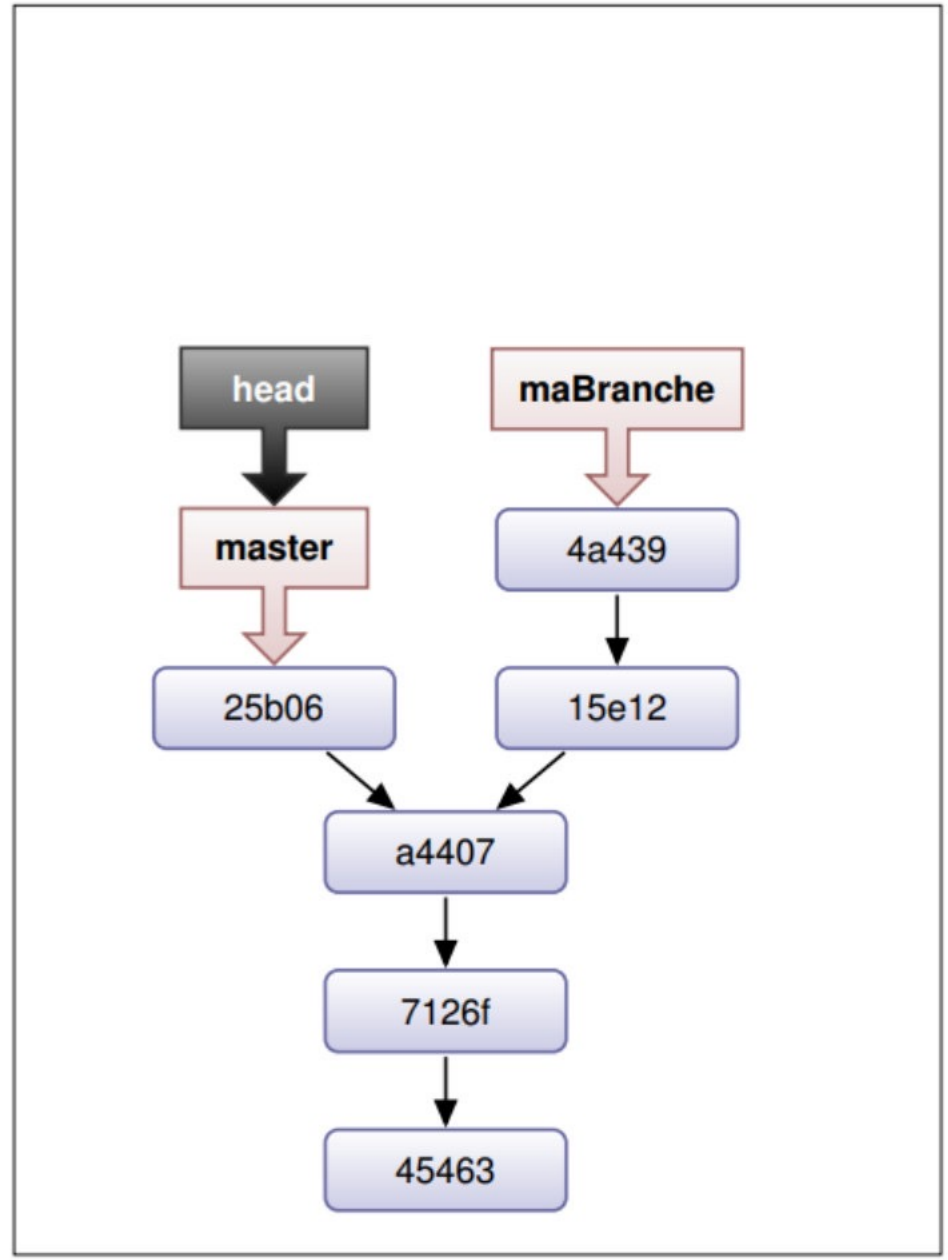
```
ls
 foo.txt dir

git branch maBranche

git checkout maBranche

touch fichier1.txt
ls
 dir fichier1.txt foo.txt
git add fichier1.txt
git commit -m "Add fichier1.txt"

git checkout master
ls
 dir foo.txt
```



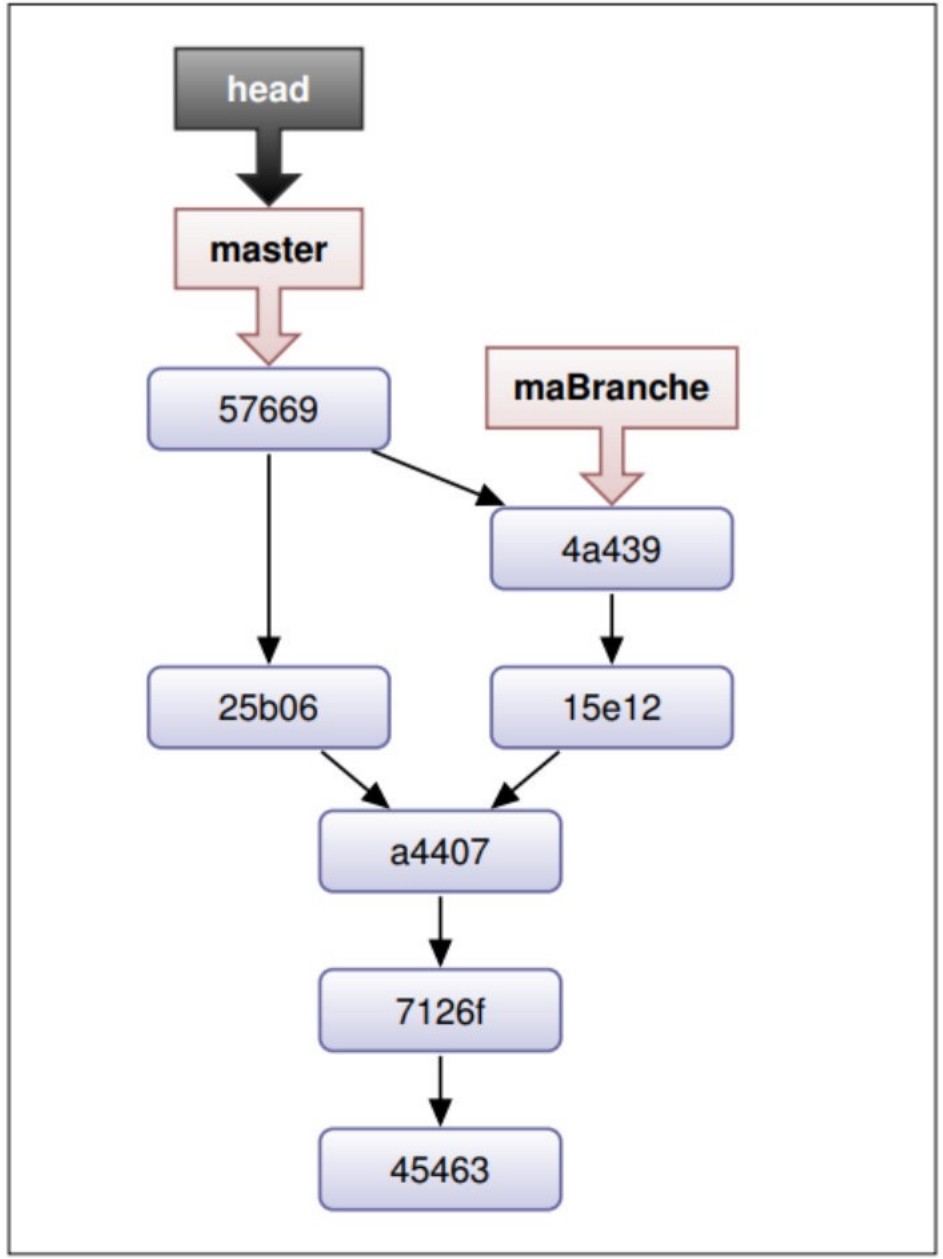
```
ls
 foo.txt dir

git branch maBranche

git checkout maBranche

touch fichier1.txt
ls
 dir fichier1.txt foo.txt
git add fichier1.txt
git commit -m "Add fichier1.txt"

git checkout master
ls
 dir foo.txt

touch fichier2.txt
git add fichier2.txt
git commit -m "Add fichier2.txt"
```



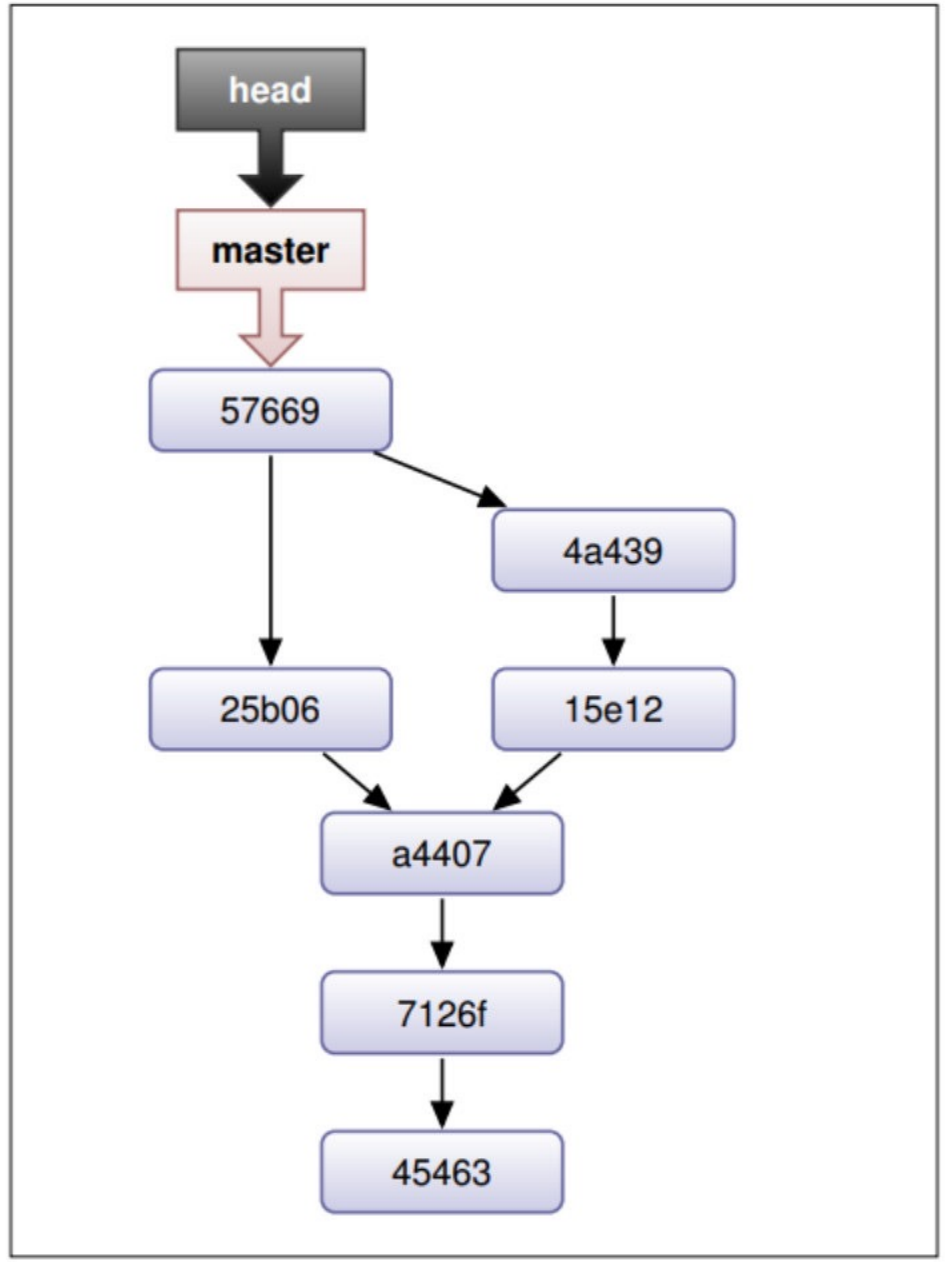
```
ls
 dir fichier2.txt foo.txt

git checkout maBranche
ls
 dir fichier1.txt foo.txt

echo "titi" > fichier1.txt
git commit -am "Modif
fichier1.txt"

git checkout master
```



```
ls
 dir fichier2.txt foo.txt

git checkout maBranche
ls
 dir fichier1.txt foo.txt

echo "titi" > fichier1.txt
git commit -am "Modif
fichier1.txt"

git checkout master
git merge maBranche
ls
 dir fichier1.txt fichier2.txt
foo.txt
```

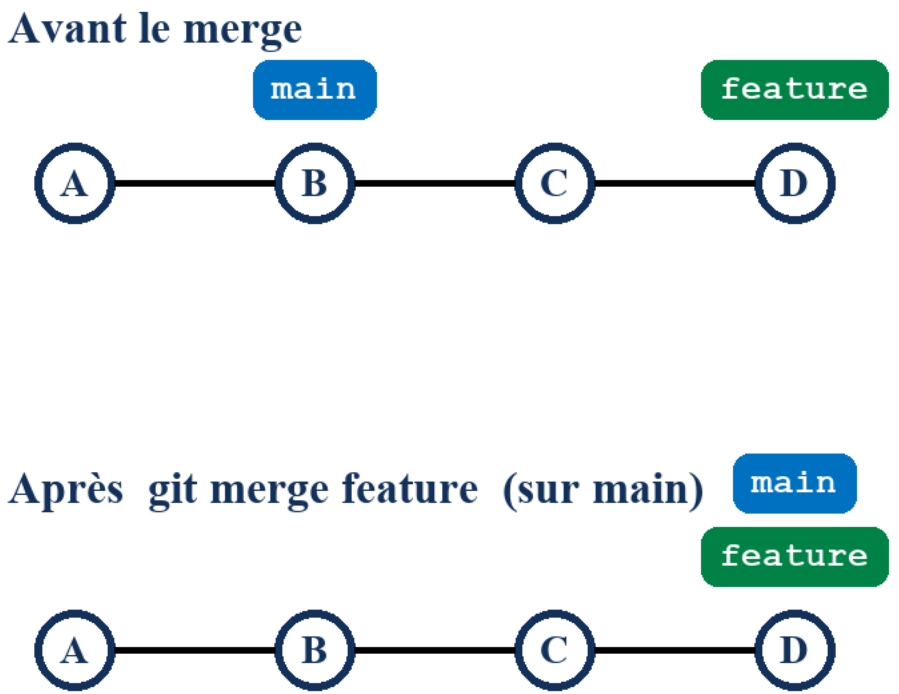


```
ls
 dir fichier2.txt foo.txt

git checkout maBranche
ls
 dir fichier1.txt foo.txt

echo "titi" > fichier1.txt
git commit -am "Modif
fichier1.txt"

git checkout master
git merge maBranche
ls
 dir fichier1.txt fichier2.txt
foo.txt

git branch -d maBranche
```



FAST-FORWARD

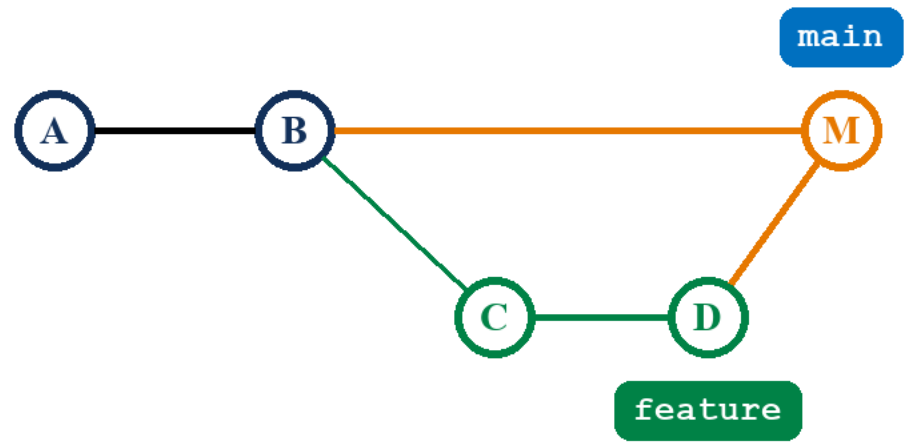
- Possible quand la branche cible n'a **aucun commit en plus** : l'historique n'a pas divergé.
- Git avance simplement le pointeur de la branche : **pas de commit de fusion**.
- Historique linéaire, plus simple à lire.
- **Limite :** on ne voit plus qu'une branche a existé.
- `git merge --ff-only` : n'autorise que ce cas, sinon le merge est refusé.



NO FAST-FORWARD (--no-ff)

- **`git merge --no-ff :`** force un commit de fusion même si un fast-forward est possible.
- Le commit de fusion M a deux parents : le dernier commit de **`main`** et celui de **`feature`**.
- **Avantage :** la branche et ses commits restent visibles dans l'historique.
- **Inconvénient :** historique moins linéaire.
- Si l'historique a divergé, le commit de fusion est de toute façon nécessaire.

Après git merge --no-ff feature



CONFLIT DE MERGE : POURQUOI ?

- **Cause :** deux branches modifient la **même zone** d'un fichier (ou un fichier modifié d'un côté, supprimé de l'autre).
- **Git :** fusionne ce qui est compatible et s'arrête sur le reste.
- **Signal :** `CONFLICT (content): Merge conflict in hello.py`, puis `git status` liste les fichiers sous « Unmerged paths ».
- **Pas de panique :** `git merge --abort` revient à l'état d'avant la fusion.

```
base    : print(f'Hi {stuff}')
main    : print(f'Hello, {stuff}!')      # Hi -> Hello + "!"
feature : print(f'Hi {someone}')         # stuff -> someone
```

RÉSOUDRE UN CONFLIT À LA MAIN

```
<<<<<<< HEAD
print(f'Hello, {stuff}!')
||||||| merged common ancestors
print(f'Hi {stuff}')
=======
print(f'Hi {someone}')
>>>>>>> function-politeness
```

Résultat attendu :

```
print(f'Hello, {someone}!')
```

1. Ouvrir le fichier, repérer les marqueurs.
2. Choisir ou combiner le contenu, supprimer **tous** les marqueurs, puis tester.
3. `git add fichier`, puis `git commit` (ou `git merge --continue`).
   - Style diff3 (version de l'ancêtre commun) :
     `git config --global merge.conflictstyle diff3`

OUTILS DE RÉSOLUTION ET PRÉVENTION

Outils

- Éditeur de fusion de VS Code
- `git mergetool` avec kdiff3, Meld...
- Même procédure pour `pull`, `stash pop`, `cherry-pick`
- Rebase : `git rebase --continue` ou `--abort`

Prévenir les conflits

- Petits commits, branches courtes
- `git pull` souvent, surtout avant de pousser
- Se répartir le travail sur des fichiers différents
- Formater le code de la même façon (formateur partagé)

REBASE

git merge

- Combine les deux historiques
- Crée un commit de fusion à **deux parents**
- L'historique de chaque branche est préservé

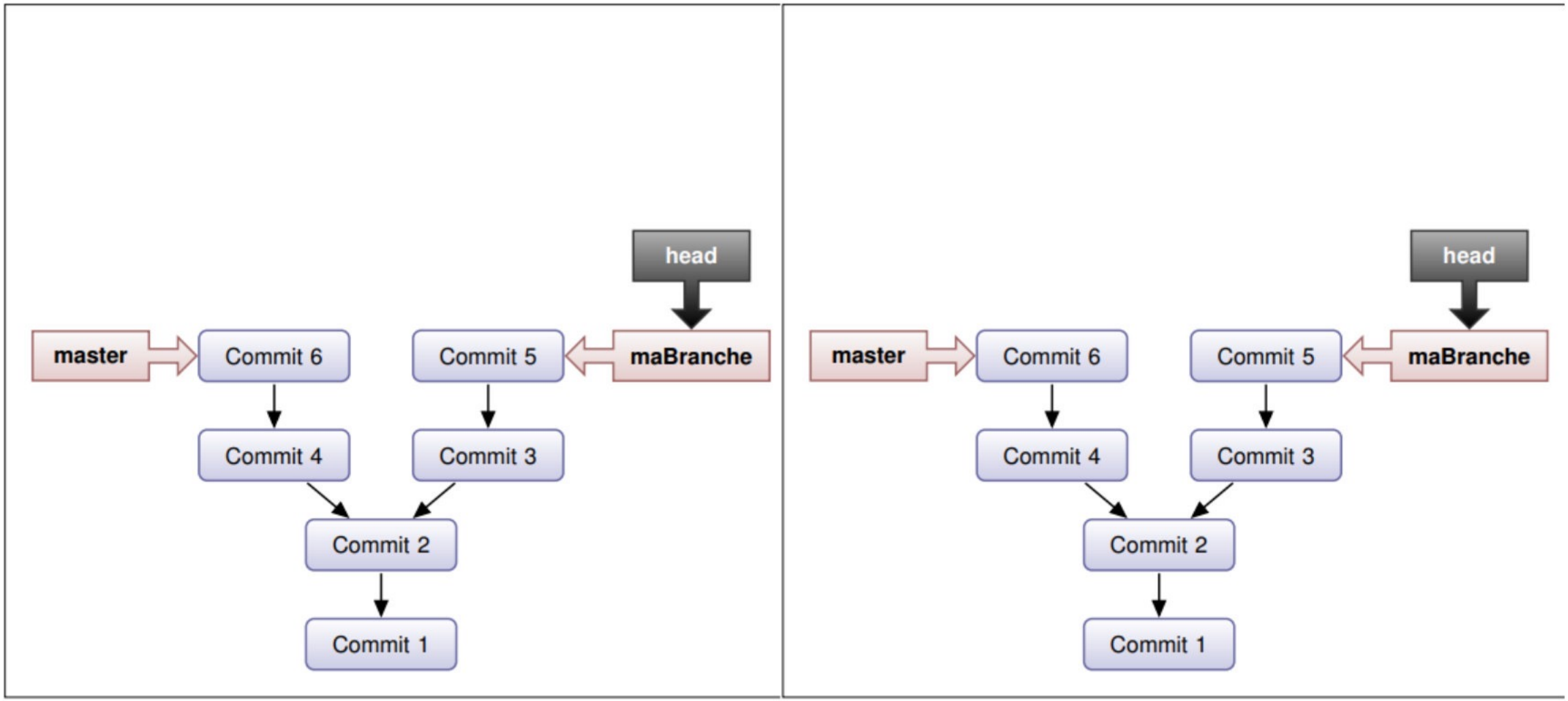
git rebase

- Rejoue les commits de la branche courante au-dessus d'une autre base
- Historique **linéaire**, pas de commit de fusion
- Réécrit l'historique : nouveaux identifiants

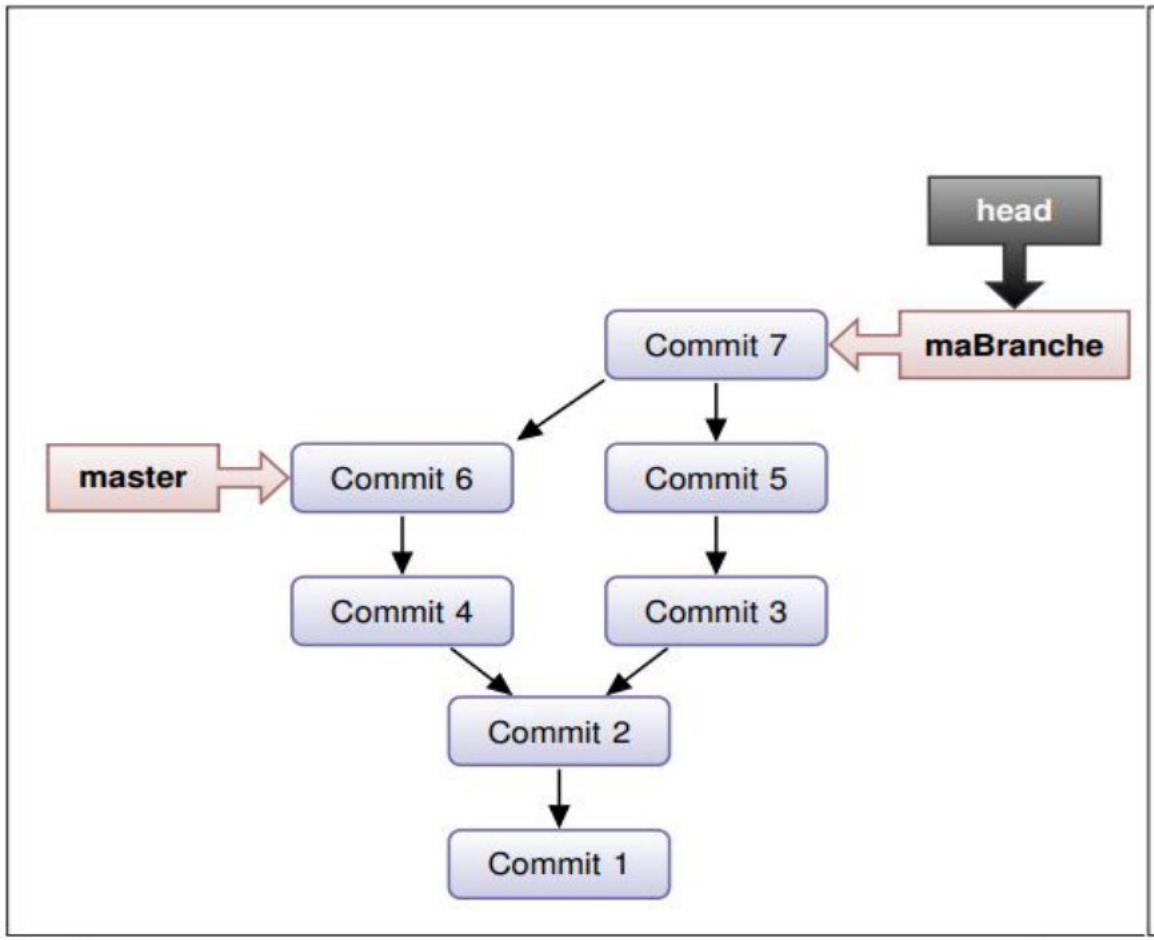
```
$ git checkout feature
$ git rebase main
```

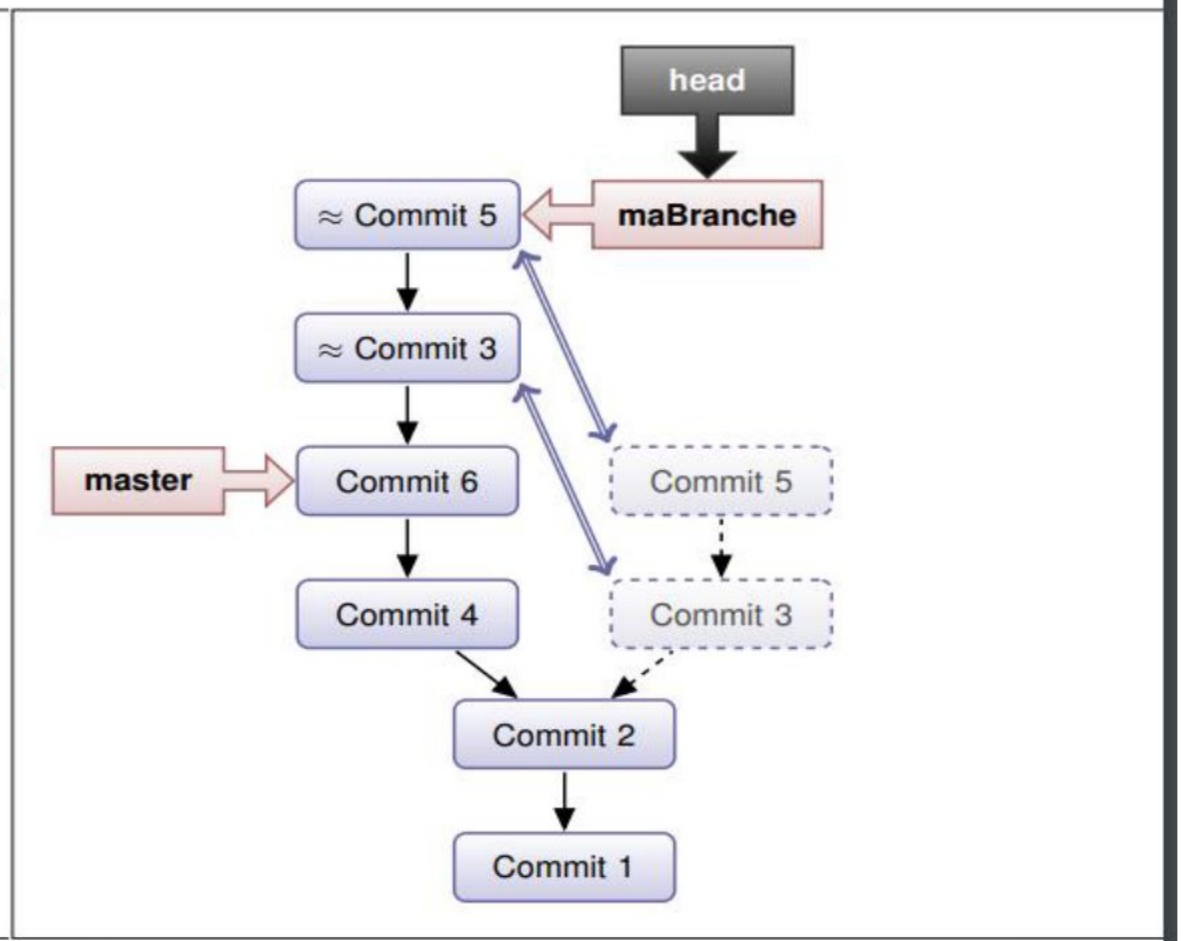
Règle d'or : ne jamais rebaser des commits déjà partagés

GIT MERGE VS GIT REBASE



GIT MERGE VS GIT REBASE






```
git checkout maBranche
git merge master
```

```
git checkout maBranche
git rebase master
```

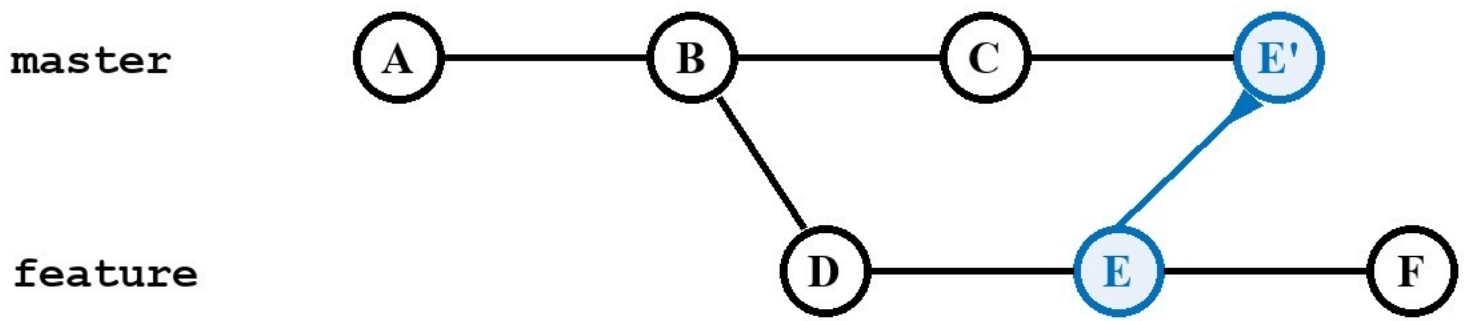
REBASE INTERACTIF

```
$ git rebase -i HEAD~3      # les 3 derniers commits
$ git rebase -i --root      # tous les commits
```

Action	Effet
`pick`	garde le commit tel quel
`reword`	garde le commit, modifie son message
`edit`	s'arrête pour modifier le contenu du commit
`squash`	fond le commit dans le précédent (messages combinés)
`fixup`	comme squash, en jetant le message du commit
`drop`	supprime le commit

Git cherry-pick : appliquer un commit précis

- Principe : git cherry-pick <commit> reprend les modifications d'un commit d'une autre branche et crée un nouveau commit équivalent sur la branche courante (nouvel identifiant, même message).
- Différence : avec merge ou rebase on intègre toute une branche ; ici, on choisit seulement certains commits.
- Cas d'usage : appliquer un correctif sur plusieurs branches (release et master), récupérer un commit fait sur la mauvaise branche, reprendre une seule fonctionnalité d'une branche non terminée.




```
git checkout master
git cherry-pick <id de E>
```

E' = copie de E (nouvel id)

Git cherry-pick : commandes et précautions

- `git cherry-pick <commit>` : applique un commit sur la branche courante (identifiant visible avec git log --oneline).
- `git cherry-pick A B` : applique plusieurs commits ; A..B applique les commits après A jusqu'à B inclus.
- `git cherry-pick -n <commit>` : applique les modifications sans créer de commit.
- `git cherry-pick -x <commit>` : ajoute « (cherry picked from commit ...) » au message, pour la traçabilité.
- Conflit : résoudre, git add, puis git cherry-pick --continue ; git cherry-pick --abort annule l'opération.
- Précaution : le commit est dupliqué (deux identifiants). À utiliser avec modération : cela peut compliquer l'historique et créer des conflits lors d'un merge ultérieur.

MESSAGE DE COMMIT

Un bon message

- Décrire **quoi** et **pourquoi**, pas comment
- Première ligne courte (50 caractères environ)
- Un commit = un changement logique
- **Mauvais :** « Modifie la fonction f pour tester la variable a »
- **Bon :** « Vérifie les droits de l'utilisateur avant d'exécuter l'action X »

Conventional Commits

- Format : `type(portée): description`
- `feat` : nouvelle fonctionnalité
- `fix` : correction de bug
- `docs`, `refactor`, `test`, `chore`, `ci`
- **Exemple :**
  `feat(auth): ajoute la connexion par mail`
- **Avec Jira :**
  `PROJ-123: corrige le calcul de TVA`

Git stash : mettre son travail de côté

- Problème : vous travaillez sur une branche, vos modifications ne sont pas prêtes à être commitées et il faut changer de branche en urgence (correction de bug, pull, revue de code).
- Solution : git stash range les modifications en cours dans une pile (stash) et remet le répertoire de travail dans l'état du dernier commit (HEAD).
- Mis de côté : les fichiers suivis modifiés (indexés ou non). Les nouveaux fichiers non suivis ne sont pas inclus par défaut.

Après git stash : répertoire propre, identique au dernier commit.

Git stash : les commandes (1/2)

- `git stash` : met de côté les modifications des fichiers suivis (équivalent de git stash push).
- `git stash push -m "message"` : ajoute une description au stash (recommandé).
- `git stash -u` : inclut aussi les fichiers non suivis ; -a inclut en plus les fichiers ignorés.
- `git stash list` : liste les stash : stash@{0} est le plus récent.
- `git stash show -p stash@{0}` : affiche le contenu (diff) d'un stash.

Git stash : les commandes (2/2)

- **`git stash pop`** : réapplique le dernier stash puis le supprime de la pile.
- **`git stash apply stash@{n}`** : réapplique un stash en le conservant dans la pile.
- **`git stash drop stash@{n}`** : supprime un stash ; git stash clear vide toute la pile.
- **`git stash branch nom_branche`** : crée une branche à partir du commit d'origine du stash, y applique le stash et le supprime.
- Conflit : si pop ou apply provoque un conflit, on le résout comme un merge ; avec pop, le stash n'est pas supprimé tant que le conflit n'est pas réglé (git stash drop ensuite).

Git stash : exemple et bonnes pratiques

```
$ git status                                  # modifications en cours sur la branche feature
$ git stash push -m "WIP login"               # travail mis de côté
$ git checkout master                         # répertoire de travail propre
  ... correction du bug, commit, push ...
$ git checkout feature
$ git stash pop                               # le travail en cours est restauré
```

- Local : un stash n'est pas envoyé par git push.
- Toujours décrire : utiliser -m pour retrouver facilement un stash.
- Temporaire : ne pas accumuler de stash ; pour un travail plus long, créer une branche.
- Vérifier : git stash list avant de pop ou de drop.

ANNULER ET CORRIGER (1/2)

Situation	Commande
Annuler les modifications d'un fichier non indexé	`git restore fichier` (ancien : `git checkout -- fichier`)
Retirer un fichier de l'index (garder les modifications)	`git restore --staged fichier` (ancien : `git reset fichier`)
Corriger le dernier commit (message ou contenu)	`git commit --amend`
Annuler un commit déjà poussé, sans réécrire l'historique	`git revert <commit>`
Ne plus suivre un fichier (sans le supprimer du disque)	`git rm --cached fichier`

`--amend` réécrit l'historique : à éviter sur un commit déjà poussé et partagé.

ANNULER ET CORRIGER (2/2) : RESET ET REFLOG

Commande	Commit	Modifications
`git reset --soft HEAD~1`	supprimé	conservées et indexées
`git reset HEAD~1` **(--mixed)**	supprimé	conservées, non indexées
`git reset --hard HEAD~1`	supprimé	**perdues** (destructif)

- **reset** déplace la branche : pour le travail **local**. `revert` ajoute un commit inverse : pour l'historique **partagé**.
- **git reflog :** journal des déplacements de `HEAD`, pour retrouver un commit « perdu » (après un `reset --hard` ou un rebase).

TAGS ET VERSIONS

```
$ git tag v1.0.0                          # tag léger
$ git tag -a v1.0.0 -m "Première version"   # tag annoté
$ git tag                                 # liste les tags
$ git push origin v1.0.0                  # envoie un tag (ou : git push --tags)
$ git checkout v1.0.0                     # se placer sur une version
```

- **Tag :** nom fixe donné à un commit, pour marquer une version.
- **Versionnement sémantique `MAJOR.MINOR.PATCH` :** MAJOR = rupture de compatibilité ; MINOR = ajout compatible ; PATCH = correction.
- **CI/CD :** un tag peut déclencher la publication d'une version (image Docker, release).

STRATÉGIES DE BRANCHES

Stratégie	Principe	Convient à
Trunk-based	tout le monde intègre souvent dans `main` ; branches très courtes	équipes avec une CI solide, déploiement continu
GitHub Flow	`main` toujours déployable ; une branche par fonctionnalité, Pull Request, merge, déploiement	applications web, livraisons fréquentes
GitFlow	`main`, `develop`, `feature/*`, `release/*`, `hotfix/*`	versions planifiées, plusieurs versions maintenues

> Nommage conseillé : `feature/PROJ-123-connexion`, `fix/PROJ-456-tva`.

PULL REQUEST ET REVUE DE CODE

- **Description :** quoi et pourquoi, lien vers le ticket (`PROJ-123`, `Closes #12`).
- **Revue :** les collègues relisent et commentent ; la **CI** doit être verte.
- **Protection de branche :** relectures et tests obligatoires avant de fusionner `main`.
- **Mode de fusion :** merge commit, **squash** (un seul commit) ou rebase.
- **Bonne pratique :** petites demandes, relues plus vite et mieux.

RÉCAPITULATIF : COMMANDES GIT ESSENTIELLES

Démarrer

- `git config`
- `git init`
- `git clone`
- `git remote add`

Au quotidien

- `git status`, `git diff`
- `git add`, `git commit`
- `git log`
- `git push`, `git pull`, `git fetch`

Branches et historique

- `git branch`, `git switch`
- `git merge`, `git rebase`
- `git cherry-pick`, `git stash`
- `git restore`, `git reset`, `git revert`
- `git tag`, `git reflog`

QUESTIONS DE RÉVISION : GIT

Q1. Quelle différence entre `git fetch` et `git pull` ?

Q2. Dans quel cas un merge est-il un fast-forward ?

Q3. Comment résoudre un conflit ? Que séparent les marqueurs `<<<<<<<`, `=======` et `>>>>>>>` ?

Q4. Pourquoi ne faut-il pas rebaser une branche déjà poussée et partagée ?

Q5. Quelle commande annule un commit déjà poussé sans réécrire l'historique ?

Planification agile avec Jira

Agilité, Scrum, Kanban et suivi du travail

L'AGILITÉ : LE MANIFESTE AGILE (2001)

Les individus et leurs interactions	plus que	les processus et les outils
Des logiciels opérationnels	plus que	une documentation exhaustive
La collaboration avec les clients	plus que	la négociation contractuelle
L'adaptation au changement	plus que	le suivi d'un plan

On valorise davantage les éléments de gauche, sans nier la valeur de ceux de droite.

En pratique : livrer souvent, par petits incréments, et recueillir des retours.

SCRUM : RÔLES ET ARTEFACTS

Rôles

- **Product Owner :** porte la valeur, priorise le Product Backlog.
- **Scrum Master :** facilite la méthode, lève les obstacles.
- **Équipe de développement :** réalise le travail, s'auto-organise.

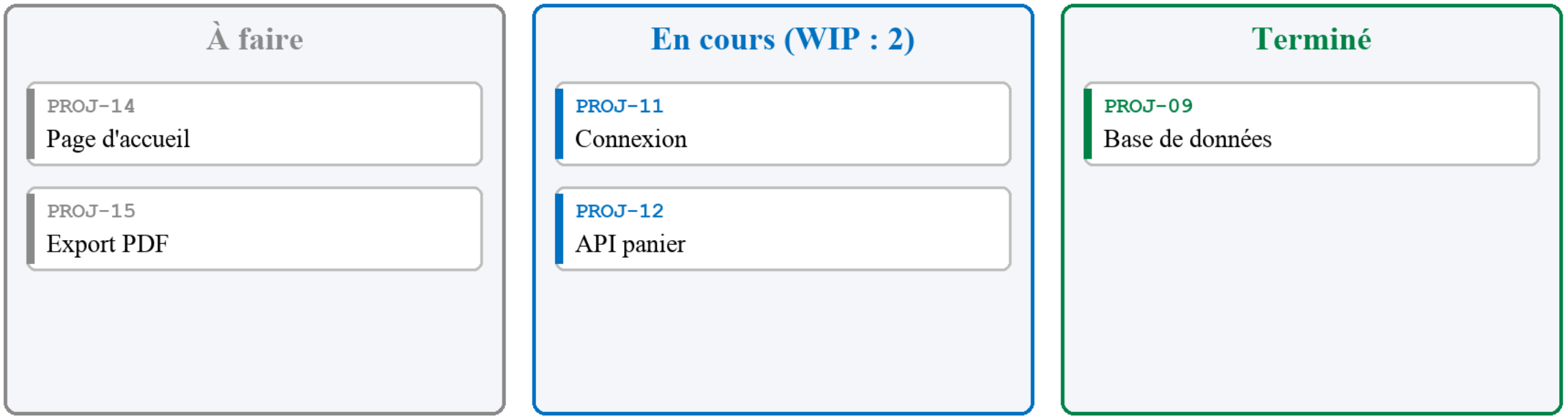
Artefacts

- **Product Backlog :** liste priorisée de tout ce qui est à faire.
- **Sprint Backlog :** ce que l'équipe traite pendant le sprint.
- **Incrément :** résultat utilisable à la fin du sprint, conforme à la **Definition of Done**.

SCRUM : LE CYCLE D'UN SPRINT

- **Sprint Planning :** choisir le travail dans le Product Backlog, fixer l'objectif du sprint.
- **Daily Scrum :** point quotidien de 15 minutes pendant le sprint.
- **Sprint Review :** démonstration de l'incrément aux parties prenantes, recueil des retours.
- **Rétrospective :** améliorer la façon de travailler de l'équipe.

KANBAN : VISUALISER ET LIMITER LE TRAVAIL



- **Visualiser :** le tableau montre où en est le travail.
- **Limiter le WIP :** peu de tickets en même temps, pour finir avant de commencer.
- **Flux continu :** pas d'itérations fixes, un ticket est tiré quand une place se libère.

SCRUM OU KANBAN ?

Critère	Scrum	Kanban
Cadence	sprints de durée fixe	flux continu
Rôles	Product Owner, Scrum Master, équipe	aucun rôle imposé
Changements	évités pendant le sprint	possibles à tout moment
Mesures	vélocité, burndown	lead time, cycle time, flux cumulé
Convient à	développement de produit par itérations	maintenance, support, demandes imprévisibles

JIRA : PRÉSENTATION

- **Jira Software (Atlassian) :** suivi de tickets et gestion de projets agiles.
- **Méthodes :** projets **Scrum** (sprints, backlog) ou **Kanban** (flux continu).
- **Déploiement :** cloud ou auto-hébergé ; offre gratuite pour les petites équipes.
- **Intégrations :** GitHub, GitLab, Bitbucket, Slack, Confluence.
- **Alternatives :** Trello, GitHub Projects, GitLab Issues, Linear.

LES CONCEPTS DE JIRA

Projet
regroupe le travail d'une équipe

Ticket (issue)
unité de travail : fonctionnalité, bug, tâche

Backlog
liste priorisée des tickets à faire

Sprint
période de travail (projet Scrum)

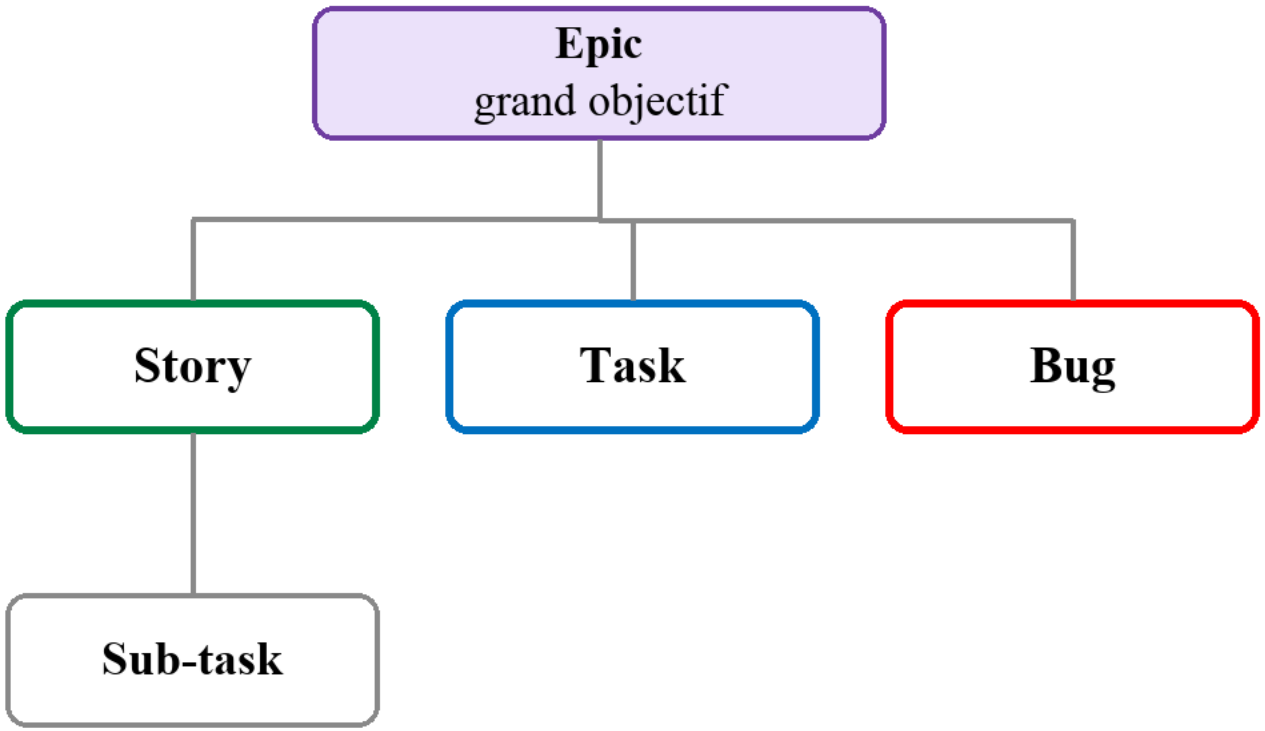
Board
tableau visuel, une colonne par statut

Workflow
statuts et transitions d'un ticket

Version (release)
tickets livrés ensemble

Filtres (JQL), dashboards
recherche et suivi

TYPES DE TICKETS ET HIÉRARCHIE



- **Epic :** grand objectif, découpé en stories.
- **Story :** besoin vu par l'utilisateur.
- **Task :** travail technique.
- **Bug :** défaut à corriger.
- **Sub-task :** étape d'un ticket.
- **Règle :** une story doit tenir dans un sprint, sinon on la découpe.

ANATOMIE D'UN TICKET

Champ	Rôle
Clé (`PROJ-123`)	identifiant unique : code du projet + numéro
Résumé, description	ce qu'il faut faire, pourquoi, comment le vérifier
Type, priorité	Story, Task, Bug... ; urgence du ticket
Responsable (assignee), rapporteur	qui le traite ; qui l'a créé
Statut	position dans le workflow (À faire, En cours, Terminé...)
Estimation, sprint, version	story points ; sprint prévu ; version livrée
Étiquettes, composants, liens	classer ; relier (« bloque », « est lié à »)

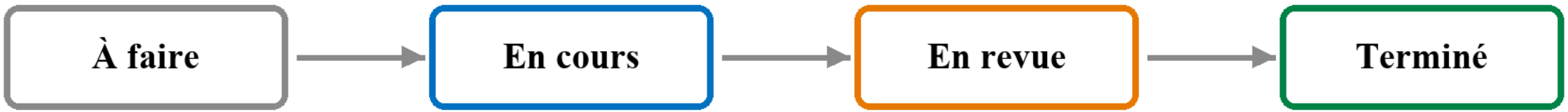
RÉDIGER UNE BONNE USER STORY

En tant que <rôle>, je veux <action> afin de <bénéfice>

« En tant qu'étudiant, je veux me connecter avec mon email afin d'accéder à mes cours. »

- **Critères d'acceptation :** comment savoir qu'elle est terminée ? Ex. : étant donné un compte valide, quand je saisis mes identifiants, alors j'accède à la page d'accueil.
- **INVEST :** Indépendante, Négociable, apportant de la Valeur, Estimable, Small (petite), Testable.
- **Definition of Done :** règles communes : code relu, tests passés, déployé en préproduction...

WORKFLOW JIRA : STATUTS ET TRANSITIONS

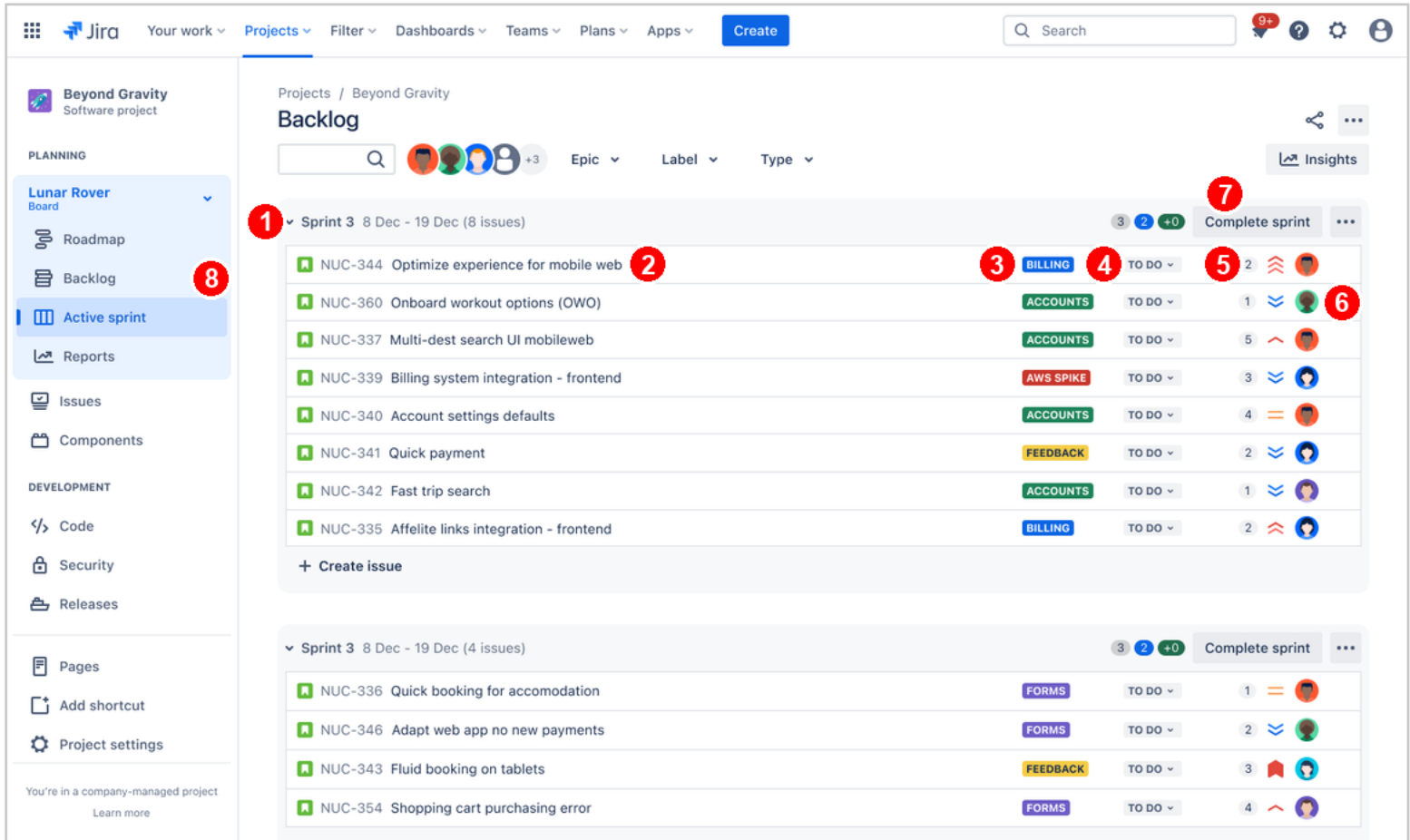


- **Workflow :** un ensemble de **statuts** et de **transitions**.
- **Par défaut :** À faire, En cours, Terminé ; personnalisable (En revue, En test...).
- **Board :** une colonne = un ou plusieurs statuts ; déplacer une carte déclenche la transition.
- **Règles :** on peut imposer des conditions (par exemple un responsable obligatoire).

BACKLOG ET SPRINTS

- **Backlog :** liste priorisée (haut = plus prioritaire), raffinée régulièrement par le Product Owner.
- **Estimation :** en **story points** (suite de Fibonacci : 1, 2, 3, 5, 8, 13...), décidée en équipe.
- **Sprint :** on y glisse des tickets du backlog, on fixe l'objectif et les dates.
- **Fin de sprint :** les tickets non terminés retournent au backlog ou passent au sprint suivant.

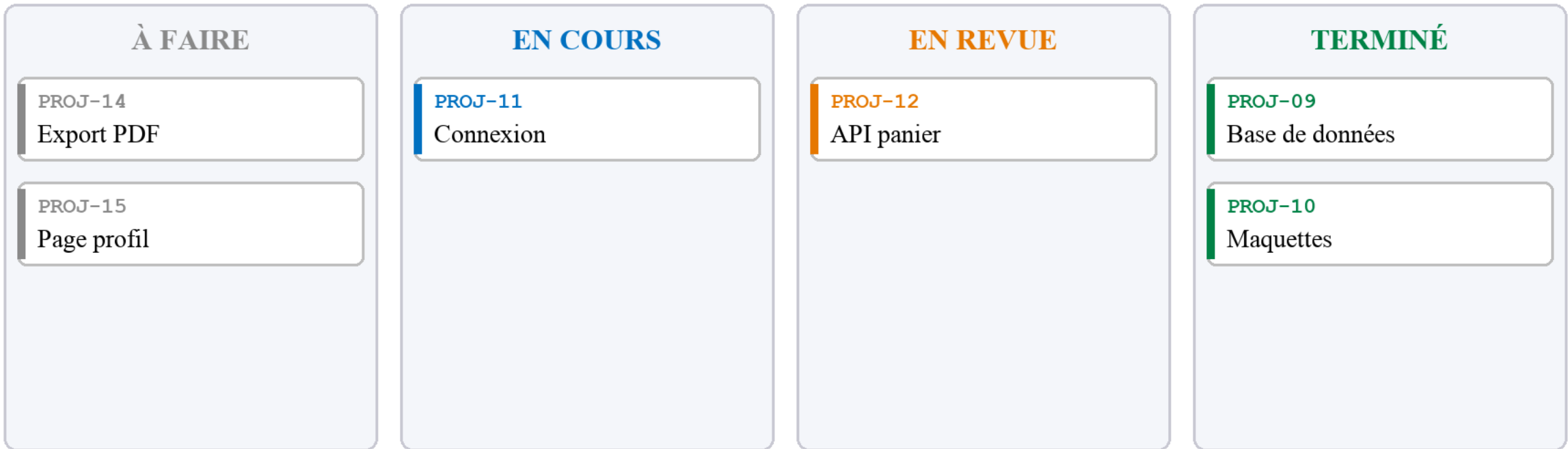
UN VRAI BACKLOG JIRA



Capture : Jira (Atlassian)

1. Sprint : nom, dates, nombre de tickets
2. Ticket : clé (NUC-344) et résumé
3. Étiquette (label) : classe le ticket
4. Statut du ticket (TO DO)
5. Estimation en story points
6. Priorité (icône) et responsable (avatar)
7. « Complete sprint » : clôture le sprint
8. Menu : Backlog, Active sprint (tableau), Reports

LE TABLEAU (BOARD)



- **Scrum :** tickets du sprint courant. **Kanban** : flux continu avec limites de WIP.
- **Glisser-déposer :** change le statut du ticket. **Swimlanes** : regrouper par responsable ou par epic.

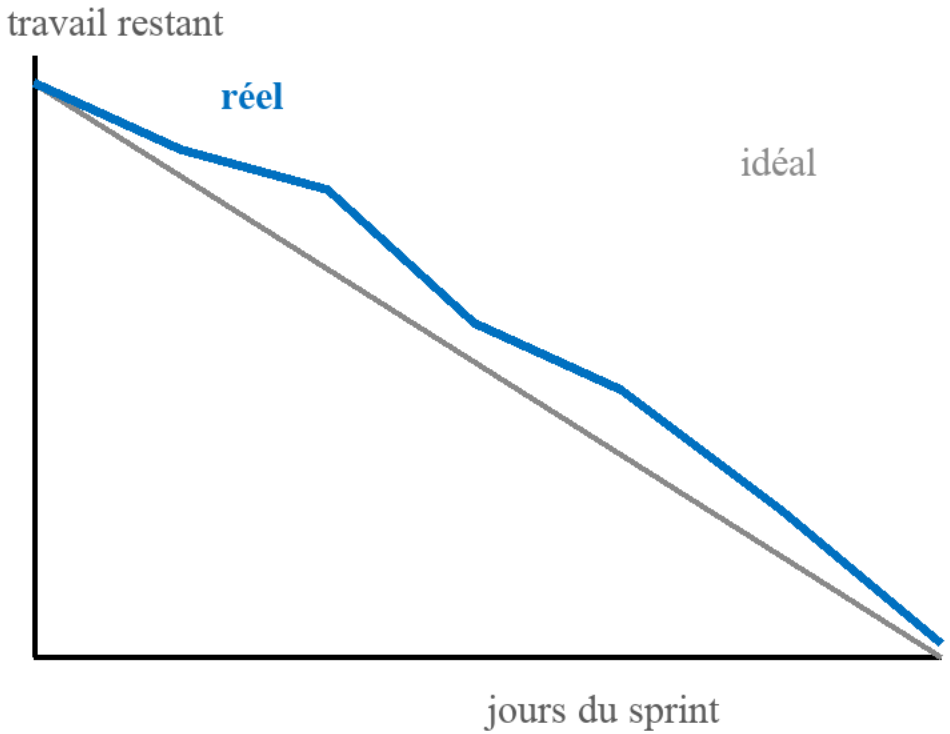
FILTRER AVEC JQL (JIRA QUERY LANGUAGE)

```
project = PROJ AND assignee = currentUser() AND statusCategory != Done

project = PROJ AND type = Bug AND priority = High ORDER BY created DESC

sprint in openSprints() AND status = "In Progress"

labels = urgent AND resolution = Unresolved
```

- **JQL :** champs, opérateurs, **`AND`** / **`OR`**, tri avec **`ORDER BY`**.
- **Filtre enregistré :** réutilisable dans un board ou un **dashboard**.

SUIVRE L'AVANCEMENT : LES RAPPORTS

- **Burndown :** travail restant du sprint, face à une courbe idéale.
- **Vélocité :** points livrés par sprint, pour prévoir la capacité.
- **Flux cumulé (Kanban) :** repérer les goulots d'étranglement.
- **Usage :** alimenter la rétrospective, pas juger les personnes.



JIRA ET GIT : RELIER LE CODE ET LES TICKETS

```
$ git checkout -b feature/PROJ-123-connexion
$ git commit -m "PROJ-123: ajoute le formulaire de connexion"
$ git push -u origin feature/PROJ-123-connexion
```

- **Clé du ticket** dans la branche, le commit et la Pull Request : le lien est créé automatiquement.
- **Le ticket affiche** branches, commits, Pull Requests, builds et déploiements.
- **Automatisation :** le ticket passe en « Terminé » quand la Pull Request est fusionnée.

BONNES PRATIQUES DE PLANIFICATION AGILE

- Des tickets **petits**, terminables en un sprint.
- Des critères d'acceptation écrits **avant** de commencer.
- Un backlog raffiné régulièrement et priorisé.
- Un travail en cours **limité** : finir avant de commencer.
- Une estimation faite en équipe, pas par une seule personne.
- Ne pas modifier le sprint en cours ; tenir les rétrospectives.

QUESTIONS DE RÉVISION : PLANIFICATION AGILE

Q1. Quels sont les trois rôles de Scrum ?

Q2. Quelle différence entre Product Backlog et Sprint Backlog ?

Q3. Quelles différences entre Scrum et Kanban ?

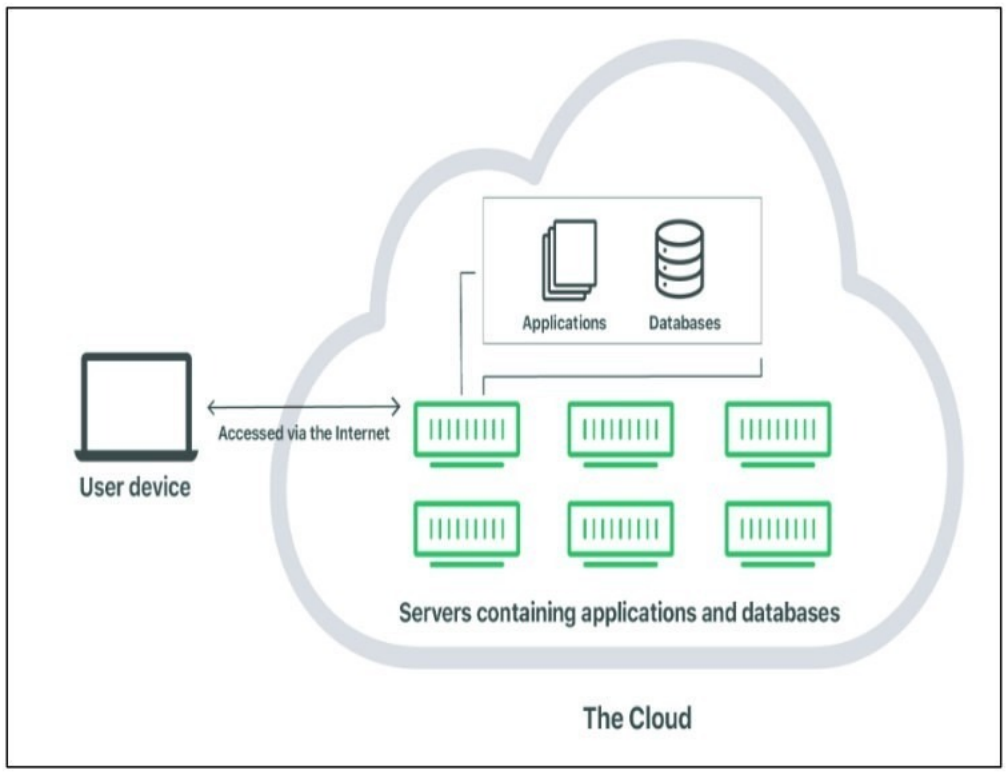
Q4. Quelle différence entre une story et une task ?

Q5. Comment relier un commit à un ticket Jira ?

DOCKER

DÉFINITION DU CLOUD

Le terme « cloud » désigne les serveurs accessibles sur Internet, ainsi que les logiciels et bases de données qui fonctionnent sur ces serveurs. Les serveurs situés dans le cloud sont hébergés au sein de datacenters répartis dans le monde entier. L'utilisation du cloud computing permet aux utilisateurs et aux entreprises de s'affranchir de la nécessité de gérer des serveurs physiques eux-mêmes ou d'exécuter des applications logicielles sur leurs propres équipements.



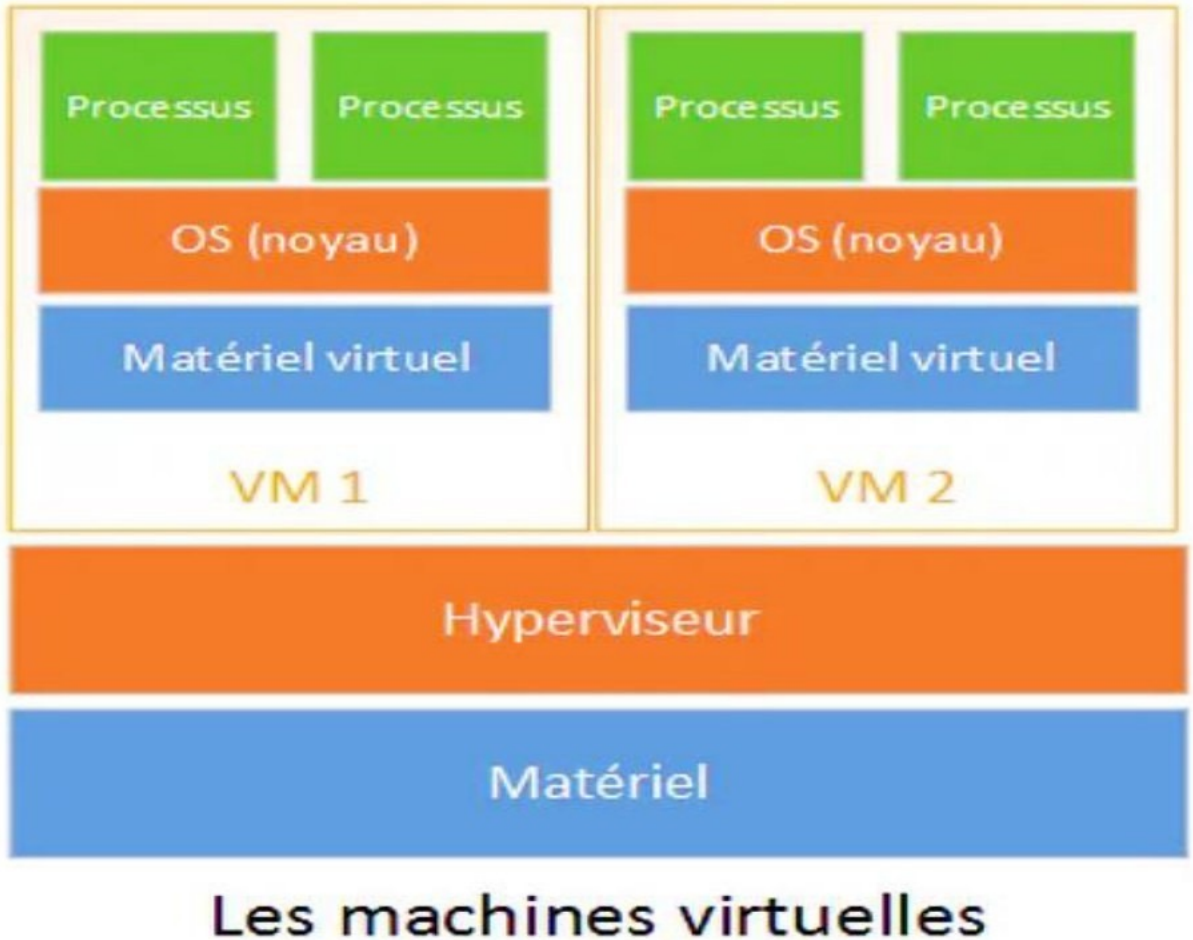
LA VIRTUALISATION

- La virtualisation est un mécanisme informatique qui consiste à faire fonctionner plusieurs systèmes, serveurs ou applications, sur un même serveur physique.
- La virtualisation est une technologie qui permet de créer et d'exécuter une version virtuelle des machines physiques, ainsi que des ressources associées.

LA VIRTUALISATION

La virtualisation est une technologie qui vous permet de ***créer plusieurs environnements simulés ou ressources dédiées à partir d'un seul système physique***.
Son logiciel, appelé ***hyperviseur***, est directement relié au matériel et vous permet de fragmenter ce système unique en plusieurs environnements sécurisés distincts. C'est ce que l'on appelle les machines virtuelles.

LA VIRTUALISATION



Les machines virtuelles

LA VIRTUALISATION ≠ CLOUD COMPUTING

Si le Cloud Computing s'appuie sur la virtualisation, les deux concepts diffèrent :

- la virtualisation permet de désolidariser les environnements informatiques de leurs machines. Comment? => *Chaque machine virtuelle dispose de ses propres ressources allouées, telles que la mémoire, le processeur, le stockage, etc*

- le Cloud Computing consiste à héberger et exploiter des données sur des serveurs distants, par le biais du réseau internet. Comment ? => *Cloud Computing vise à fournir des services informatiques à la demande sur Internet*

LE CLOUD COMPUTING

- Le cloud computing désigne un ensemble de principes et d'approches visant à ***mettre à la disposition d'utilisateurs qui en font la demande,*** quel que soit le réseau, des ressources de calcul, de réseau et de stockage, ainsi que des services, des plateformes et des applications.
- Ces ressources d'infrastructure, services et applications proviennent d'un cloud, c'est-à-dire ***d'un pool de ressources virtuelles orchestrées par des logiciels de gestion et d'automatisation et accessibles à la demande***.

LA VIRTUALISATION ≠ CLOUD COMPUTING

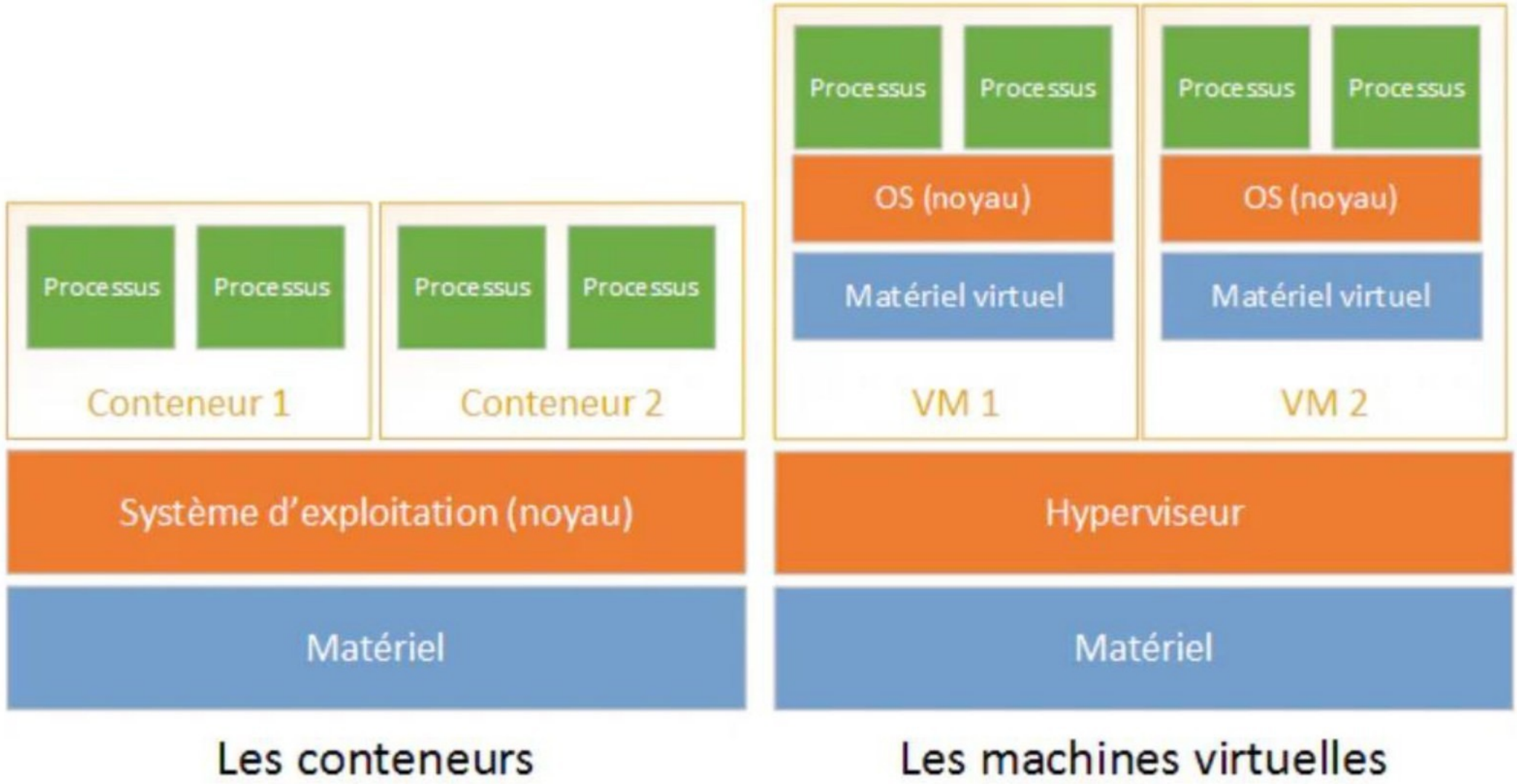
	Virtualisation	Cloud Computing
Objet	Créer plusieurs environnements simulés à partir d'un même système physique	Regrouper et automatiser des ressources virtuelles pour une utilisation à la demande
Utilisation	Fournir des ressources en paquets à des utilisateurs spécifiques pour une tâche spécifique	Fournir des ressources variables à des groupes d'utilisateurs pour diverses tâches
Configuration	À partir d'une image	A partir d'un modèle
Durée	Années (long terme)	Heure ou mois (court terme)

MACHINES VIRTUELLES ≠ CONTENEURS

- La notion de conteneur est aussi souvent rapprochée de celle de virtualisation. Mais une fois de plus, des différences existent.
- Comme son nom l'indique, ***une machine virtuelle est l'imitation virtuelle d'un appareil informatique créée, dans le cadre de la virtualisation, à l'aide d'un logiciel hyperviseur, et doté d'un système d'exploitation (ou OS) complet.***

- La virtualisation par conteneurisation, quant à elle, ***consiste à cloisonner directement au niveau du système d'exploitation.*** Ainsi, chaque conteneur exécute son environnement, ***mais partage le même OS hôte***. C'est pour cette raison que les conteneurs servent généralement à la virtualisation d'un programme, et non d'un serveur dans son intégralité

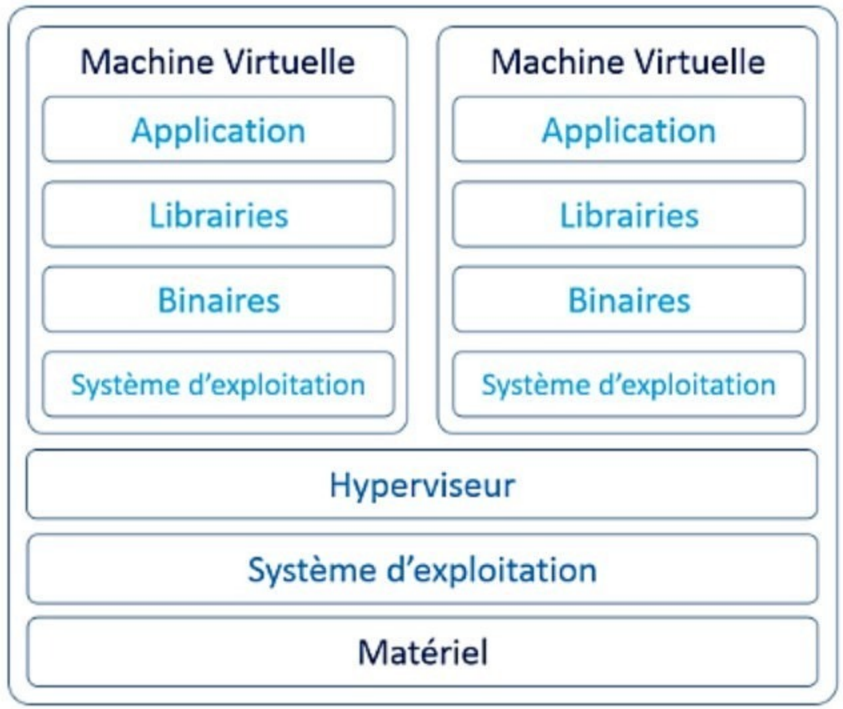
MACHINES VIRTUELLES ≠ CONTENEURS

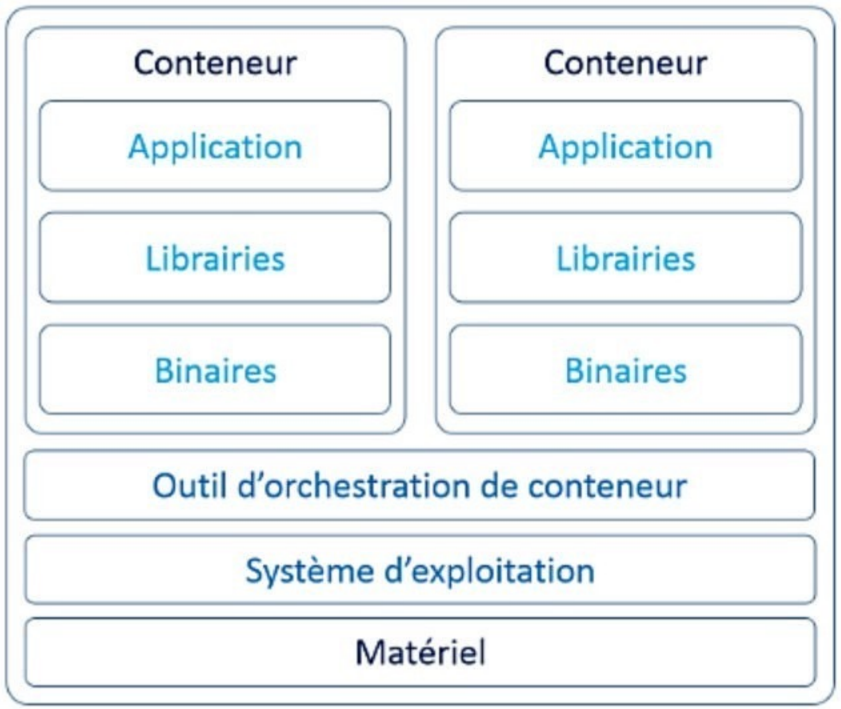


MACHINES VIRTUELLES ≠ CONTENEURS

Machine Virtuelle	Dans le cas de la virtualisation traditionnelle avec des machines virtuelles, chaque machine virtuelle dispose de son propre système d'exploitation. Ainsi, lors de l'exécution d'applications intégrées à des machines virtuelles, l'utilisation de la mémoire peut être supérieure à ce qui est nécessaire et les machines virtuelles peuvent commencer à utiliser les ressources requises par l'hôte.
Conteneur	Contrairement aux applications classiques, les applications conteneurisées partagent un environnement de système d'exploitation (noyau), elles utilisent donc moins de ressources que des machines virtuelles complètes et réduisent la pression sur la mémoire de l'hôte

MACHINES VIRTUELLES ≠ CONTENEURS

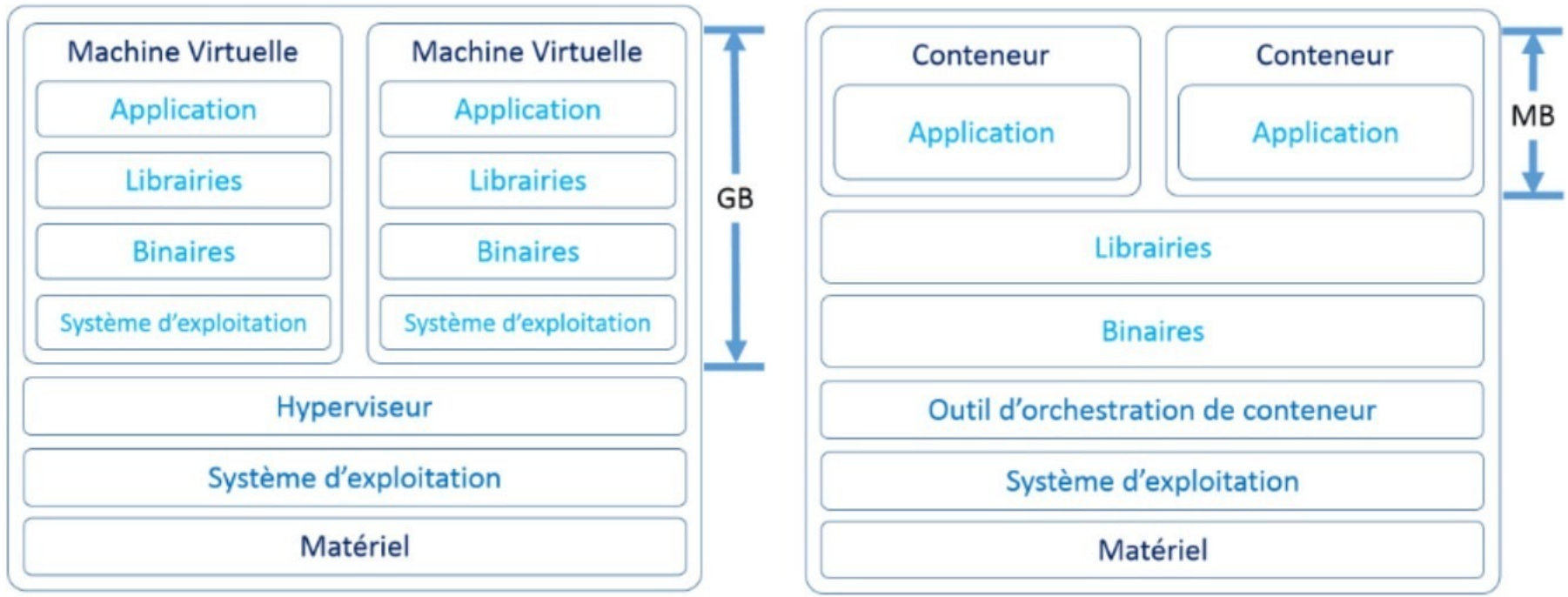




MACHINES VIRTUELLES ≠ CONTENEURS

Machine Virtuelle	Les machines virtuelles traditionnelles peuvent occuper beaucoup d'espace disque: elles contiennent un système d'exploitation complet et les outils associés, en plus de l'application hébergée par la machine virtuelle.
Conteneur	Les conteneurs sont relativement légers: ils ne contiennent que les bibliothèques et les outils nécessaires à l'exécution de l'application conteneurisée. Ils sont donc plus compacts que les machines virtuelles et démarrent plus rapidement.

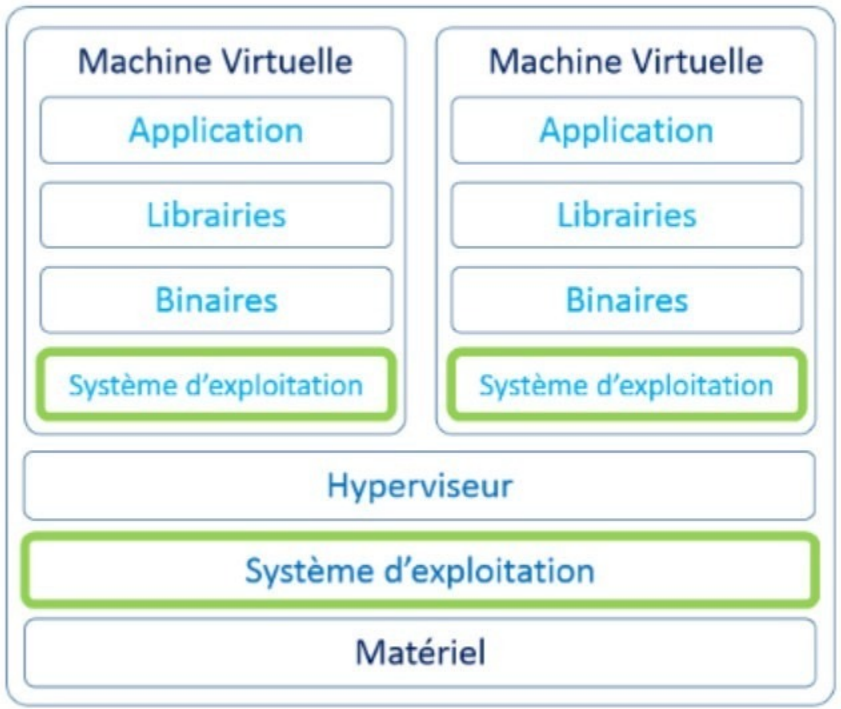
MACHINES VIRTUELLES ≠ CONTENEURS



MACHINES VIRTUELLES ≠ CONTENEURS

Machine Virtuelle	En ce qui concerne la mise à jour ou la correction du système d'exploitation, les machines traditionnelles doivent être mises à jour une par une: chaque système d'exploitation invité doit être corrigé séparément.
Conteneur	Avec les conteneurs, seul le système d'exploitation de l'hôte du conteneur (la machine hébergeant les conteneurs) doit être mis à jour. Cela simplifie considérablement la maintenance.

MACHINES VIRTUELLES ≠ CONTENEURS



AVANTAGES DES CONTENEURS

- **Performances :** pas d'hyperviseur ni d'OS invité ; un conteneur est un processus sur le noyau de l'hôte, avec une faible surcharge.
- **Démarrage rapide :** pas de système complet à démarrer.
- **Images légères :** uniquement l'application et ses dépendances ; moins de stockage, transfert plus rapide.
- **Portabilité :** la même image tourne sur le poste du développeur, en test et dans le cloud.
- **Reproductibilité :** fin du « ça marche sur ma machine ».

COMMENT FONCTIONNE UN CONTENEUR ?

Namespaces

- Isolent ce que le processus **voit**
- Processus, réseau, système de fichiers, nom de machine, utilisateurs

Cgroups

- Limitent ce que le processus **consomme**
- CPU, mémoire, entrées-sorties

Système de fichiers union

- Combine les couches de l'image en lecture seule
- Plus une couche en écriture par conteneur

- **Noyau partagé :** isolation plus faible qu'une VM.
- **Mac et Windows :** Docker Desktop fait tourner une petite VM Linux.

docker

DEFINITION

Docker est une plateforme de conteneurs ayant largement contribué à la démocratisation de la conteneurisation.
Elle permet de créer facilement des conteneurs et des applications basées sur les conteneurs. Il en existe d'autres, mais celle-ci est la plus utilisée. Elle est par ailleurs plus facile à déployer et à utiliser que ses concurrentes.

Il s'agit d'une plateforme logicielle open source permettant de

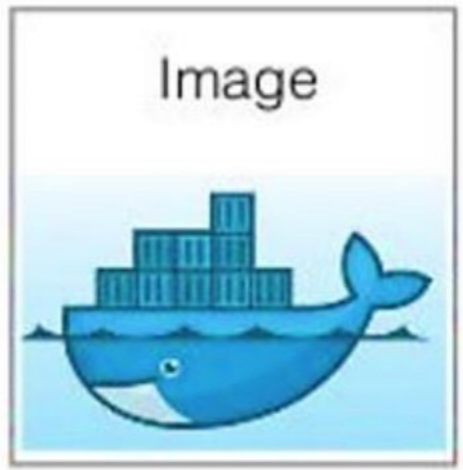
- Créer,
- Déployer
- Gérer des containers d'applications virtualisées sur un système d'exploitation.

DEFINITION

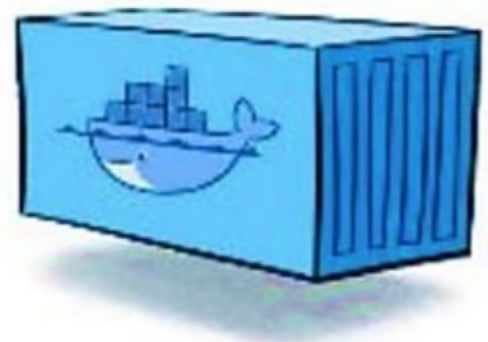
Docker est composé de trois éléments :

- le daemon Docker qui s'exécute en arrière-plan et qui s'occupe de gérer les conteneurs (Containerd avec runC)
- une API de type REST qui permet de communiquer avec le daemon
- Le client en CLI (command line interface) : commande docker

build



run



Dockerfile

Docker Image

Docker Container

Fichier DOCKER

Il s'agit d'un fichier texte simple contenant une collection de commandes ou de procédures. Ces commandes et directives que nous exécutons agissent sur l'image de base configurée pour créer une nouvelle image Docker.

Un Dockerfile est le code source de l'image Docker. Un Dockerfile est un fichier texte contenant diverses instructions et configurations. La commande FROM dans un Dockerfile identifie l'image de base à partir de laquelle vous construisez.

LES IMAGES DOCKER

- **Un modèle en lecture seule :** le système de fichiers de l'application (binaire, bibliothèques, configuration) et des métadonnées (commande par défaut, ports...).
- **Des couches :** chaque instruction du Dockerfile ajoute une couche.
- **Un nom :** `nom:tag` (exemple `nginx:1.27`) et un digest SHA-256.
- **D'où elle vient :** construite avec `docker build`, ou téléchargée avec `docker pull`.

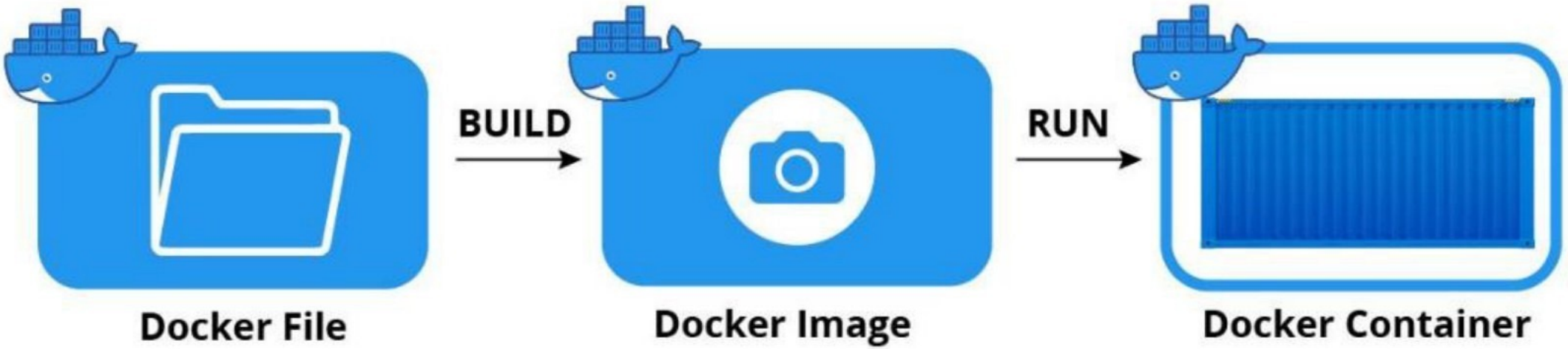
> Une image est à un conteneur ce qu'une classe est à un objet (ou une recette à un plat).

LES CONTENEURS

- **Une instance d'une image :** un processus isolé avec son système de fichiers, son réseau et ses ressources.
- **Plusieurs conteneurs** peuvent être créés à partir de la même image.
- **Couche en écriture :** ajoutée au-dessus de l'image, **supprimée avec le conteneur**.
- **Données durables :** à stocker dans des **volumes**.
- **Éphémère :** conçu pour être jetable et remplaçable.

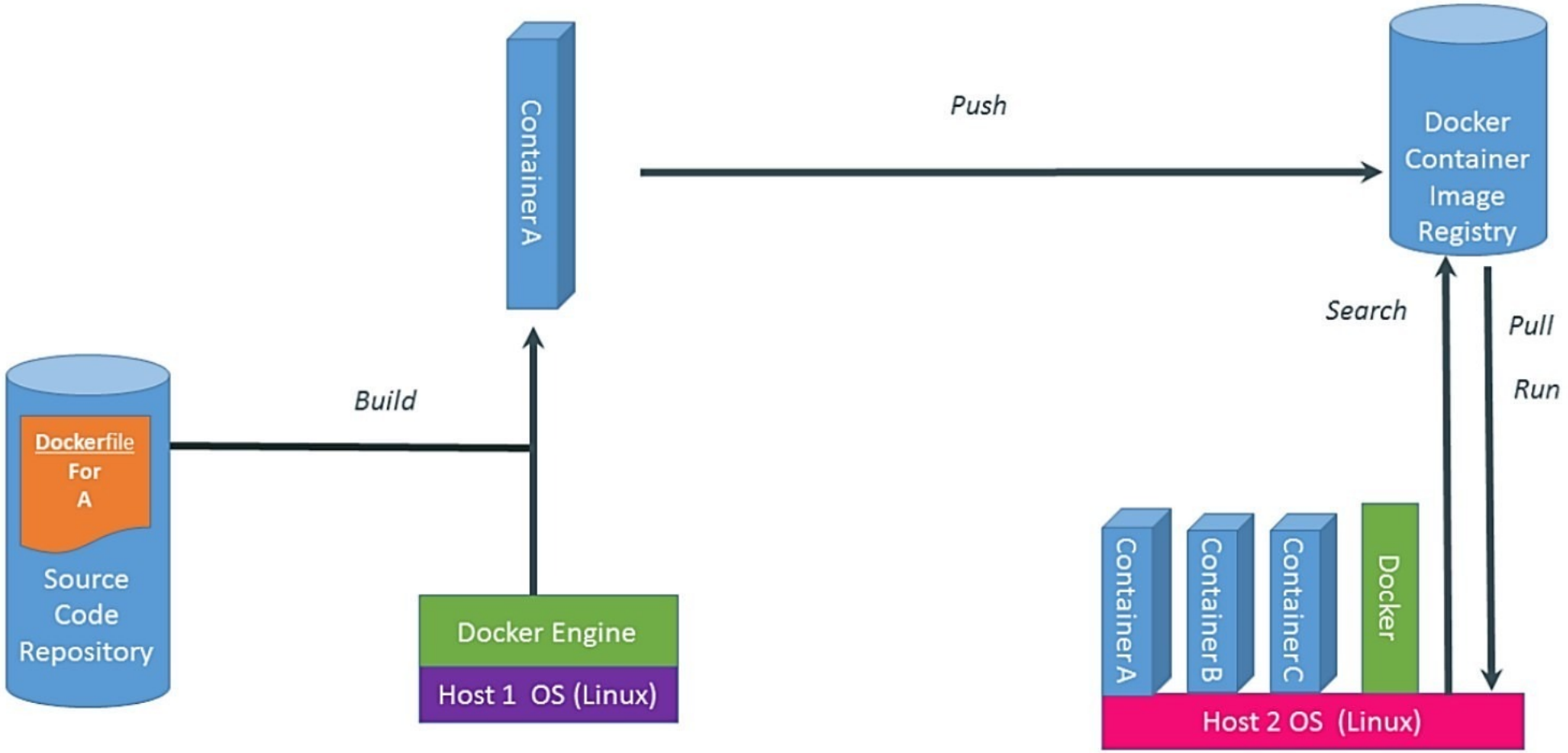
DOCKER HUB

Docker HUB : Dépôt public d’images mises à disposition par Docker



•Créez un fichier Docker et incluez les instructions pour créer votre image Docker.
•Exécutez la commande docker build pour créer votre image Docker.
•Utilisez la commande docker run pour créer des conteneurs maintenant que l'image docker est prête à être utilisée.

PRINCIPES DE FONCTIONNEMENT DE DOCKER



DOCKER: LES BRIQUES PRINCIPALES

Docker engine

Un environnement d'exécution et un ensemble de services pour manipuler des conteneurs docker sur une machine.

Une application client-serveur

Le serveur -- Un daemon (processus persistant) qui gère les conteneurs sur une machine

Le client -- Une interface en ligne de commande

Un/des registres d'images docker

Bibliothèque d'images disponibles

Docker Hub

PRINCIPES DES IMAGES DOCKER

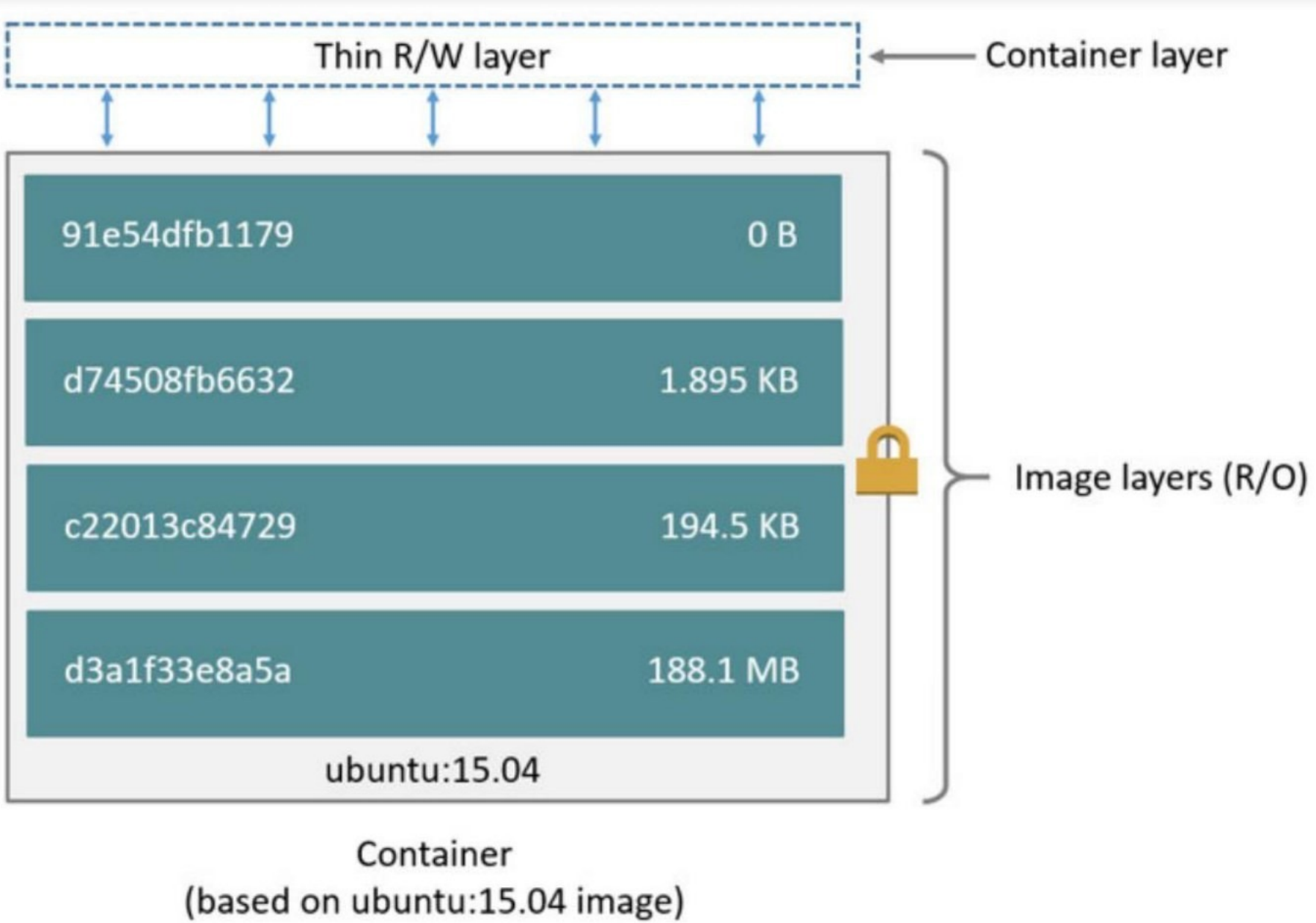
Fondé sur l'utilisation d'un Union File System

- Crée la vision d'un système de fichier cohérent à partir de fichiers/répertoires appartenant à des systèmes de fichiers différents

Un ensemble de couches

- Une image est composée d'un ensemble de couches (layers)
- L'Union File System est utilisé pour combiner ces couches
- Le file system utilisé par défaut s'appelle overlay2
- Chaque couche correspond à une instruction dans le fichier Dockerfile décrivant l'image.

Un système de fichiers Union, dans le contexte de Docker et de la conteneurisation, est une technologie qui permet de combiner plusieurs couches de systèmes de fichiers en une seule vue unifiée.


Thin R/W layer
Container layer
91e54dfb1179
0 B
d74508fb6632
1.895 KB
c22013c84729
194.5 KB
d3a1f33e8a5a
188.1 MB
ubuntu:15.04
Image layers (R/O)
Container
(based on ubuntu:15.04 image)

DOCKER IMAGE LAYERS

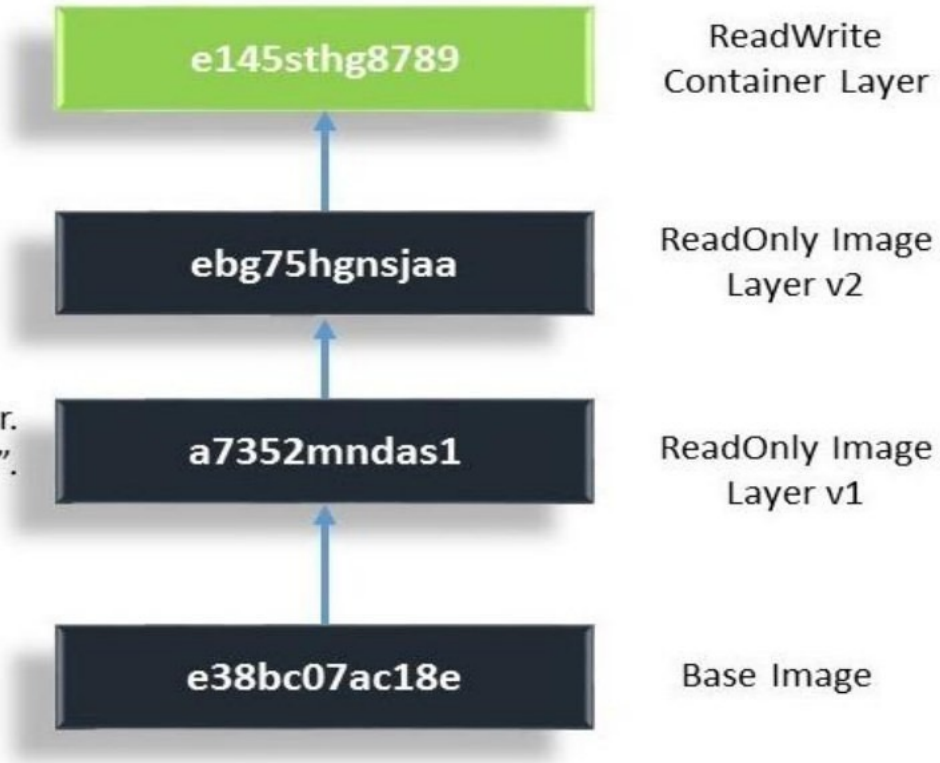
www.learnitguide.net

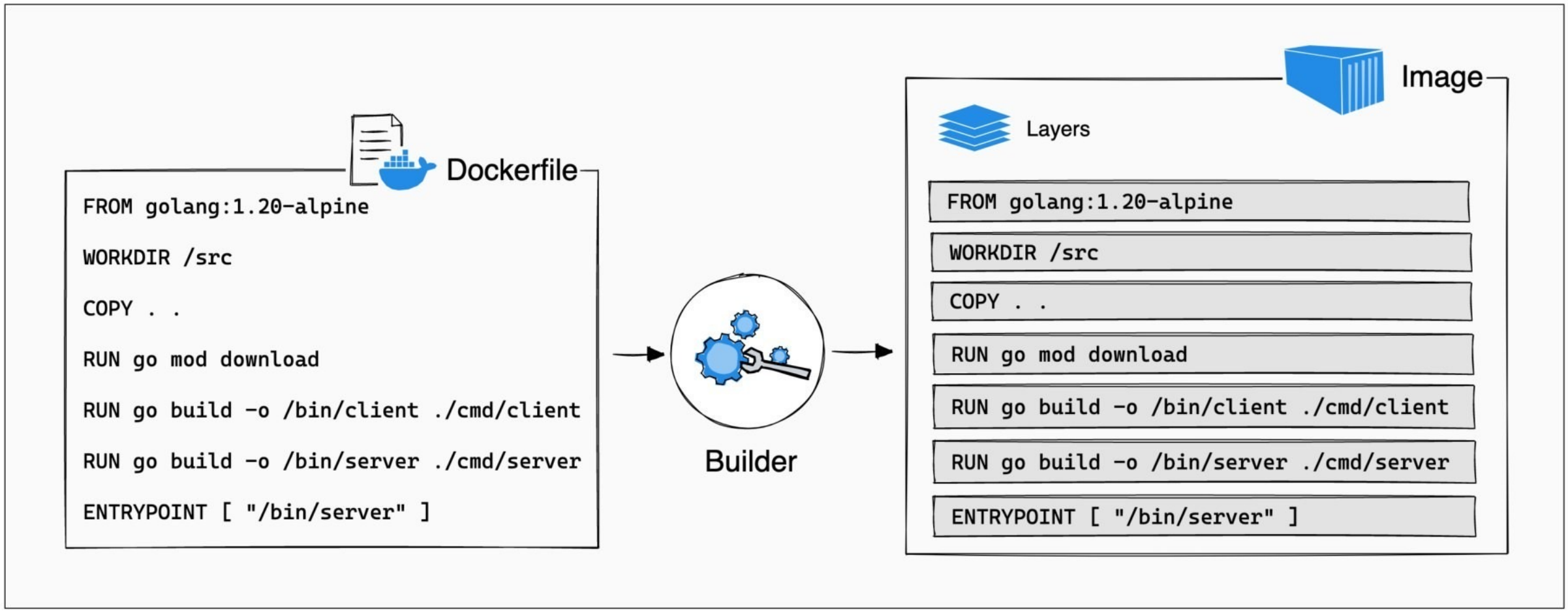
1. Started Image Layer 3 as a container and accessible by users

1. Started Image Layer v1 as a container.
2. Installed and Configured https web server.
3. Committed new layer v2

1. Started Base Image (**docker.io/centos**) as a container.
2. Package Updated on Base Image using “yum update”.
3. Committed new layer v1

Pulled CentOS image from Docker Hub using docker pull command. **Repo: docker.io/centos**




Dockerfile
FROM golang:1.20-alpine
WORKDIR /src
COPY . .
RUN go mod download
RUN go build -o /bin/client ./cmd/client
RUN go build -o /bin/server ./cmd/server
ENTRYPOINT ["/bin/server"]
Builder
Image
Layers
FROM golang:1.20-alpine
WORKDIR /src
COPY . .
RUN go mod download
RUN go build -o /bin/client ./cmd/client
RUN go build -o /bin/server ./cmd/server
ENTRYPOINT ["/bin/server"]

EXEMPLE DE DOCKERFILE

```
FROM node:22-alpine              # image de base légère, version fixée
WORKDIR /app                     # répertoire de travail
COPY package*.json ./            # d'abord les dépendances (cache)
RUN npm ci --omit=dev            # installation des dépendances
COPY . .                         # puis le code de l'application
EXPOSE 3000                      # port documenté
USER node                        # ne pas tourner en root
CMD ["node", "server.js"]        # commande de démarrage
```

```
$ docker build -t monapp:1.0 .
$ docker run -d -p 3000:3000 monapp:1.0
```

- **Chaque instruction** = une couche ; l'ordre compte pour le cache.

Les autres couches

- Les couches correspondent aux différentes modifications qui sont faites pour construire l'image à partir de l'image de base.
- Pour sauvegarder une nouvelle image, il suffit de sauvegarder les nouvelles couches qui ont été créées au dessus de l'image de base.
- Dans un conteneur en cours d'exécution, il existe une couche supplémentaire accessible en écriture
- Toutes les écritures vers le système de fichier faites à l'exécution du conteneur sont stockées dans cette couche.
- Les autres couches, définies au sein de l'image utilisée pour instancier le conteneur, ne sont accessibles qu'en lecture.
- Cette couche est supprimée à la suppression du conteneur

Optimisation des performances

- Partage des couches identiques entre conteneurs diminue l'espace de stockage utilisé par les conteneurs.

Affichage de l'ensemble des couches d'une image

```
$ docker history image_name
```

Avantages liés aux mécanismes de couches

Chaque couche est stockée une seule fois localement

Si certaines couches nécessaires pour une image à télécharger sont déjà présentes, pas besoin de les télécharger à nouveau.

Réduction de l'espace de stockage

Optimisations à l'exécution

Démarrage rapide

Démarrer un conteneur nécessite simplement de créer la couche accessible en écriture

Faible utilisation de l'espace disque

Si plusieurs containers sont instanciés à partir de la même image, ils partagent les couches en lecture seule.

Identification des couches/images

Dans la version courante de Docker :

- Chaque couche est identifiée par un **digest** qui est un hash du contenu de la couche
- Une image est définie par un ensemble de metadonnées (principalement la liste ordonnée des identifiants des couches constituant l'image)
- L'identifiant de l'image est créé en calculant un hash de ces metadonnées

CRÉER DES IMAGES

2 manières

- **Interactivement**
- **En utilisant un Dockerfile**

Avant de commencer: Premiers pas avec docker

L'outil à la ligne de commande pour exécuter des commandes Docker :

```
$ docker
```

```
docker run hello-world
```

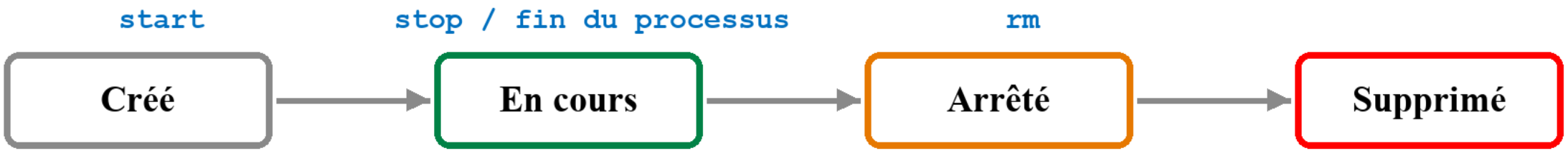
Docker : Nous voulons exécuter une commande docker
Run : Commande pour créer et exécuter un conteneur docker
hello-world: Nom de l'image à partir de laquelle est construit le conteneur

docker pull: télécharge une image explicitement
docker run: Commande servant à créer un conteneur à partir d'une image. Télécharge l'image si elle n'est pas présente localement.

Que va-t-il se passer?

L'image à charger est hello-world:latest
Vérification: est ce que l'image est présente localement?
Sinon, télécharger l'image depuis Docker Hub
Charger l'image dans le conteneur et exécuter la commande par défaut définie pour ce conteneur

CYCLE DE VIE D'UN CONTENEUR



- `docker create` crée le conteneur ; `docker run` = `create` + `start`.
- **Arrêté** ne veut pas dire supprimé : il apparaît dans `docker ps -a` jusqu'au `docker rm`.
- **Un conteneur vit tant que son processus principal tourne :** un `bash` lancé avec `-d` s'arrête aussitôt.

COMMANDES DOCKER ESSENTIELLES

Commande	Rôle
`docker run [options] image`	crée et démarre un conteneur
`docker ps` / `docker ps -a`	conteneurs en cours / tous les conteneurs
`docker stop|start|restart nom`	arrêter, démarrer, redémarrer
`docker logs [-f] nom`	afficher (suivre) les journaux
`docker exec -it nom sh`	ouvrir un shell dans un conteneur en cours
`docker rm [-f] nom`	supprimer un conteneur
`docker images` / `docker rmi`	lister / supprimer des images
`docker inspect nom`	détails : IP, volumes, configuration
`docker system prune`	nettoyer ce qui est inutilisé

LES OPTIONS DE DOCKER RUN

Option	Effet
`-d`	démarre en arrière-plan (mode détaché)
`-it`	mode interactif avec un terminal
`--name nom`	donne un nom au conteneur (sinon nom aléatoire)
`--rm`	supprime le conteneur dès qu'il s'arrête
`-p hôte:conteneur`	publie un port du conteneur sur l'hôte
`-e VAR=valeur`	définit une variable d'environnement
`-v source:destination`	monte un volume ou un dossier de l'hôte

PORTS ET VARIABLES D'ENVIRONNEMENT

```
$ docker run -d --name web -p 8080:80 nginx      # http://localhost:8080
$ docker run -d --name db -e MYSQL_ROOT_PASSWORD=secret mysql:8
$ docker port web 80                             # port hôte attribué
```

- **`-p 8080:80`** : port 8080 de l'hôte vers port 80 du conteneur ; **`-p 80`** : port hôte choisi par Docker.
- **`EXPOSE`** documente un port dans le Dockerfile, mais ne le **publie** pas.
- **`-e` / `--env-file`** : configuration injectée au démarrage ; pas de mot de passe en dur dans l'image.
- **Réseau :** des conteneurs sur un même réseau Docker se joignent par leur nom (c'est ce que fait Compose).

VOLUMES ET PERSISTANCE DES DONNÉES

La couche en écriture d'un conteneur disparaît avec lui : les données durables vont dans un volume.

Type	Syntaxe	Usage
Volume nommé	`-v data:/var/lib/mysql`	données (base de données) ; géré par Docker, recommandé
Bind mount	`-v ~/site:/var/www/html`	dossier de l'hôte ; pratique en développement (TP4)
tmpfs	`--tmpfs /tmp`	stockage en mémoire, temporaire

- **Commandes :** `docker volume ls`, `inspect`, `rm`, `prune`.

Construire une image interactivement

Objectifs

Créer une image à partir d'une image de base dans laquelle nous allons installer cowsay

Les étapes

1. Créer un conteneur à partir de l'image de base
2. Installer le logiciel manuellement dans le conteneur et en faire une nouvelle image
3. Jouer avec:
    docker commit
    docker tag
    docker diff

Configuration du conteneur

Démarrer un conteneur Ubuntu

```
$ docker run -it ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
ea362f368469: Pull complete
Digest: sha256:b5a61709a9a44284d88fb12e5c48db0409cfad5b69d4ff8224077c57302df9cf
Status: Downloaded newer image for ubuntu:latest
root@f461e2e7afff:/#
```

f461e2e7afff est l'id du conteneur créé

Installer le programme dans le conteneur

```
root@f461e2e7afff:/# apt-get update
root@f461e2e7afff:/# apt-get install cowsay
```

Configuration du conteneur

Quitter la session interactive

```
root@f461e2e7afff:/# exit
```

Inspecter les changements

```
$ docker diff f461e2e7afff
C /var
C /var/log
C /var/log/apt
C /var/log/apt/history.log
A /var/log/apt/term.log
C /var/log/apt/eipp.log.xz
C /var/log/dpkg.log
...
```

C: fichier ou répertoire modifié
A: fichier ou répertoire ajouté

Sauvegarder les changements dans une nouvelle image

```
$ docker commit <yourContainerId>
<newImageId>
```

Exécution de la nouvelle image

```
$ docker run -it <newImageId>
root@7267696dc8c6:/# /usr/games/cowsay bonjour
... ca marche
```

Tagger une image

On peut tagger une image pour lui associer un nom (plus facile à manipuler qu'un identifiant)

```
$ docker tag <newImageId> cowsay
```

Il faut maintenant automatiser le processus!

Les Dockerfile

- Un Dockerfile est une recette décrivant comment construire une image
- Contient une suite d'instructions
- La commande docker build permet de créer une image à partir d'un Dockerfile

ÉLÉMENTS DE SYNTAXE D'UN DOCKERFILE

Instruction	Rôle
`FROM`	image de base à partir de laquelle on construit
`LABEL`	métadonnées (auteur, version...) ; remplace `MAINTAINER`
`RUN`	commande exécutée **à la construction** (crée une couche)
`COPY` / `ADD`	copie des fichiers du contexte de build vers l'image (préférer `COPY`)
`WORKDIR`	répertoire de travail pour les instructions suivantes
`ENV` / `ARG`	variable d'environnement (reste dans l'image) / variable de build seulement
`EXPOSE`	documente le port d'écoute du conteneur
`USER`	utilisateur qui exécute le processus (éviter root)
`VOLUME`	déclare un point de montage pour des données
`CMD` / `ENTRYPOINT`	commande exécutée **au démarrage** du conteneur

RUN, CMD ET ENTRYPOINT

Instruction	Quand	Remplaçable avec docker run ?
`RUN`	à la construction de l'image	non
`CMD`	au démarrage du conteneur	oui : `docker run image autre-commande`
`ENTRYPOINT`	au démarrage, programme fixe	ses arguments sont ajoutés (remplacement avec `--entrypoint`)

```
ENTRYPOINT ["/usr/sbin/apache2"]      # le programme
CMD ["-D", "FOREGROUND"]              # ses arguments par défaut
```

- **Forme exec `["a", "b"]`** (recommandée) ; la forme shell `a b` lance un `/bin/sh -c`.

Notre premier Dockerfile

1- La création d'un Dockerfile doit se faire dans un nouveau répertoire vide

```
mkdir my_image
```

2- Créer le Dockerfile

```
FROM ubuntu
RUN apt-get update
RUN apt-get -y install cowsay
```

3- Construire notre image

```
$ docker build -t cowsay .
```

-t permet de tagger avec un nom l'image qui va être créée
. indique le contexte de construction de l'image (où se trouve le Dockerfile)

4- Que se passe-t-il?

```
$ docker build -t cowsay .
Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu
 ---> d13c942271d6
Step 2/3 : RUN apt-get update
 ---> Running in 80f5510281d9
Removing intermediate container 80f5510281d9
 ---> cb1643c4393c
Step 3/3 : RUN apt-get -y install cowsay
 ---> Running in 01834650511b
Removing intermediate container 01834650511b
 ---> 9ca55c5ccc54
Successfully built 9ca55c5ccc54
Successfully tagged cowsay:latest
```

Le build context est envoyé vers le démon docker (contenu du répertoir .)
A chaque étape:

- Un conteneur est créé pour exécuter l'étape (Running in ...)
- Les modifications sont committées dans une nouvelle image (---> ...)
- Le conteneur est supprimé
- La nouvelle image est utilisée pour la prochaine étape

5- Construction de l'image

docker build
Crée une image à partir d'un ficher Dockerfile

```
docker build -t docker-whale .
```

Crée l'image docker-whale avec le contexte correspondant au répertoire courant (le '.' est nécessaire)
Par défaut, le Dockerfile est cherché à la racine du contexte

BONNES PRATIQUES DOCKERFILE

- **Image de base :** légère (`alpine`, `slim`), avec un tag **fixe** (pas `latest`).
- **Cache :** dépendances d'abord, code ensuite.
- **Moins de couches :** regrouper les `apt-get update && apt-get install ...` dans un seul `RUN`.
- `.dockerignore` **:** exclure `.git`, `node_modules`, `.env` du contexte de build.
- **Sécurité :** aucun secret dans l'image ; ne pas tourner en root (`USER`).
- **Un processus principal** par conteneur.
- **BuildKit :** builder par défaut des Docker récents ; sortie du build différente des captures précédentes.

BUILD MULTI-ÉTAPES (MULTI-STAGE)

```
FROM node:22 AS build                    # étape 1 : compiler
WORKDIR /app
COPY . .
RUN npm ci && npm run build

FROM nginx:alpine                        # étape 2 : image finale minimale
COPY --from=build /app/dist /usr/share/nginx/html
```

- **Plusieurs `FROM` :** chaque étape repart d'une image.
- **`COPY --from=build` :** récupère uniquement le résultat de l'étape précédente.
- **Image finale** plus **petite** et plus **sûre** : ni compilateur ni code source.

REGISTRES ET TAGS D'IMAGES

```
$ docker login --username=monuser
$ docker tag monapp:1.0 monuser/monapp:1.0      # nommer l'image
$ docker push monuser/monapp:1.0                # publier dans le registre
$ docker pull monuser/monapp:1.0                # récupérer
```

- **Registre :** serveur d'images (Docker Hub, GHCR, registre privé).
- **Nom complet :** `registre/utilisateur/image:tag`.
- `latest` : tag par défaut, **pas** une garantie de dernière version : en production, utiliser des tags précis (`1.0.3`).

DOCKER-COMPOSE

- On sait créer des images
  - de manière manuelle
  - de manière automatisée
- On sait lancer des conteneurs

PROBLÈMES :

- On veut coordonner des conteneurs
- On veut simplifier la gestion multi-conteneurs

DOCKER-COMPOSE

COMMENT FAIRE POUR

- Gérer les deux conteneurs à la volée?
- Gérer des volumes?
- Gérer des ports différents?
- Me souvenir de ces commandes?

DOCKER-COMPOSE

SOLUTION

- Compose vous permet d'éviter de gérer individuellement des conteneurs qui forment les différents services de votre application.
- Outil qui définit et exécute des applications multi-conteneurs
- Utilise un fichier de configuration dans lequel vous définissez les services de l'application
- A l'aide d'une simple commande, vous contrôlez le cycle de vie de tous les conteneurs qui exécutent les différents services de l'application.

DOCKER-COMPOSE

UTILISATION

- Vous définissez l'environnement de votre application pour qu'il soit possible de la générer de n'importe où
    - à l'aide de Dockerfile
    - à l'aide d'image officielle
- Vous définissez vos services dans un fichier *dockercompose.yml* pour les exécuter et les isoler.
- Exécutez docker-compose qui se chargera d'exécuter l'ensemble de votre application

DOCKER-COMPOSE

Qu'est ce que Docker Compose?

- Un outil complémentaire du Docker Engine
  - présenté dans les versions récentes comme un plugin de Docker Engine
  - Ancienne syntaxe: docker-compose command
  - Nouvelle syntaxe: docker compose command
- Permet de décrire une application construite à base de conteneurs dans un fichier YAML
- Docker permet d'implémenter des micro-services, cela signifie que les conteneurs se limitent souvent à une tâche simple.

DOCKER-COMPOSE

Les principes du Docker-compose

- L'utilisateur décrit son application (multi) conteneurs dans un fichier YAML appelé *dockercompose.yml*
- Exécuter *docker-compose up* pour démarrer l'application

  Compose télécharge automatique les images, les build (si nécessaire), et démarre les conteneurs

  Compose peut configurer des volumes, le réseau, et toutes autres options liées à Docker

Exemple du fichier docker-compose.yml

```yaml
version: "3.9"
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - .:/code
      - logvolume01:/var/log
    depends_on:
      - redis
  redis:
    image: redis
volumes:
  logvolume01:
```

Exemple du fichier docker-compose.yml

- 2 services: web et redis
  - ✓ Par défaut, Compose créé un réseau bridge indépendant du réseau par défaut
  - ✓ La découverte de service fonctionne sur ce réseau

    web peut contacter redis en utilisant son nom
- Un conteneur pour chaque service est créé
  - ✓ Pour web, une image est d'abord recréé à partir du Dockerfile présent dans le répertoire courant
  - ✓ Pour redis, l'image redis est récupérée depuis Docker Hub
- Configuration du réseau
  - ✓ Le port 5000 de la machine hote est associé au port 5000 du conteneur web

Exemple 2 du fichier docker-compose.yml

```
version: '2'
services:
  db:
    image: mysql:5.7
    volumes:
      - "./.data/db:/var/lib/mysql"
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: wordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress

  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    links:
```

Exemple 2 du fichier docker-compose.yml :

Démarrage d'une application

```
$ docker-compose up -d
Creating wordpress_db_1
Creating wordpress_wordpress_1
```

Les différents services qui composent mon application ont été démarrés, avec la configuration et l'environnement qui va bien.

Information de mon application

On utilise la commande ps de Compose:

```
$ docker-compose ps
       Name                     Command               State          Ports
-------------------------------------------------------------------------------
wordpress_db_1          docker-entrypoint.sh mysqld      Up      3306/tcp
wordpress_wordpress_1   /entrypoint.sh apache2-for ...   Up      0.0.0.0:8000->80/tcp

$ docker-compose ps wordpress
       Name                     Command               State          Ports
-------------------------------------------------------------------------------
wordpress_wordpress_1   /entrypoint.sh apache2-for ...   Up      0.0.0.0:8000->80/tcp
```

Exemple 2 du fichier docker-compose.yml :

Conteneurs Classiques

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS
8d086aeba614        wordpress:latest    "/entrypoint.sh apach"   7 minutes ago       Up 7 minutes
689405bb755d        mysql:5.7           "docker-entrypoint.sh"   7 minutes ago       Up 7 minutes
```

Les services s'exécutent via des conteneurs sur l'hôte. Les commandes docker classiques sont toujours fonctionnelles.

Passage à l'échelle

On peut passer à l'échelle un service. Autrement dit, on peut augmenter/diminuer le nombre de conteneurs exécutant un service. Par défaut, Compose exécute chaque service avec un conteneur.
On utilise la commande scale pour changer le nombre de réplicas d'un service:

```
$ docker-compose scale db=3
$ docker-compose ps db
    Name                  Command            State    Ports
-------------------------------------------------------------
wordpress_db_1   docker-entrypoint.sh mysqld   Up      3306/tcp
wordpress_db_2   docker-entrypoint.sh mysqld   Up      3306/tcp
wordpress_db_3   docker-entrypoint.sh mysqld   Up      3306/tcp
$ docker-compose scale db=2
```

DOCKER COMPOSE : COMMANDES UTILES

Commande	Rôle
`docker compose up -d`	démarre toute la pile en arrière-plan
`docker compose ps` / `logs -f`	état des services / suivre les journaux
`docker compose exec service sh`	ouvrir un shell dans un service
`docker compose stop` / `down`	arrêter / arrêter **et supprimer** conteneurs et réseau
`docker compose down -v`	supprime aussi les volumes : **les données sont perdues**
`docker compose up -d --scale web=3`	3 conteneurs pour le service `web`

- **Syntaxe :** `docker compose` (plugin) remplace `docker-compose` ; la clé `version:` est obsolète ; `scale` est remplacé par `up --scale`.
- **Secrets :** variables dans un fichier `.env`, pas de mot de passe en clair dans le YAML.

A RETENIR

- Compose est un outil pour définir, lancer et gérer des services qui sont définis comme une ou plusieurs instances d'un conteneur,
- Compose utilise un fichier de configuration YAML comme définition de l'environnement,
- Avec docker-compose on peut générer des images, lancer et gérer des services, …
- Certaines commandes de docker-compose sont équivalentes à l'outil docker, mais s'appliquent seulement aux conteneurs de la configuration de compose.

QUESTIONS DE RÉVISION : DOCKER

Q1. Quelle différence entre une image et un conteneur ?

Q2. Pourquoi un conteneur lancé avec `-d` et `/bin/bash` s'arrête-t-il aussitôt ?

Q3. Que signifie `-p 8080:80` ?

Q4. Comment conserver les données d'une base MySQL après suppression du conteneur ?

Q5. Quel est l'intérêt d'un build multi-étapes ?

LES KUBERNETES

Problématique

Comment gérer des containers qui sont dispersés dans un grand nombre de machines ?
Dans le cas où il y a un container qui pose un problème, comment le détecter ?
Lequel aurait besoin de plus de réplications pour subvenir à une subite augmentation de la charge ?

Solution

L'équipe opérationnelle monitore les serveurs et si un de ces derniers tombe en panne, il devra créer de nouveaux containers avec les services déchus dans un autre serveur.

Solution automatisée

Kubernetes permet justement d'automatiser le déploiement, la gestion de demande de puissance et la gestion des applications containerisées.

LES KUBERNETES

- Kubernetes est un orchestrateur de multiples conteneurs notamment des conteneurs dockers.
- Sa responsabilité est de surveiller les différentes instances de micro containers qu'il supervise et d'en réguler la réplication sur différentes machines en fonction de la montée en charge des requêtes.
- Maintenir la haute disponibilité (maintenir les conteneurs up et assurer l'état des services qui ont été décrits).

LES KUBERNETES

- **Docker** est une technologie d'exécution de conteneurs qui vous permet de ***créer, tester et déployer des applications*** plus rapidement qu'avec des méthodes traditionnelles.
- **Kubernetes** est un outil d'orchestration de conteneurs qui vous permet ***de mettre à l'échelle vos systèmes de conteneurs*** afin que vous puissiez gérer, coordonner et planifier les conteneurs à grande échelle.

ARCHITECTURE : CONTROL PLANE ET NŒUDS DE TRAVAIL

- **Control plane (plan de contrôle) :** gère le cluster ; décide où placer les pods et réagit aux pannes.
- **Worker nodes (nœuds de travail) :** fournissent la capacité de calcul et exécutent les pods.
- **Pas d'ordres entre nœuds :** tout passe par l'API server du control plane.
- **Vocabulaire :** les anciens termes « master », « maître-esclave » et « minions » ont été remplacés par control plane et worker node.

L'architecture de Kubernetes

- **Volumes (persistent ou non persistent) :**

  Ils constituent les lieux d'échange entre les pods

  Persistent dans ce cas on va les stocker à l'extérieur des pods

  Non persistent à l'intérieur des pods.

- **Déploiements :**

  Gestion de création/suppression des pods

  Gestion des nom de réplicas (les réplicas sets)

  Assurer du respect des descriptions des relations entre conteneurs

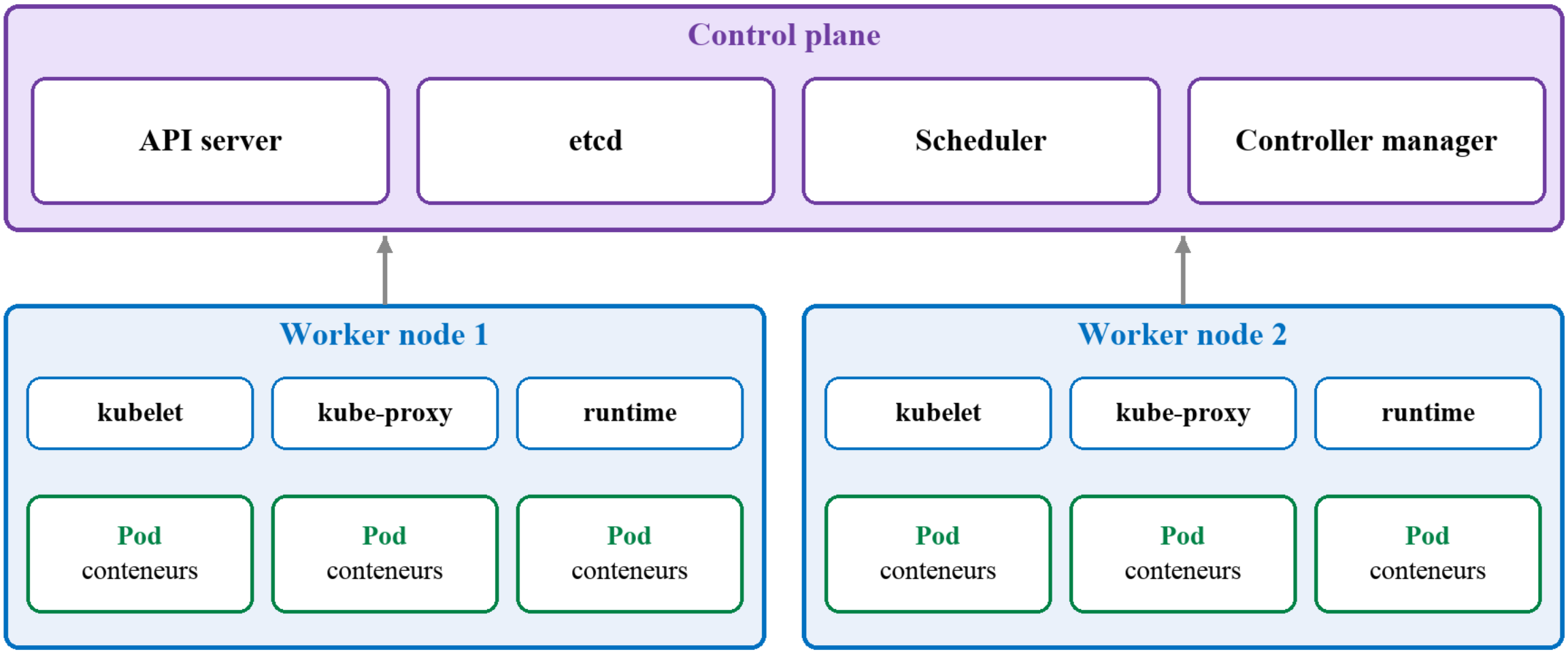
- **Namespaces :**

  Cluster virtuel (ensemble de services) à l'intérieur de KBS

  Cloisonner à l'intérieur de notre cluster pour des services qui ne travaillent pas ensemble et avoir une cohérence des droits des utilisateurs pour accès aux services qui travaillent ensemble et gérer le cloisonnement si on a des services totalement différents.

  Segmenter les pods

CLUSTER KUBERNETES



LES PODS

- **Plus petite unité déployable :** un ou plusieurs conteneurs qui partagent la **même IP** et les mêmes volumes.
- **Kubernetes gère des pods,** pas directement des conteneurs.
- **Généralement :** un conteneur principal par pod (parfois des conteneurs auxiliaires, les « sidecars »).
- **Éphémère :** un pod disparu n'est pas réparé, il est **remplacé** par un nouveau (autre nom, autre IP).
- **Décrit** par un manifeste YAML.

Les notions et concepts de Kubernetes

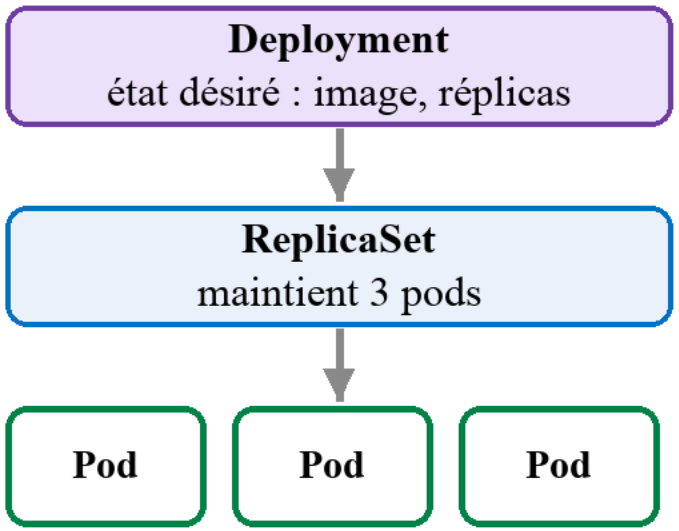
- **Services:**

  Le service se place au dessus des pods ce qui permet de faire une abstraction des pods.
  Le service évite la communication par IP comme dans le cas des docker (IP peut changer puisqu'on travaille avec les conteneurs).
  Un service est compose d'un couple IP + Port fixe , qui permet de communiquer avec des conteneurs

LES OBJETS KUBERNETES PRINCIPAUX

Objet	Rôle
Pod	exécute un ou plusieurs conteneurs
ReplicaSet	maintient un nombre de pods identiques
Deployment	gère les ReplicaSets : réplicas, mises à jour, retour arrière
Service	adresse stable et répartition de charge devant des pods
Namespace	cluster virtuel pour séparer équipes ou environnements
ConfigMap / Secret	configuration / données sensibles
PersistentVolume / Claim	stockage qui survit aux pods
Ingress	routage HTTP(S) vers plusieurs services

DEPLOYMENT ET REPLICASET



- **Deployment :** on déclare l'état **désiré** (image, nombre de réplicas).
- **ReplicaSet :** garde le bon nombre de pods : un pod supprimé est **recréé** (auto-réparation).
- **Mise à l'échelle :**
  `kubectl scale --replicas=3 deployment/app`
  .
- **Mises à jour :** progressives, avec retour arrière possible.

MANIFESTE YAML : ANATOMIE D'UN DEPLOYMENT

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloworld
  labels:
    app: helloworld
spec:
  replicas: 3
  selector:
    matchLabels:
      app: helloworld
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: helloworld
        image: particule/helloworld
        ports:
        - containerPort: 80
```

- **`apiVersion`, `kind` :** type d'objet.
- **`metadata` :** nom et labels.
- **`spec` :** état désiré.
- **`selector` ↔ `template.labels` :** lient le Deployment à ses pods.
- **Appliquer :** **`kubectl apply -f fichier.yaml`**.
- **Indentation :** espaces uniquement, jamais de tabulations.

SERVICES : EXPOSER LES PODS

- **Service :** adresse et nom **stables** devant un groupe de pods choisis par labels ; répartit la charge.

Type	Accès
`ClusterIP` **(défaut)**	uniquement à l'intérieur du cluster
`NodePort`	port élevé (30000-32767) ouvert sur chaque nœud (TP7)
`LoadBalancer`	équilibreur de charge fourni par le cloud
Ingress (objet distinct)	routage HTTP(S) vers plusieurs services par hôte ou chemin

- **Créer :**
  `kubectl expose deployment app --port=80 --type=NodePort`.

CONFIGURATION ET DONNÉES

- **ConfigMap :** configuration non sensible (variables, fichiers).
- **Secret :** données sensibles. **Encodées en base64, pas chiffrées par défaut** : restreindre l'accès (RBAC), activer le chiffrement.
- **Volumes persistants :** `PersistentVolume` et `PersistentVolumeClaim` gardent les données au-delà de la vie d'un pod.
- **Namespace :** sépare logiquement équipes ou environnements.

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```

KUBECTL : COMMANDES ESSENTIELLES

Commande	Rôle
`kubectl get pods|deploy|svc|nodes`	lister des objets
`kubectl describe pod nom`	détail et événements d'un objet
`kubectl apply -f fichier.yaml`	créer ou mettre à jour depuis un manifeste
`kubectl delete -f fichier.yaml`	supprimer les objets d'un manifeste (ou `delete pod nom`)
`kubectl logs nom`	journaux d'un pod
`kubectl exec -it nom -- sh`	ouvrir un shell dans un pod
`kubectl scale --replicas=3 deployment/app`	changer le nombre de réplicas
`kubectl rollout status|undo deployment/app`	suivre / annuler une mise à jour
`kubectl port-forward svc/app 8080:80`	accéder à un service depuis sa machine

MISES À JOUR PROGRESSIVES ET RETOUR ARRIÈRE

```
$ kubectl set image deployment/app app=monuser/app:2.0
$ kubectl rollout status deployment/app      # suivre la mise à jour
$ kubectl rollout undo deployment/app        # revenir à la version précédente
```

- **Rolling update :** les pods sont remplacés **progressivement** ; le service continue de répondre.
- **Retour arrière : `rollout undo`**, si la nouvelle version pose problème.

SANTÉ ET RESSOURCES : PROBES, REQUESTS ET LIMITS

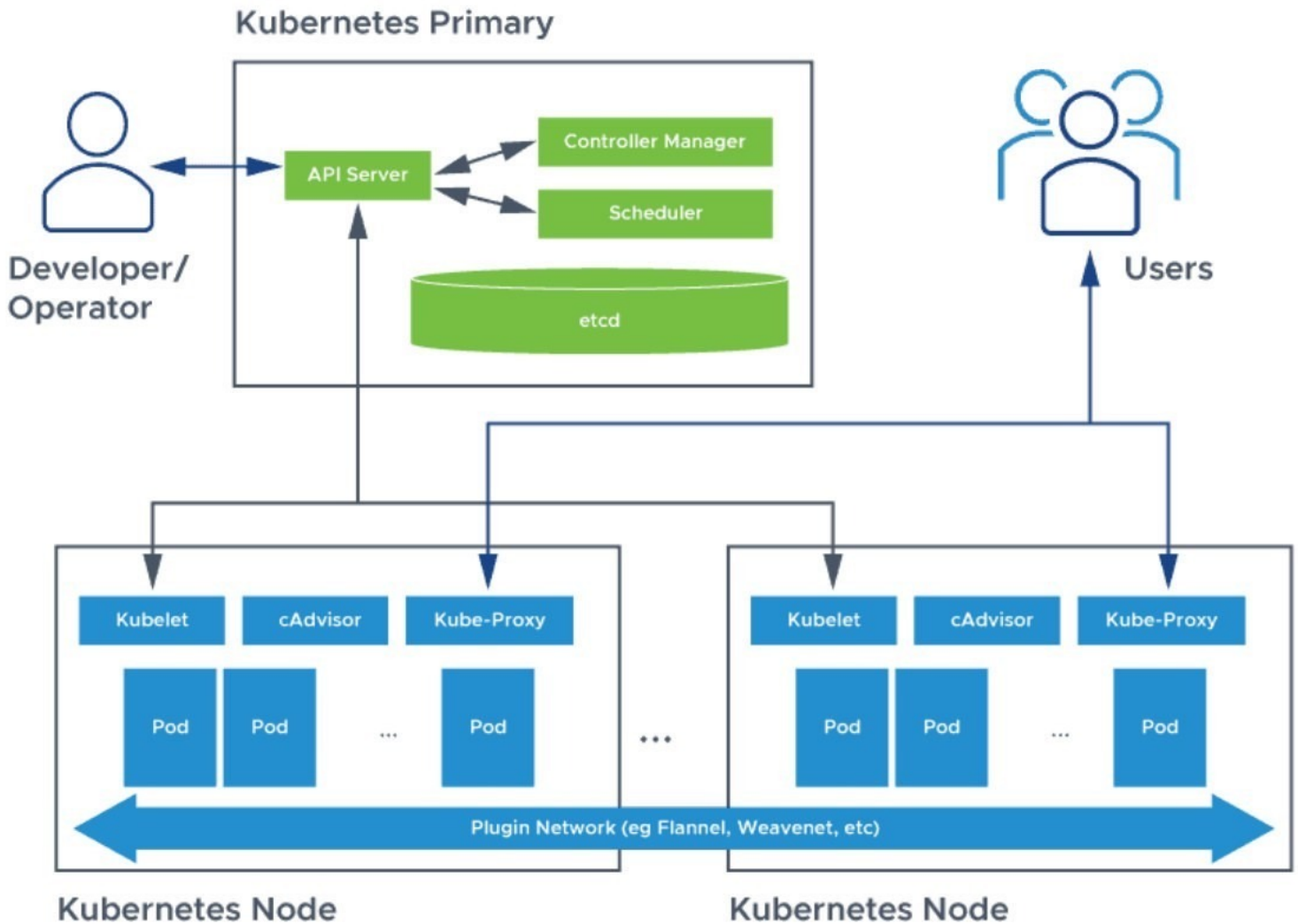
```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 80
readinessProbe:
  httpGet:
    path: /ready
    port: 80
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

- **liveness :** l'application est-elle vivante ? Sinon, le conteneur est **redémarré**.
- **readiness :** est-elle prête ? Sinon, le pod est **retiré du service**.
- **requests :** ressources réservées, utilisées par le scheduler.
- **limits :** plafond ; au-delà de la mémoire autorisée, le conteneur est arrêté.

RÔLE DU CONTROL PLANE

- **État désiré :** vous déclarez (par exemple 3 réplicas), le control plane fait converger l'état réel vers cet état.
- **Placement :** il décide sur quel nœud chaque pod s'exécute.
- **Pannes :** un pod ne « migre » pas : les contrôleurs créent de **nouveaux pods** sur des nœuds sains.
- **Haute disponibilité :** plusieurs instances du control plane.

Architecture du Kubernetes

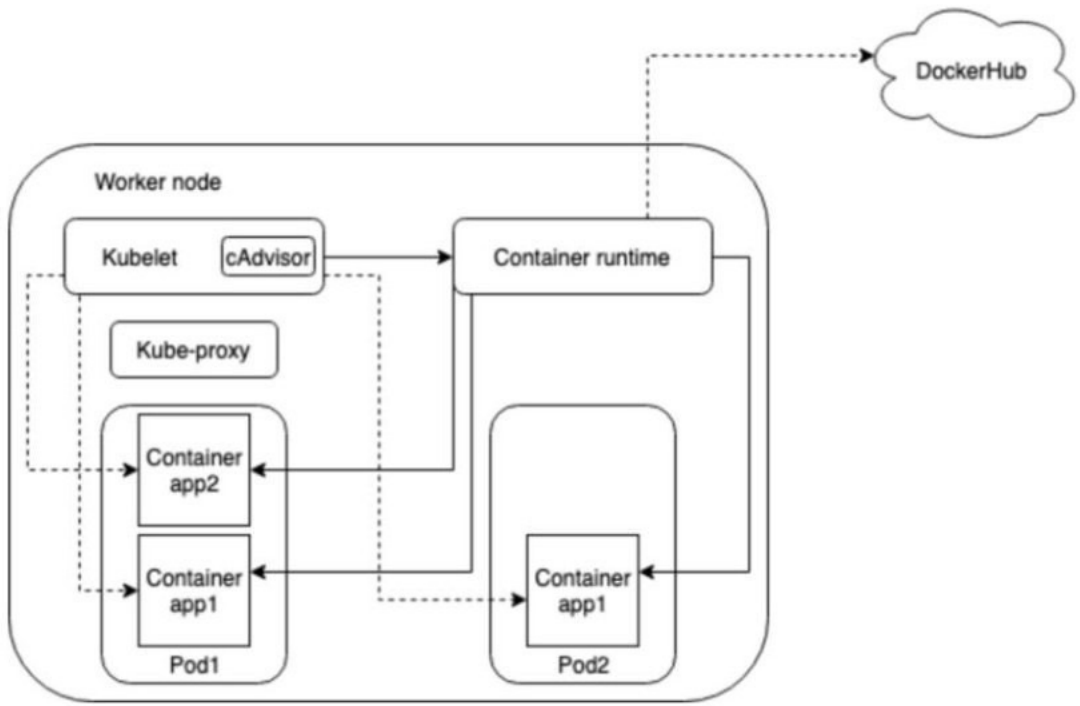


COMPOSANTS DU CONTROL PLANE

Composant	Rôle
`etcd`	base clé-valeur distribuée et persistante : stocke l'état du cluster
`kube-apiserver`	porte d'entrée : API REST par laquelle passent toutes les communications (dont `kubectl`)
`kube-scheduler`	choisit le nœud où placer chaque nouveau pod (ressources, contraintes)
`kube-controller-manager`	contrôleurs qui maintiennent l'état désiré (réplicas, nœuds...)

Worker Node

Un worker node (WN) est une machine physique ou une VM qui détient toutes les ressources nécessaires afin de garantir l'exécution d'un ou plusieurs pods. Cette entité va héberger tous les services qu'un développeur aura décidé de déployer.



WORKER NODE : MOTEUR DE CONTENEURS

- **Machine physique ou virtuelle** qui exécute les pods.
- **Moteur de conteneurs : `containerd`** ou **`CRI-O`** (interface CRI).
- **Docker Engine n'est plus requis :** le support direct (dockershim) a été retiré dans Kubernetes 1.24.
- **Vos images Docker** fonctionnent toujours (standard OCI).
- **TP7 :** Minikube utilise Docker comme driver pour créer le nœud local.

COMPOSANTS D'UN WORKER NODE

Composant	Rôle
`kubelet`	agent du nœud : vérifie que les conteneurs des pods assignés tournent et sont sains ; rend compte au control plane
`kube-proxy`	règles réseau : achemine le trafic des services vers les bons pods
Moteur de conteneurs	`containerd` ou `CRI-O` : exécute les conteneurs
cAdvisor (intégré au kubelet)	collecte CPU, mémoire, disque, réseau ; métriques lues par metrics-server ou Prometheus

QUESTIONS DE RÉVISION : KUBERNETES

Q1. Quelle différence entre un Pod et un Deployment ?

Q2. Que se passe-t-il si on supprime un pod géré par un Deployment ? Pourquoi ?

Q3. À quoi sert un Service ? Citez deux types.

Q4. Quels sont les rôles de l'API server, d'etcd et du scheduler ?

Q5. Comment mettre à jour une application sans interruption, et comment revenir en arrière ?

CI/CD et pipelines

Automatiser le chemin du code vers la production

POURQUOI UN PIPELINE ?

- **Sans automatisation :** récupérer le code, compiler, tester, construire, copier, redémarrer... à la main : lent, oublis, pratiques différentes.
- **Pipeline :** suite d'étapes **automatisées**, déclenchée par un événement (push, Pull Request, tag, planification).
- **Pipeline as code :** décrit dans un fichier versionné avec le code.
- **Résultat :** chaque modification suit le même chemin, de façon **reproductible**.

LES ÉTAPES D'UN PIPELINE

Étape	Ce que l'on fait	Exemples
Build	installer les dépendances, compiler	`npm ci`, `mvn package`
Test	tests unitaires, style, analyse statique	`npm test`, `pytest`, linter
Package	produire le livrable	image Docker, archive
Publish	stocker le livrable	push vers un registre d'images
Deploy (staging)	déployer en environnement de test	`kubectl apply`, Compose
Tests d'acceptation	vérifier le besoin, la performance	tests end-to-end, de charge
Deploy (production)	mise en production	manuelle (livraison) ou automatique (déploiement)

ANATOMIE D'UN PIPELINE

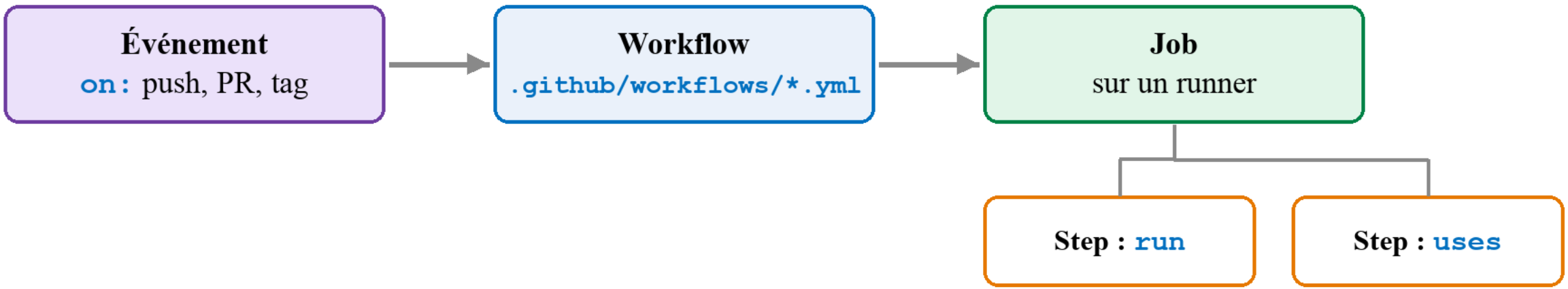
Terme	Signification
Déclencheur (trigger)	événement qui lance le pipeline : push, Pull Request, tag, planification
Job	groupe d'étapes exécutées sur la même machine ; en parallèle ou à la suite
Étape (step)	une commande ou une action réutilisable
Runner / agent	machine (virtuelle ou conteneur) qui exécute le job
Artefact	fichier produit par un job et transmis à un autre
Cache	réutilise les dépendances pour accélérer
Variables, secrets	configuration ; données sensibles masquées
Environnement	cible de déploiement : dev, staging, production

LES OUTILS DE CI/CD

Outil	Fichier de configuration	Particularité
Jenkins	`Jenkinsfile`	auto-hébergé, très extensible (plugins)
GitHub Actions	`.github/workflows/*.yml`	intégré à GitHub, nombreuses actions prêtes à l'emploi
GitLab CI/CD	`.gitlab-ci.yml`	intégré à GitLab
Azure Pipelines	`azure-pipelines.yml`	écosystème Microsoft
CircleCI	`.circleci/config.yml`	service cloud dédié au CI/CD

> Mêmes concepts partout : déclencheur, jobs, étapes, runners. Seule la syntaxe change.

GITHUB ACTIONS : VOCABULAIRE



- **Workflow :** fichier YAML dans `.github/workflows/`.
- **Événement :** ce qui le déclenche (`on:`).
- **Job :** suite d'étapes sur un **runner** (`ubuntu-latest`).
- **Step :** une commande (`run`) ou une action réutilisable (`uses`), comme `actions/checkout`.

EXEMPLE : WORKFLOW D'INTÉGRATION CONTINUE

```yaml
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test
```

- **on :** push sur **main** et Pull Request.
- **runs-on :** le runner.
- **uses :** action réutilisable (récupérer le code, installer Node).
- **run :** commande shell.
- **Échec** d'une commande = job en échec = workflow rouge.

LA CI DANS LA PULL REQUEST

- **Contrôle automatique :** le workflow se déclenche sur la Pull Request ; le résultat (coche verte ou croix rouge) s'affiche dedans.
- **Protection de branche :** exiger des contrôles verts **et** une relecture avant de fusionner dans `main`.
- **Résultat :** du code qui casse les tests ne peut pas arriver dans la branche principale.
- **Journaux :** onglet **Actions** du dépôt ; un job peut être relancé.
- **Badge :** affiche l'état du build dans le `README`.

EXEMPLE : CONSTRUIRE ET PUBLIER UNE IMAGE DOCKER

```yaml
name: Docker
on:
  push:
    tags: [ 'v*' ]
jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USER }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: monuser/monapp:${{ github.ref_name }}
```

- **Déclencheur :** un tag `v1.0.0`.
- `login-action` : connexion au registre.
- `build-push-action` : construit l'image et la publie.
- **Secrets :** identifiants lus dans les **secrets** du dépôt, jamais écrits dans le YAML.

SECRETS ET VARIABLES DANS UN PIPELINE

- **Jamais** dans le dépôt, ni dans le fichier YAML : ils resteraient dans l'historique Git.
- **Où :** paramètres du dépôt, **Secrets and variables** ; lus avec `${{ secrets.NOM }}` et masqués dans les journaux.
- **Variables :** pour les valeurs non sensibles.
- **Moindre privilège :** jeton d'accès à droits limités plutôt que mot de passe ; rotation régulière.
- **Fuite :** révoquer le secret immédiatement.

DU CI AU CD : LES ENVIRONNEMENTS

- **Construire une seule fois,** promouvoir le **même artefact** d'un environnement à l'autre.
- **Livraison continue :** approbation manuelle avant la production (GitHub : reviewers requis sur l'environnement).
- **Déploiement continu :** aucune validation, la production suit automatiquement.

STRATÉGIES DE DÉPLOIEMENT

Stratégie	Principe	Point clé
Recreate	arrêt de l'ancienne version, puis démarrage de la nouvelle	simple, mais **coupure** de service
Rolling update	remplacement progressif des instances	sans interruption ; défaut de Kubernetes
Blue/green	deux environnements ; bascule du trafic de l'un à l'autre	retour arrière instantané ; ressources doublées
Canary	la nouvelle version reçoit d'abord une petite part du trafic	risque limité ; progression graduelle

> Les feature flags permettent d'activer une fonctionnalité sans redéployer.

DÉPLOYER SUR KUBERNETES DEPUIS LE PIPELINE

```yaml
deploy:
  needs: docker                          # après le job qui publie l'image
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: kubectl apply -f k8s/         # accès au cluster : secret (kubeconfig)
    - run: kubectl rollout status deployment/app
```

- **Tag d'image :** précis (hash du commit), pas `latest`.
- **GitOps :** Argo CD ou Flux synchronisent le cluster avec un dépôt Git, **source de vérité** du déploiement.

BONNES PRATIQUES DE PIPELINE

- **Rapide :** cache des dépendances, jobs en parallèle, tests lents en dernier.
- **Échouer vite :** les contrôles peu coûteux d'abord.
- **Reproductible :** versions fixées, mêmes conteneurs.
- **Construire une fois,** promouvoir l'artefact.
- **Protéger `main` :** contrôles obligatoires ; protéger les secrets.
- **Réparer tout de suite** un build cassé.
- **Tests fiables :** des tests instables détruisent la confiance.

MESURER : LES MÉTRIQUES DORA

Fréquence de déploiement
à quelle fréquence met-on en production ?

Délai de mise en production
temps entre un commit et son déploiement

Taux d'échec des changements
part des déploiements qui provoquent un incident

Temps de rétablissement
durée pour réparer après un incident

Vitesse (bleu) et stabilité (orange)

Les meilleures équipes sont bonnes sur les deux : vitesse et stabilité ne s'opposent pas.

On mesure le système, pas les personnes.

À RETENIR : CI/CD

- **CI :** intégrer souvent, build et tests automatiques à chaque modification.
- **Livraison continue :** chaque version validée est prête ; production sur décision humaine.
- **Déploiement continu :** production automatique après les tests.
- **Pipeline as code :** étapes décrites dans un fichier versionné.
- **Secrets :** jamais dans le dépôt.
- **Mesurer :** métriques DORA.

QUESTIONS DE RÉVISION : CI/CD

Q1. Quelle différence entre livraison continue et déploiement continu ?

Q2. Citez les éléments d'un workflow GitHub Actions.

Q3. Où stocker un mot de passe utilisé par un pipeline ?

Q4. Pourquoi construire l'artefact une seule fois et le promouvoir ?

Q5. Citez trois stratégies de déploiement.

INFRASTRUCTURE AS CODE (IAC)

- **Principe :** décrire l'infrastructure (serveurs, réseaux, conteneurs) dans des fichiers **versionnés**.
- **Avantages :** reproductible, relu en Pull Request, testable, documentation à jour.
- **Déclaratif** (on décrit l'état voulu : Terraform, Compose, Kubernetes) ou impératif (on décrit les étapes).
- **Idempotence :** relancer l'outil donne le même résultat.
- **Outils :** **Terraform** / OpenTofu (provisionner), **Ansible** (configurer), Pulumi, CloudFormation.
- **Déjà vu :** `docker-compose.yml` et les manifestes Kubernetes sont de l'IaC.

IAC : EXEMPLES TERRAFORM ET ANSIBLE

Terraform

```
resource "docker_image" "nginx" {
  name = "nginx:1.27"
}

resource "docker_container" "web" {
  name  = "web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = 8080
  }
}
```

Ansible

```
- hosts: web
  become: true
  tasks:
    - name: Installer nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Démarrer nginx
      ansible.builtin.service:
        name: nginx
        state: started
```

- **Complémentaires :** Terraform **crée** l'infrastructure, Ansible **configure** ce qui tourne dessus.

SUPERVISION ET OBSERVABILITÉ

Métriques

- Valeurs chiffrées dans le temps
- Prometheus, Grafana

Logs

- Journaux d'événements
- ELK, Loki

Traces

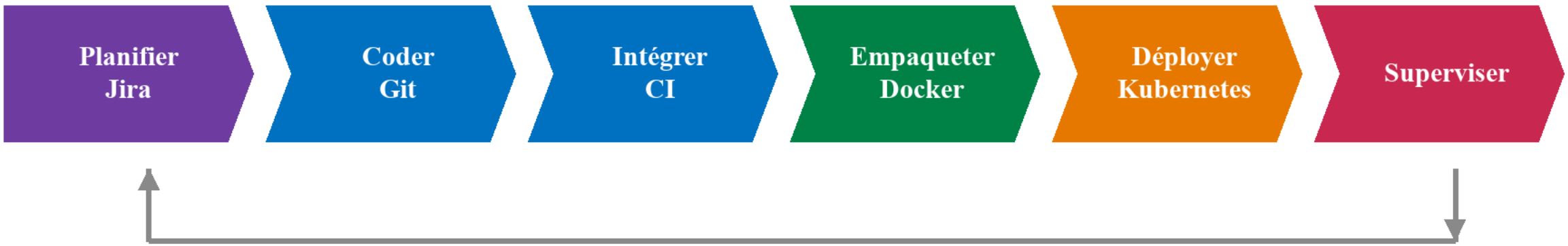
- Suivre une requête à travers les services
- OpenTelemetry, Jaeger

- **Alertes :** être prévenu d'un incident (Alertmanager...).
- **4 signaux :** latence, trafic, erreurs, saturation.
- **Boucle de retour :** ce que l'on observe alimente la planification (pilier **mesure**).

DEVSECOPS : INTÉGRER LA SÉCURITÉ

- **Shift left :** la sécurité dès le début, dans le pipeline, pas à la fin.
- **Étapes automatiques :** détection de secrets, analyse statique (CodeQL), dépendances vulnérables (Dependabot), scan d'images (Trivy).
- **Bonnes pratiques :** moindre privilège, images minimales, mises à jour régulières.

SYNTHÈSE : LA BOUCLE DEVOPS



Les outils automatisent ; la culture (collaboration, responsabilité partagée, amélioration continue) fait la différence.

POUR ALLER PLUS LOIN

- **Git :** livre Pro Git (git-scm.com/book), gratuit et disponible en français.
- **Docker :** documentation officielle (docs.docker.com).
- **Kubernetes :** documentation officielle (kubernetes.io/docs).
- **GitHub Actions :** docs.github.com/actions.
- **Agilité :** Atlassian Agile Coach (atlassian.com/agile).
- **Métriques :** DORA (dora.dev).
- **Applications cloud :** The Twelve-Factor App (12factor.net).
- **Lectures :** Accelerate (Forsgren, Humble, Kim) ; The Phoenix Project.